Lecture 3: Digital Watermarking

Slava Voloshynovskiy

May 15, 2025

Table of Contents

Section 1

Main Applications

3.1 Main Applications of Watermarking

Overview: Digital watermarking serves various purposes across multiple domains.

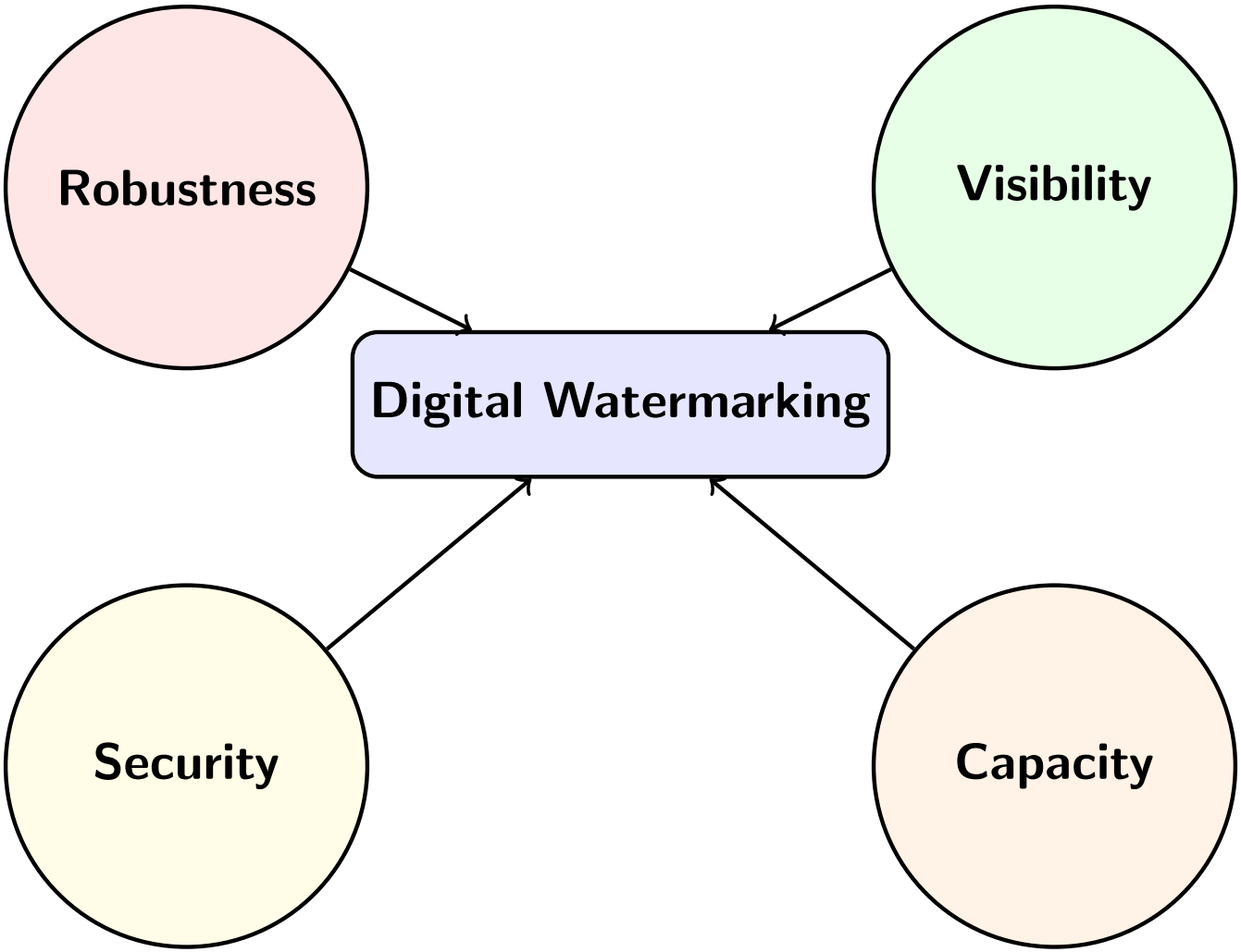
- **Robust Watermarking:**
  - Copyright protection.
  - Distinguishing real vs synthetic data.
  - Broadcast monitoring, tracking, and tracing solutions.
- **Steganography:**
  - Enables secure communication without revealing the existence of hidden information.
- **Tamper Detection:**
  - Verifies the integrity of digital content.
  - Used for digital image authentication and tampering detection.

3.1 Requirements and Challenges

Key Requirements and Challenges:

- Different techniques prioritize different requirements:
  - Robustness (in robust watermarking).
  - Statistical non-detectability (in steganography).
  - Detectability of modifications (in tamper detection).
- Attacker knowledge varies:
  - Known or unknown watermark embedding methods.
  - Known or unknown key management strategies.
- **Common Element:** All techniques involve **data hiding**—modifying content invisibly to hide information:
  - In robust watermarking: Focus on perceptual invisibility and robustness.
  - In steganography: Focus on non-detectability and statistical invisibility.

3.1 Trade-Off in Digital Watermarking



3.1 Focus on Robust Digital Watermarking

Why Focus on Robust Digital Watermarking?

- Despite their differences, watermarking, steganography, and tamper detection share common principles.
- Due to the limited scope of this class, we will primarily focus on:
  - Foundations and building blocks of **robust digital watermarking**.
- Once these foundations are covered, we will extend the discussion to:
  - **Steganographic communications.**
  - **Tamper detection techniques.**

Section 2

Foundation of Robust Digital Watermarking

3.2 Foundation of Robust Digital Watermarking

Introduction:

- This part will focus on the **fundamental principles of robust digital watermarking**.
- **Key Assumptions at this Stage:**

Assumptions

- **Pixel domain embedding without perceptual masking.**
- **Only image processing degradations:**
  - JPEG compression.
  - Noise addition.
  - Intensity changes.
- **No geometrical synchronization issues:**
  - No rotation.
  - No scaling, cropping, or shearing.

- **Future Scope:** Future sections will address:
  - Handling perceptual masking.
  - Addressing geometrical distortions.

3.2 Aspects of Digital Watermarking

Two main types of digital watermarking:

Zero-Bit Watermarking (Detection Problem)

- **Problem:** Determine if an image is watermarked using a given key.
- **Encoding function:** $\mathbf{y} = \phi(\mathbf{x}, \mathbf{k})$, where $\mathbf{k}$ is the secret key.
- **Detector:** $g_{\text{0-bit}}(\mathbf{v}, \mathbf{k}) \rightarrow \{0, 1\}$, where $\mathbf{v}$ is the distorted version of $\mathbf{y}$.

L-Bit Watermarking (Decoding Problem)

- **Problem:** Embed and retrieve L secret bits.
- **Encoding function:** $\mathbf{y} = \phi(\mathbf{x}, \mathbf{k}, \mathbf{m})$, where $\mathbf{m}$ is the secret message.
- **Decoder:** $\hat{\mathbf{m}} = g_{\text{L-bit}}(\mathbf{v}, \mathbf{k})$, where $\mathbf{v}$ is the distorted version of $\mathbf{y}$.

3.2 Distortion Metrics due to Watermark Embedding

Measuring Distortions:

Distortion Measure

- Distortion between the original image $\mathbf{x}$ and the watermarked image $\mathbf{y}$ is defined as:

$$D_{\mathbf{xy}} = \frac{1}{M}\|\mathbf{x} - \mathbf{y}\|_2^2, \quad \text{where} \quad M = H \times W \times C$$

Peak Signal-to-Noise Ratio (PSNR)

- The Peak Signal-to-Noise Ratio (PSNR) is computed as:

$$\text{PSNR} = 10 \log_{10} \left(\frac{255^2}{D_{\mathbf{xy}}} \right)$$

3.2 Distortion Metrics due to Attacks

Measuring Distortions:

Distortion Measure

- Distortion between the watermarked image $\mathbf{y}$ and the observed image $\mathbf{v}$ is defined as:

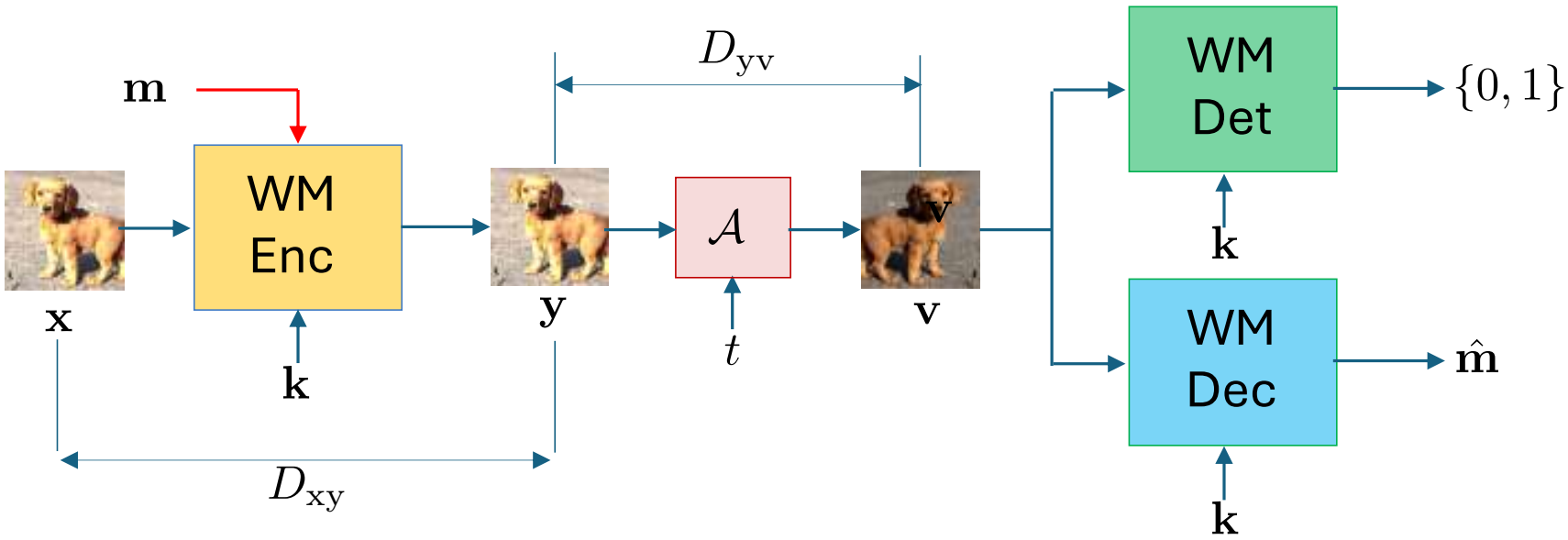
$$D_{\mathbf{yv}} = \frac{1}{M}\|\mathbf{y} - \mathbf{v}\|_2^2, \quad \text{where} \quad M = H \times W \times C$$

- This metric is applicable when no geometric transformations (e.g., rotation, scaling, or cropping) occur.

3.2 General Diagram of Digital Watermarking

The general process of digital watermarking includes:

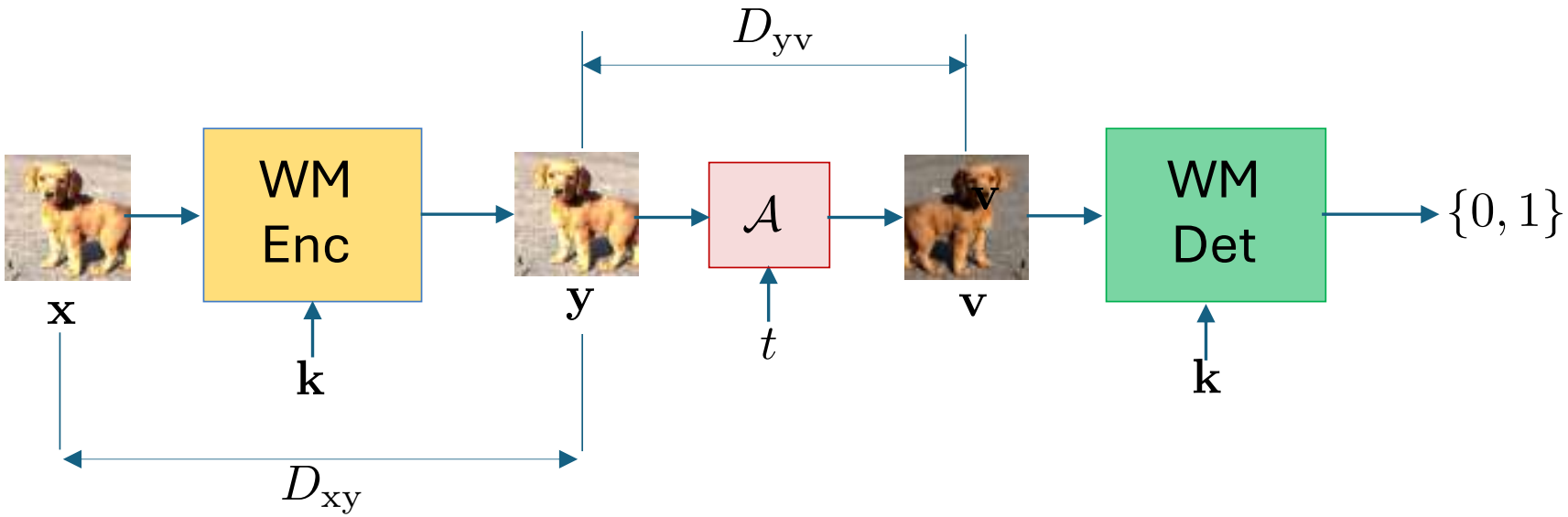
1. **WM encoding and embedding:** Generate and embed the watermark.
2. **Channel Degradation:** Apply distortions (noise, compression).
3. **Detection or Decoding:** Extract or detect the watermark.



Foundations of Digital Watermarking

0-Bit Watermarking or Detection Problem

3.2 Detection problem



3.2 Watermark Embedding Model

Watermark Embedding Equation

$$\mathbf{y} = \phi(\mathbf{x}, \mathbf{k}) = \mathbf{x} + \alpha\mathbf{w}, \quad \text{where} \quad \mathbf{w} = \text{Gen}(\mathbf{k})$$

- $\mathbf{y}$: Watermarked image.
- α: Strength of the watermark (we will assume $\alpha = 1$ for simplicity)
- $\mathbf{w}$: Watermark generated from the secret key $\mathbf{k}$ and having the same size as the image ($H \times W \times C$).

Key Insights

- Since the secret key $\mathbf{k}$ is known, the watermark $\mathbf{w}$ is deterministic and can be considered as a known signal for detection.
- The original image $\mathbf{x}$ is treated as a completely random signal, representing noise from the detector's perspective.

3.2 Original Hypothesis Formulation

Goal: Detect the presence of a watermark in a given image.

Hypotheses

Hypotheses:

- **Under H_0:**
  - **(a)** $\mathbf{v} = \mathbf{x}$ — no watermark present.
  - **(b)** $\mathbf{v} = \mathbf{x} + \mathbf{w}' + \mathbf{z}$ — incorrect watermark $\mathbf{w}'$ generated using key $\mathbf{k}' \neq \mathbf{k}$ and added distortion $\mathbf{z}$.
- **Under H_1:** $\mathbf{v} = \mathbf{x} + \mathbf{w} + \mathbf{z}$, where $\mathbf{w}$ is generated from the key $\mathbf{k}$.

Key Parameters:

- $\mathbf{x}$: Original image (we also use interchangeably $\mathbf{x}_0$).
- $\mathbf{k}$: Secret key used to generate the watermark.
- $\mathbf{w}$: Watermark signal.
- $\mathbf{z}$: Additive noise simulating distortion.

3.2 Simplified Hypothesis Formulation

Simplified Hypotheses:

Hypothesis Formulation after Aggregation

- **Under H_0:** $\mathbf{v} = \mathbf{x}$, where $\mathbf{x} = \mathbf{x}_0 + \mathbf{z}$ (random signal combining the original image and distortion).
- **Under H_1:** $\mathbf{v} = \mathbf{x} + \mathbf{w}$.

Justification

- Since both $\mathbf{x}_0$ (original image) and $\mathbf{z}$ (distortion) are random, they can be aggregated into a single random vector $\mathbf{x} = \mathbf{x}_0 + \mathbf{z}$.
- This simplification allows us to reduce the composite hypothesis H_0 into a single form where $\mathbf{v}$ is only compared to $\mathbf{x}$.

3.2 Hypothesis Setup

Hypotheses:

Hypothesis Formulation

- Under H_0: $\mathbf{v} = \mathbf{x}, \quad \mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}_\mathbf{x}, \mathbf{K}_{\mathbf{xx}})$
- Under H_1: $\mathbf{v} = \mathbf{x} + \mathbf{w}$, $\mathbf{w}$ is deterministic

Important Assumptions:

- Mean vector $\boldsymbol{\mu}_\mathbf{x}$ might play a critical role in detection as interference.
- Covariance matrix $\mathbf{K}_{\mathbf{xx}}$ is assumed diagonal ($\sigma_x^2\mathbf{I}$) for simplicity, but real-world scenarios may involve non-diagonal $\mathbf{K}_{\mathbf{xx}}$.
- The watermark $\mathbf{w}$ is **deterministic**, generated from a secret key.
- The watermark $\mathbf{w}$ has a **bounded** L_2-norm:

$$\|\mathbf{w}\|_2^2 = \text{defined constant}$$

Detection Score

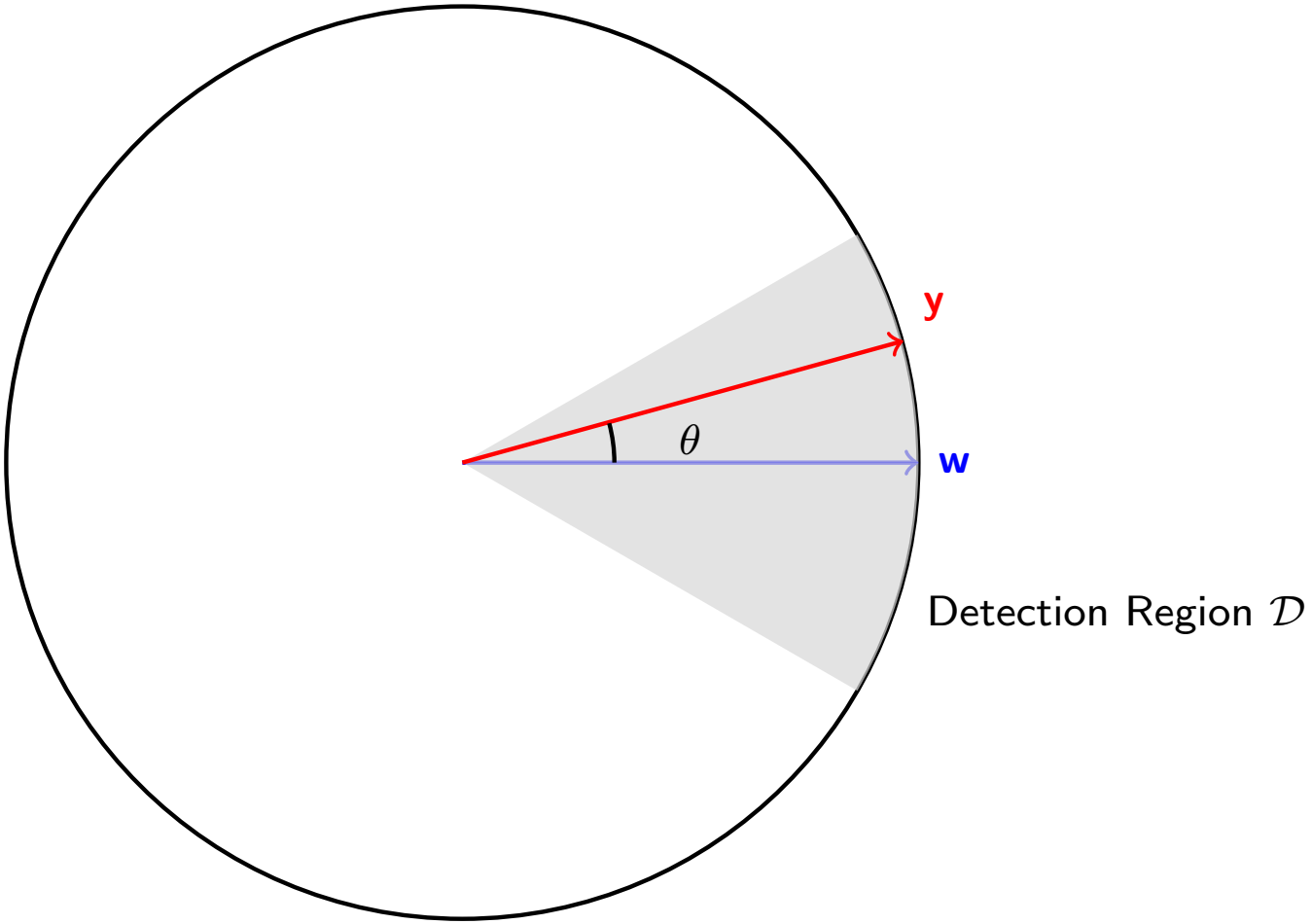
Score Definition (see Lecture 2)

$$S(\mathbf{v}) = \frac{1}{M}\mathbf{v}^\top \mathbf{w}, \quad M = H \cdot W \cdot C$$

Key Points:

- The score measures correlation between the observed signal and the watermark.
- Normalization by M ensures the score is independent of image size.

Inner Product Test for Detection



Explanation: The inner product test compares the angle between the observed vector **y** and the watermark **w**. If the angle θ is within the predefined threshold (gray detection region around **w**), **y** is considered to carry the watermark.

Inner Product Test and Cosine Similarity

Key Formulas:

Cosine Similarity

$$\cos\theta = \frac{\mathbf{y}^\top \mathbf{w}}{\|\mathbf{y}\|_2 \|\mathbf{w}\|_2}$$

Detection Score:

$$S(\mathbf{y}) = \mathbf{y}^\top \mathbf{w} = \cos\theta \cdot \|\mathbf{y}\|_2 \|\mathbf{w}\|_2$$

Explanation:

- The cosine similarity $\cos\theta$ measures how closely aligned vectors $\mathbf{y}$ and $\mathbf{w}$ are in terms of direction.
- The detection score $S(\mathbf{y})$ depends on both the norms of $\mathbf{y}$ and $\mathbf{w}$ and their angular proximity.

Decision Region Definition and Final Detection Rule

Decision Region Definition:

Decision Region

$$\mathcal{D} = \left\{ \mathbf{y} \in \mathbb{R}^M : \left| \mathbf{y}^\top \mathbf{w} \right| > \|\mathbf{y}\|_2 \|\mathbf{w}\|_2 \cos\theta \right\}$$

Explanation:

- A detection decision is made if the inner product $\mathbf{y}^\top \mathbf{w}$ exceeds the threshold defined by the product of the norms $\|\mathbf{y}\|_2 \|\mathbf{w}\|_2$ and the cosine similarity threshold $\cos\theta$.
- This ensures that the decision region $\mathcal{D}$ includes all vectors sufficiently aligned with $\mathbf{w}$ within a specific angular tolerance.

Main Results: Mean and Variance under H_0 and H_1

Results under H_0 and H_1

Under H_0:

$$\mu_0 = \frac{1}{M}\boldsymbol{\mu}_{\mathbf{x}}^{\top}\mathbf{w}, \quad \sigma_0^2 = \frac{\sigma_x^2\|\mathbf{w}\|_2^2}{M^2}$$

Under H_1:

$$\mu_1 = \frac{1}{M}\boldsymbol{\mu}_{\mathbf{x}}^{\top}\mathbf{w} + \frac{1}{M}\|\mathbf{w}\|_2^2, \quad \sigma_1^2 = \frac{\sigma_x^2\|\mathbf{w}\|_2^2}{M^2}$$

Key Points:

- Mean $\boldsymbol{\mu}_{\mathbf{x}}$ may interfere with detection but its impact decreases as image size grows since $\mathbb{E}[\boldsymbol{\mu}_{\mathbf{x}}^{\top}\mathbf{w}] \to 0$ for i.i.d. Gaussian $\mathbf{w}$.
- Higher variance σ_x^2 increases score variance under both hypotheses, causing stronger PDF overlap and complicating detection.

Error Probability Formulas

Error Probabilities:

- Probability of False Acceptance (P_{FA}):

$$P_{FA}(\gamma) = Q\left(\frac{\gamma - \mu_0}{\sigma_0}\right) \tag{1}$$

- Probability of Correct Detection (P_D):

$$P_D(\gamma) = Q\left(\frac{\gamma - \mu_1}{\sigma_1}\right) \tag{2}$$

- Probability of Miss (P_{miss}):

$$P_{\text{miss}}(\gamma) = 1 - P_D(\gamma) \tag{3}$$

3.2 Expected Value and Variance under H_0

Expected Value under H_0:

$$\mathbb{E}[S(\mathbf{v}) \mid H_0] = \mathbb{E}\left[\frac{1}{M}\mathbf{x}^\top \mathbf{w}\right]$$
$$= \frac{1}{M}\mathbb{E}[\mathbf{x}]^\top \mathbf{w} = \frac{1}{M}\boldsymbol{\mu}_\mathbf{x}^\top \mathbf{w}$$

Variance under H_0:

$$\mathrm{Var}(S(\mathbf{v}) \mid H_0) = \mathbb{E}\left[(S(\mathbf{v}) - \mathbb{E}[S(\mathbf{v}) \mid H_0])^2\right]$$
$$= \frac{1}{M^2}\|\mathbf{w}\|_2^2 \cdot \sigma_x^2$$

3.2 Detailed Derivation of Variance under H_0

Variance under H_0:

$$\begin{aligned}\text{Var}(S(\mathbf{v}) \mid H_0) &= \mathbb{E}\left[(S(\mathbf{v}) - \mathbb{E}[S(\mathbf{v}) \mid H_0])^2\right] \\ &= \mathbb{E}\left[\left(\frac{1}{M}\mathbf{x}^\top \mathbf{w} - \frac{1}{M}\boldsymbol{\mu}_\mathbf{x}^\top \mathbf{w}\right)^2\right] \\ &= \frac{1}{M^2}\mathbb{E}\left[\left((\mathbf{x} - \boldsymbol{\mu}_\mathbf{x})^\top \mathbf{w}\right)^2\right]\end{aligned}$$

Since $\mathbf{x}$ is a random vector with mean $\boldsymbol{\mu}_\mathbf{x}$ and covariance matrix $\mathbf{K}_{\mathbf{xx}} = \sigma_x^2 \mathbf{I}$, we can rewrite the expression as:

$$\mathbf{x} - \boldsymbol{\mu}_\mathbf{x} \sim \mathcal{N}(\mathbf{0}, \sigma_x^2 \mathbf{I})$$

Therefore, the random variable $(\mathbf{x} - \boldsymbol{\mu}_\mathbf{x})^\top \mathbf{w}$ is a linear combination of Gaussian variables, which itself follows a Gaussian distribution with variance given by:

$$\mathbb{E}\left[\left((\mathbf{x} - \boldsymbol{\mu}_\mathbf{x})^\top \mathbf{w}\right)^2\right] = \|\mathbf{w}\|_2^2 \cdot \sigma_x^2$$

Substituting this into the expression for the variance:

$$\text{Var}(S(\mathbf{v}) \mid H_0) = \frac{1}{M^2}\|\mathbf{w}\|_2^2 \cdot \sigma_x^2$$

3.2 Expected Value and Variance Result under H_1

Expected Value under H_1:

$$\begin{aligned}\mathbb{E}[S(\mathbf{v}) \mid H_1] &= \mathbb{E}\left[\frac{1}{M}(\mathbf{x}+\mathbf{w})^\top \mathbf{w}\right] \\ &= \frac{1}{M}\left(\mathbb{E}[\mathbf{x}]^\top \mathbf{w} + \mathbf{w}^\top \mathbf{w}\right) \\ &= \frac{1}{M}\boldsymbol{\mu}_{\mathbf{x}}^\top \mathbf{w} + \frac{1}{M}\|\mathbf{w}\|_2^2\end{aligned}$$

Variance under H_1 (Result Only):

$$\mathrm{Var}(S(\mathbf{v}) \mid H_1) = \frac{1}{M^2}\|\mathbf{w}\|_2^2 \sigma_x^2$$

Note: The detailed derivation of the variance is presented in the next slide.

3.2 Simplified Derivation of Variance under H_1 (Part 1)

Variance under H_1:
We use the formula for the variance:

$$\text{Var}(S(\mathbf{v}) \mid H_1) = \mathbb{E}\left[(S(\mathbf{v}) - \mathbb{E}[S(\mathbf{v}) \mid H_1])^2\right]$$

Substitute $S(\mathbf{v}) = \frac{1}{M}(\mathbf{x}+\mathbf{w})^\top \mathbf{w}$ and $\mathbb{E}[S(\mathbf{v}) \mid H_1] = \frac{1}{M}\boldsymbol{\mu}_\mathbf{x}^\top \mathbf{w} + \frac{1}{M}\|\mathbf{w}\|_2^2$:

$$= \mathbb{E}\left[\left(\frac{1}{M}(\mathbf{x}^\top \mathbf{w} + \mathbf{w}^\top \mathbf{w}) - \frac{1}{M}\boldsymbol{\mu}_\mathbf{x}^\top \mathbf{w} - \frac{1}{M}\|\mathbf{w}\|_2^2\right)^2\right]$$

Simplify the expression inside the expectation:

$$= \mathbb{E}\left[\left(\frac{1}{M}(\mathbf{x} - \boldsymbol{\mu}_\mathbf{x})^\top \mathbf{w}\right)^2\right]$$

3.2 Simplified Derivation of Variance under H_1 (Part 2)

Since $\mathbf{x} - \boldsymbol{\mu}_\mathbf{x}$ is a zero-mean random vector with variance σ_x^2, we have:

$$= \frac{1}{M^2}\mathbb{E}\left[\left((\mathbf{x} - \boldsymbol{\mu}_\mathbf{x})^\top \mathbf{w}\right)^2\right]$$

Expanding the expectation:

$$= \frac{1}{M^2}\|\mathbf{w}\|_2^2 \cdot \sigma_x^2$$

Conclusion: The variance under H_1 simplifies to:

$$\mathrm{Var}(S(\mathbf{v}) \mid H_1) = \frac{1}{M^2}\|\mathbf{w}\|_2^2 \sigma_x^2$$

Key Conclusion

The obtained variance under H_0 and H_1 is identical, implying that the variance is entirely driven by $\|\mathbf{w}\|_2^2$ and σ_x^2.

3.2 Error Probabilities

Probability of False Acceptance:

$$P_{\text{FA}}(\gamma) = Q\left(\frac{\gamma - \frac{1}{M}\boldsymbol{\mu}_{\mathbf{x}}^{\top}\mathbf{w}}{\sqrt{\frac{\|\mathbf{w}\|_2^2\sigma_x^2}{M^2}}}\right)$$

Probability of Detection:

$$P_D(\gamma) = Q\left(\frac{\gamma - \left(\frac{1}{M}\boldsymbol{\mu}_{\mathbf{x}}^{\top}\mathbf{w} + \frac{1}{M}\|\mathbf{w}\|_2^2\right)}{\sqrt{\frac{\|\mathbf{w}\|_2^2\sigma_x^2}{M^2}}}\right)$$

Probability of Miss:

$$P_{\text{miss}}(\gamma) = 1 - P_D(\gamma)$$

3.2 Summary and Key Insights

Key Insights

- **Term $\frac{1}{M}\boldsymbol{\mu}_{\mathbf{x}}^{\top}\mathbf{w}$:** Approaches zero in expectation for large images when $\mathbf{w}$ is generated from a zero-mean key, reducing interference caused by $\boldsymbol{\mu}_{\mathbf{x}}$.
- **Difference in Means (H_1 vs H_0):** The term $\frac{1}{M}\|\mathbf{w}\|_2^2$ increases with stronger watermarks, enhancing detection performance.
- **Image Variance σ_x^2:** Higher σ_x^2 increases overlap between PDFs, reducing detection accuracy. Flat regions (small σ_x^2) improve detection, while edges (large σ_x^2) degrade it.
- **Effect of Image Size M:** Increasing M improves detection by reducing the impact of variance and amplifying the mean difference.

3.2 Performance Analysis

Parameters:

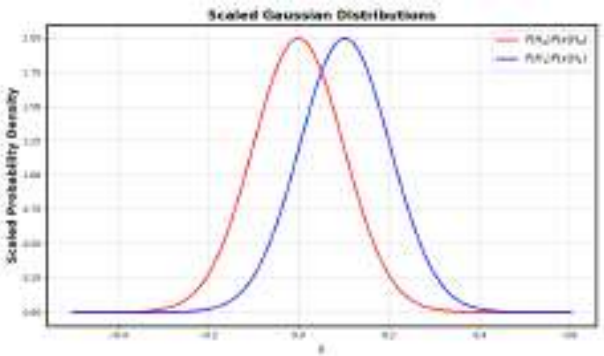
$M = 32 \times 32$

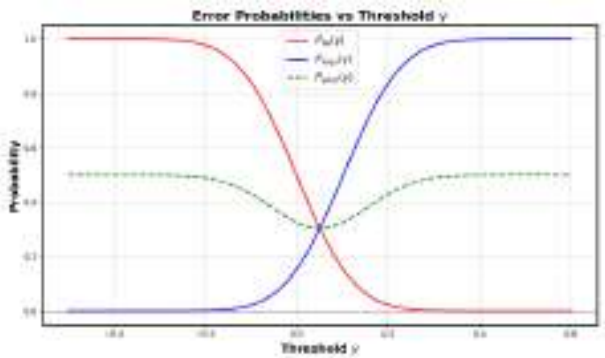
σ_x^2=100 (large variance)

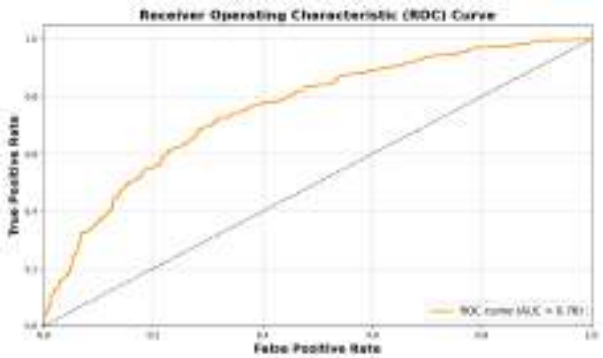
$\|\mathbf{w}\|_2$: assume we generate $\mathbf{w}$ from $\mathcal{N}(\mathbf{0}, \sigma_w^2 \mathbf{I}_M)$

The expected value $\mathbb{E}[\|\mathbf{w}\|_2] \sim M\sigma_w^2$

Let $\sigma_w^2 = 0.01$ and thus $\mathbb{E}[\|\mathbf{w}\|_2] \sim 10,24$







3.2 Performance Analysis

Parameters:

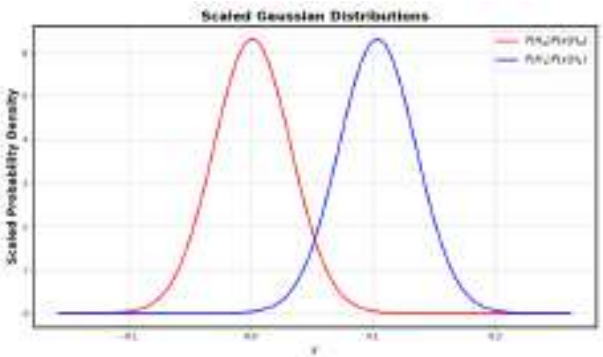
$M = 32 \times 32$

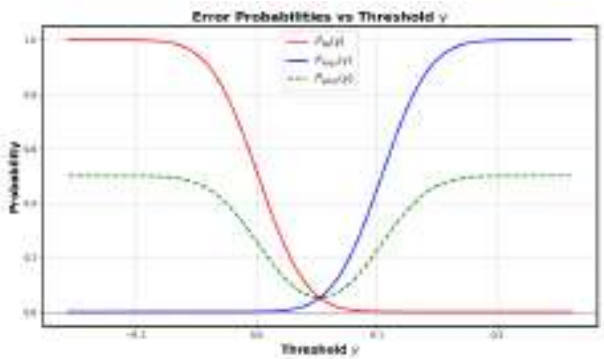
σ_x^2=10 (decreased variance)

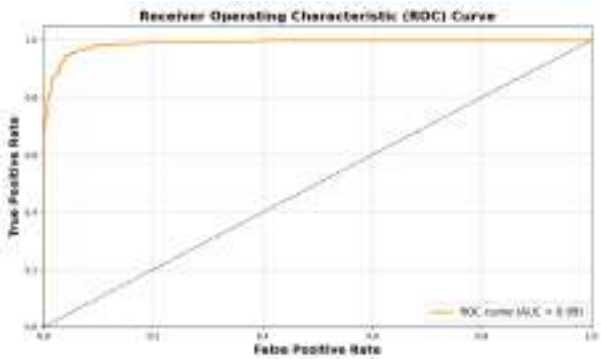
$\|\mathbf{w}\|_2$: assume we generate $\mathbf{w}$ from $\mathcal{N}(\mathbf{0}, \sigma_w^2 \mathbf{I}_M)$

The expected value $\mathbb{E}[\|\mathbf{w}\|_2] \sim M\sigma_w^2$

Let $\sigma_w^2 = 0.01$ and thus $\mathbb{E}[\|\mathbf{w}\|_2] \sim 10,24$







3.2 Impact of Image Variance (σ_x^2)

Key Observation: The variance of the image σ_x^2 directly influences detection performance by affecting the score variance.

Effect on Variance of Scores

- Larger σ_x^2 increases the variance of the score:

$$\sigma_0^2 = \sigma_1^2 = \frac{\sigma_x^2 \|\mathbf{w}\|_2^2}{M^2}$$

- Increased score variance causes more overlap between PDFs, reducing detection performance.

Effect on Different Image Regions

- **Flat regions:** Low $\sigma_x^2 \rightarrow$ less interference $\rightarrow$ better detection.
- **Edges and textures:** High $\sigma_x^2 \rightarrow$ more interference $\rightarrow$ poorer detection.

3.2 Performance Analysis

Parameters:

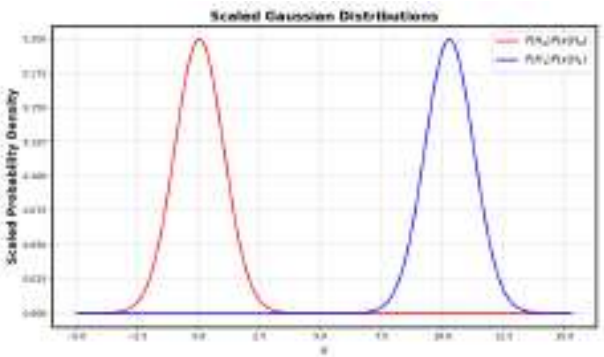
$M = 32 \times 32$

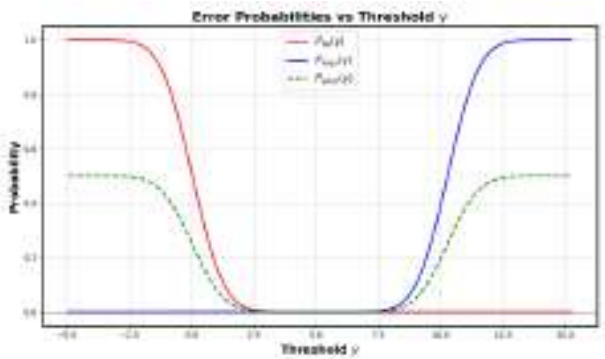
σ_x^2=10 (decreased variance)

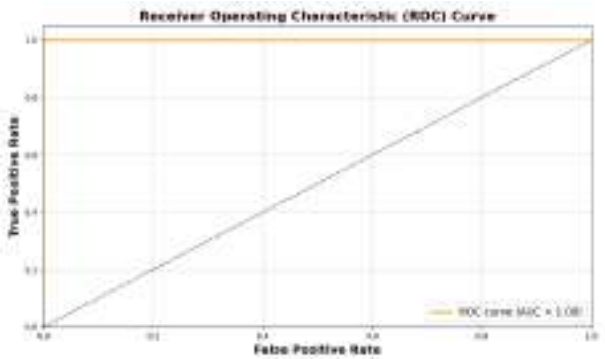
$\|\mathbf{w}\|_2$: assume we generate $\mathbf{w}$ from $\mathcal{N}(\mathbf{0}, \sigma_w^2 \mathbf{I}_M)$

The expected value $\mathbb{E}[\|\mathbf{w}\|_2] \sim M\sigma_w^2$

Let $\sigma_w^2 = 0.1$ and thus $\mathbb{E}[\|\mathbf{w}\|_2] \sim 102,4$







3.2 Impact of Watermark Strength ($\|\mathbf{w}\|$)

Key Observation: Increasing the norm of the watermark $\|\mathbf{w}\|$ improves detection performance.

Effect on Mean Difference

- The difference between the means under H_0 and H_1 increases:

$$\Delta\mu = \frac{1}{M}\|\mathbf{w}\|_2^2 \quad \text{(larger for stronger } \|\mathbf{w}\|\text{)}$$

Effect on Variance

- Variance under both hypotheses increases proportionally:

$$\sigma_0^2 = \sigma_1^2 = \frac{\sigma_x^2\|\mathbf{w}\|_2^2}{M^2}$$

- However, the mean difference increases faster than the variance.

3.2 Impact of Watermark Strength

Key Effect:

Important Insight

- Increasing $\|\mathbf{w}\|_2^2$ increases the mean difference $\Delta\mu = \frac{1}{M}\|\mathbf{w}\|_2^2$ while proportionally increasing the variance $\sigma^2 = \frac{\sigma_x^2\|\mathbf{w}\|_2^2}{M^2}$.
- **SNR**, defined as the ratio $\text{SNR} = \frac{\Delta\mu}{\sigma} = \frac{\|\mathbf{w}\|_2}{\sigma_x}$, improves with higher $\|\mathbf{w}\|_2^2$.

Conclusion: Higher $\|\mathbf{w}\|_2^2$ enhances detection performance by improving SNR and reducing error probabilities.

3.2 Performance Analysis

Parameters:

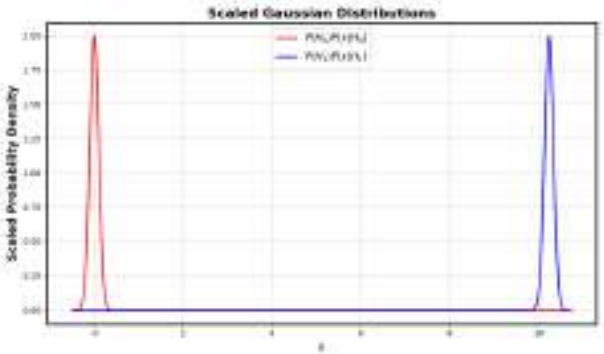
$M = 320 \times 320$ (increased image size)

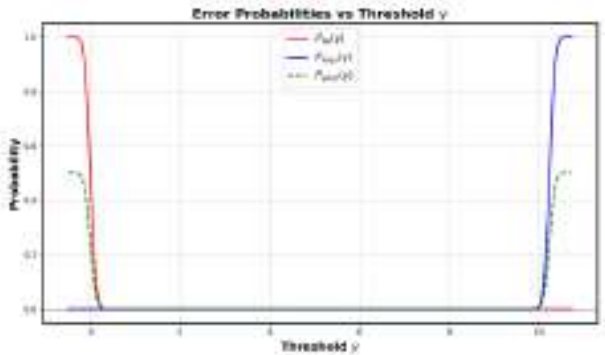
σ_x^2=100

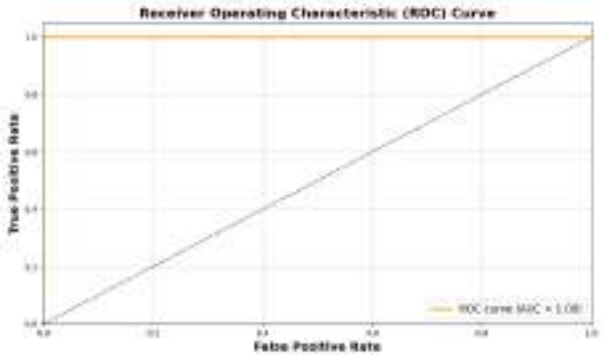
$\|\mathbf{w}\|_2$: assume we generate $\mathbf{w}$ from $\mathcal{N}(\mathbf{0}, \sigma_w^2 \mathbf{I}_M)$

The expected value $\mathbb{E}[\|\mathbf{w}\|_2] \sim \sqrt{M\sigma_w^2}$

Let $\sigma_w^2 = 0.01$ and thus $\mathbb{E}[\|\mathbf{w}\|_2] \sim 1024$







3.2 Impact of Increasing Image Size (M)

Key Observation: Increasing M affects the detection performance in two ways:

Effect on Mean Difference

- The separation between the means under H_0 and H_1 decreases:

$$\Delta\mu = \mu_1 - \mu_0 = \frac{1}{M}\|\mathbf{w}\|_2^2$$

- As M increases, $\Delta\mu$ becomes smaller, meaning the means get closer.
- **Counterintuitive insight:** Reducing $\Delta\mu$ alone might suggest worse performance.

3.2 Impact of Increasing Image Size (M)

Key Observation: Increasing M affects the detection performance in two ways:

Effect on Variance

- However, the variances under both hypotheses decrease much faster:

$$\sigma_0^2 = \sigma_1^2 = \frac{\sigma_x^2 \|\mathbf{w}\|_2^2}{M^2}$$

- As M increases, σ_0^2 and σ_1^2 decrease proportionally to $\frac{1}{M^2}$.
- Narrower distributions result in better separation between H_0 and H_1, improving detection performance.

3.2 Summary of Impact on Detection Performance

Summary of Effects:

Important Analogies

- Despite the decreasing mean difference $\Delta\mu$ with increasing M, the faster reduction in variance σ^2 leads to improved signal-to-noise ratio (SNR):

$$\text{SNR} = \frac{\Delta\mu}{\sigma_0} = \frac{\|\mathbf{w}\|_2}{\sigma_x}$$

- A higher SNR implies better detection performance, with reduced probabilities of false acceptance (P_{FA}) and miss (P_{miss}).

Conclusion:

- Increasing M results in:
  - **Narrower distributions** under both hypotheses.
  - **Lower error probabilities**, despite the reduced mean difference.
  - **Improved overall detection performance**.

3.2 Summary of Impact on Detection Performance

Summary of Effects:

Important Analogies

- When the watermark $\mathbf{w}$ is generated as a Gaussian random process with variance $\sigma_w^2 = 1$, its expected norm $\|\mathbf{w}\|_2^2$ increases proportionally to M:

$$\text{Expected Norm:} \quad \mathbb{E}[\|\mathbf{w}\|_2^2] \sim \sqrt{M}$$

- Therefore, the SNR increases as:

$$\text{SNR} = \frac{\Delta\mu}{\sigma_0} = \frac{\mathbb{E}[\|\mathbf{w}\|_2]}{\sigma_x} \propto \sqrt{M}$$

3.2 Detailed Explanation of SNR and Norm of Watermark

Key Insight: The norm of the watermark $\|\mathbf{w}\|_2^2$ is a function of M.

Impact of Norm of Watermark

- If the watermark $\mathbf{w}$ is generated as a Gaussian random process with zero mean and variance $\sigma_w^2 = 1$, its norm scales with M:

$$\|\mathbf{w}\|_2 \propto \sqrt{M}$$

- This scaling ensures that as M increases, the mean difference $\Delta\mu$ between the hypotheses does not decrease significantly in relative terms.
- The faster reduction in variance σ_0^2 with M dominates, leading to:

$$\text{SNR} \propto \sqrt{M}$$

- Therefore, larger image sizes M improve detection performance by increasing the SNR and reducing error probabilities.

Statistical Properties of $\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \sigma_w^2 \mathbf{I}_M)$

Setup: Let $\mathbf{w} = (w_1, \ldots, w_M) \in \mathbb{R}^M$, where $w_i \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0, \sigma_w^2)$. Then:

- **Squared ℓ_2-norm**:

$$\|\mathbf{w}\|_2^2 = \sum_{i=1}^{M} w_i^2 \sim \sigma_w^2 \cdot \chi_M^2$$

  $\Rightarrow$ Follows a scaled chi-squared distribution with M degrees of freedom.
- **Expected squared norm**:

$$\mathbb{E}\left[\|\mathbf{w}\|_2^2\right] = M\sigma_w^2$$

- **ℓ_2-norm (magnitude)**:

$$\|\mathbf{w}\|_2 = \sqrt{\sum_{i=1}^{M} w_i^2} \sim \sigma_w \cdot \chi_M$$

  $\Rightarrow$ Follows a chi distribution scaled by σ_w.
- **Expected norm**:

$$\mathbb{E}\left[\|\mathbf{w}\|_2\right] = \sigma_w \cdot \sqrt{2} \cdot \frac{\Gamma\left(\frac{M+1}{2}\right)}{\Gamma\left(\frac{M}{2}\right)} \sim \sigma_w \sqrt{M} \quad \text{as } M \to \infty$$

Watermark scaling

Watermark Embedding Model:

Motivation for Scaling

- In previous analysis, we considered a simple watermark embedding model:

$$\mathbf{y} = \mathbf{x} + \mathbf{w}$$

- This model is **inflexible** as it lacks control over the watermark strength, which is crucial for balancing:
  - **Perceptual requirements** $D_{\mathbf{xy}}$ (ensuring imperceptibility of the watermark).
  - **Robustness requirements** (ensuring the watermark survives attacks or transformations).
- To introduce flexibility, we multiply the watermark by a **scaling factor** α, resulting in a new embedding model:

$$\mathbf{y} = \mathbf{x} + \alpha\mathbf{w}$$

Conclusion:

Performance under watermark scaling

Results under H_0 and H_1

Under H_0:

$$\mu_0 = \frac{1}{M}\boldsymbol{\mu}_{\mathbf{x}}^{\top}\mathbf{w}, \quad \sigma_0^2 = \frac{\alpha^2\sigma_x^2\|\mathbf{w}\|_2^2}{M^2}$$

Under H_1:

$$\mu_1 = \frac{1}{M}\boldsymbol{\mu}_{\mathbf{x}}^{\top}\mathbf{w} + \frac{\alpha^2}{M}\|\mathbf{w}\|_2^2, \quad \sigma_1^2 = \frac{\alpha^2\sigma_x^2\|\mathbf{w}\|_2^2}{M^2}$$

Key Points:

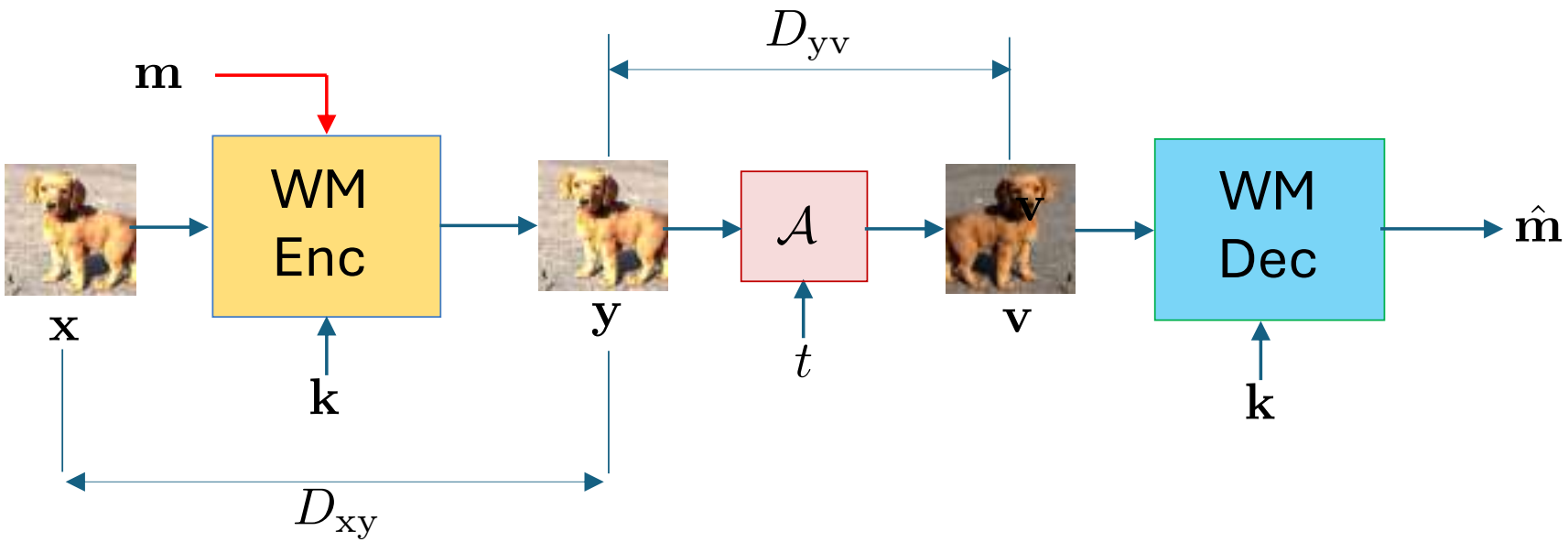
- Scaling factor α increases the mean under H_1, enhancing separability between PDFs.
- Variance under both hypotheses scales proportionally with α^2, impacting detection performance.
- A higher α improves robustness but may affect imperceptibility.

Digital Watermarking

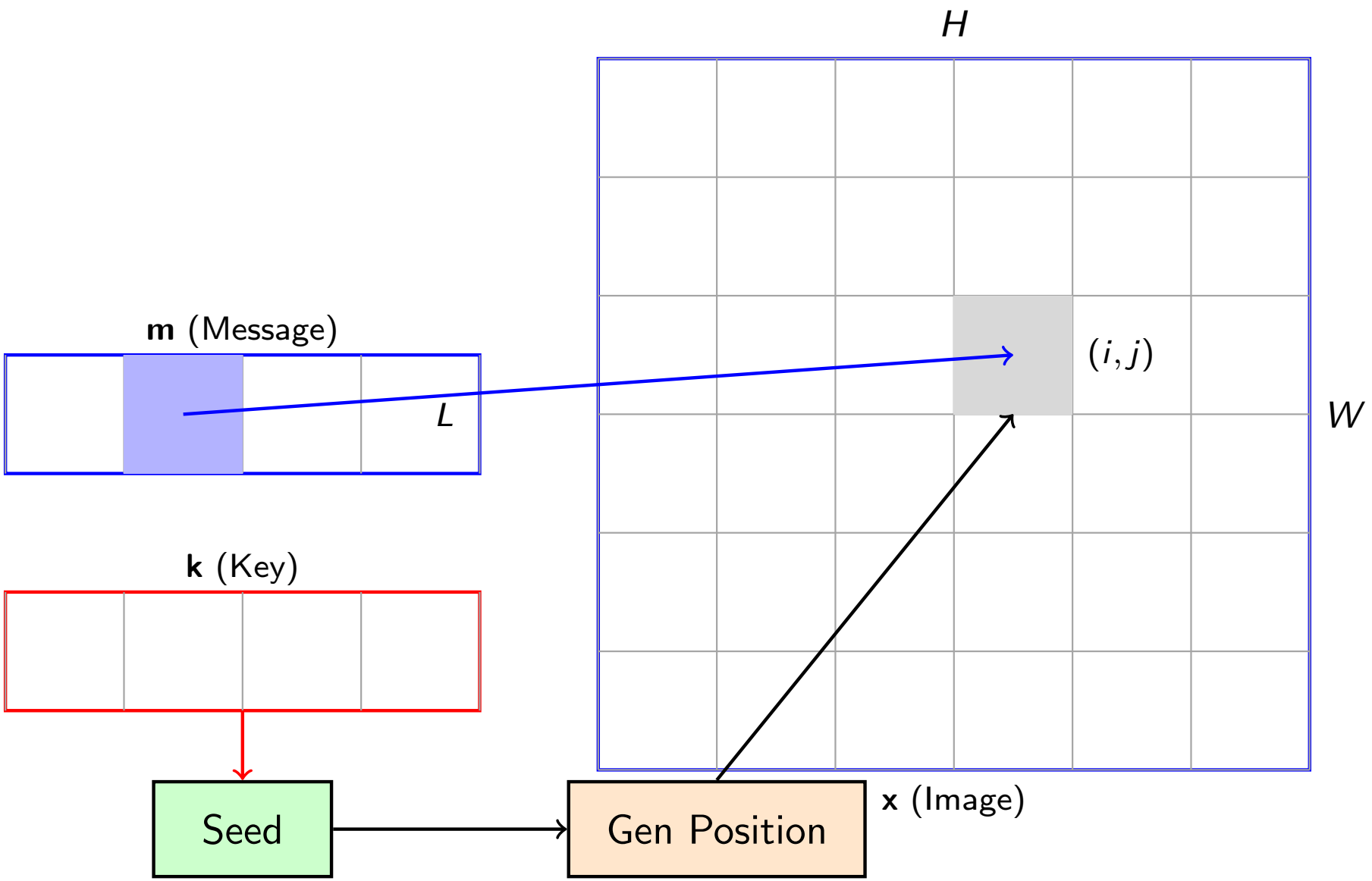
Foundations of Digital Data Hiding

L-Bit Watermarking or Decoding Problem

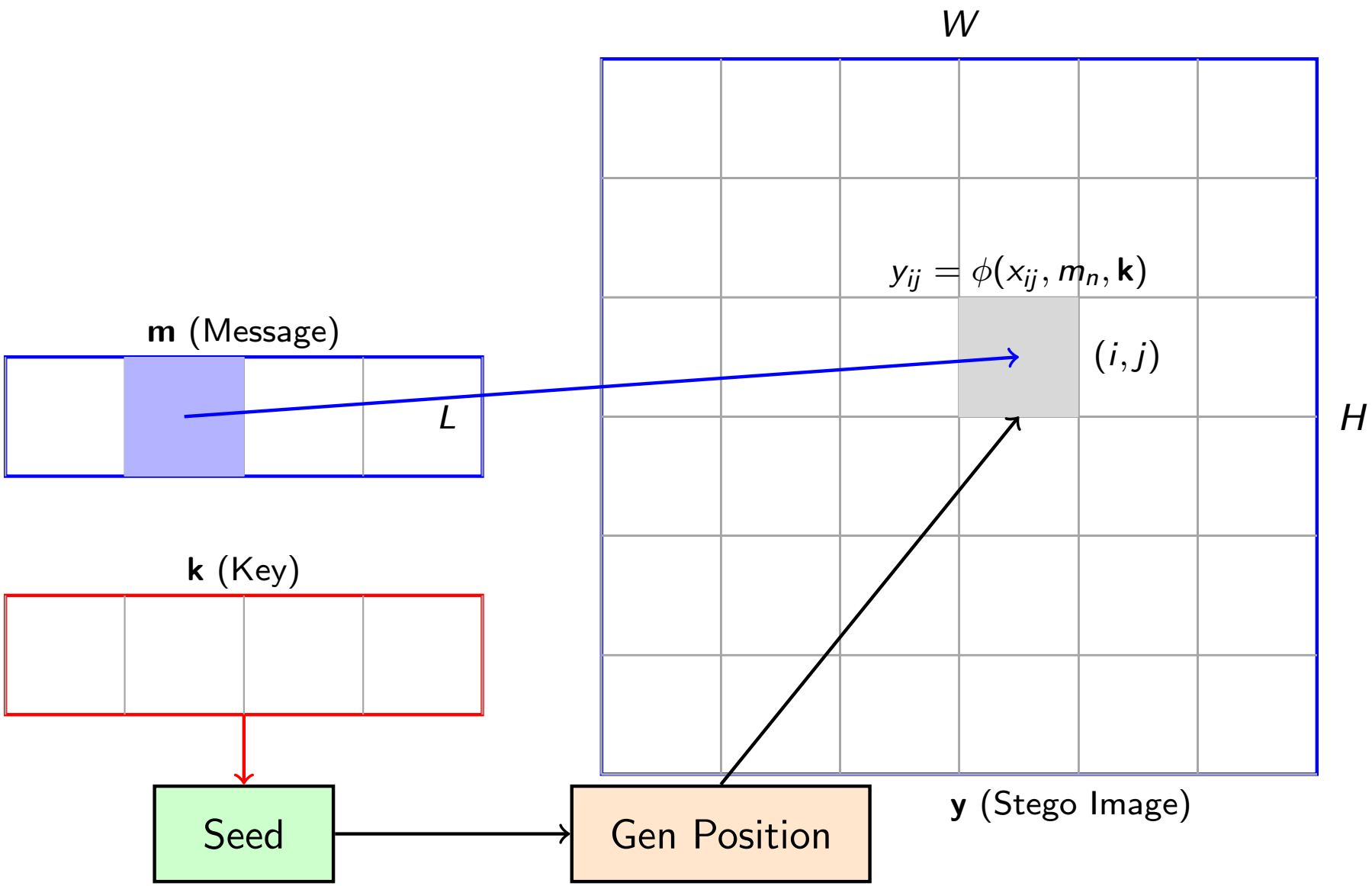
3.2 Decoding problem



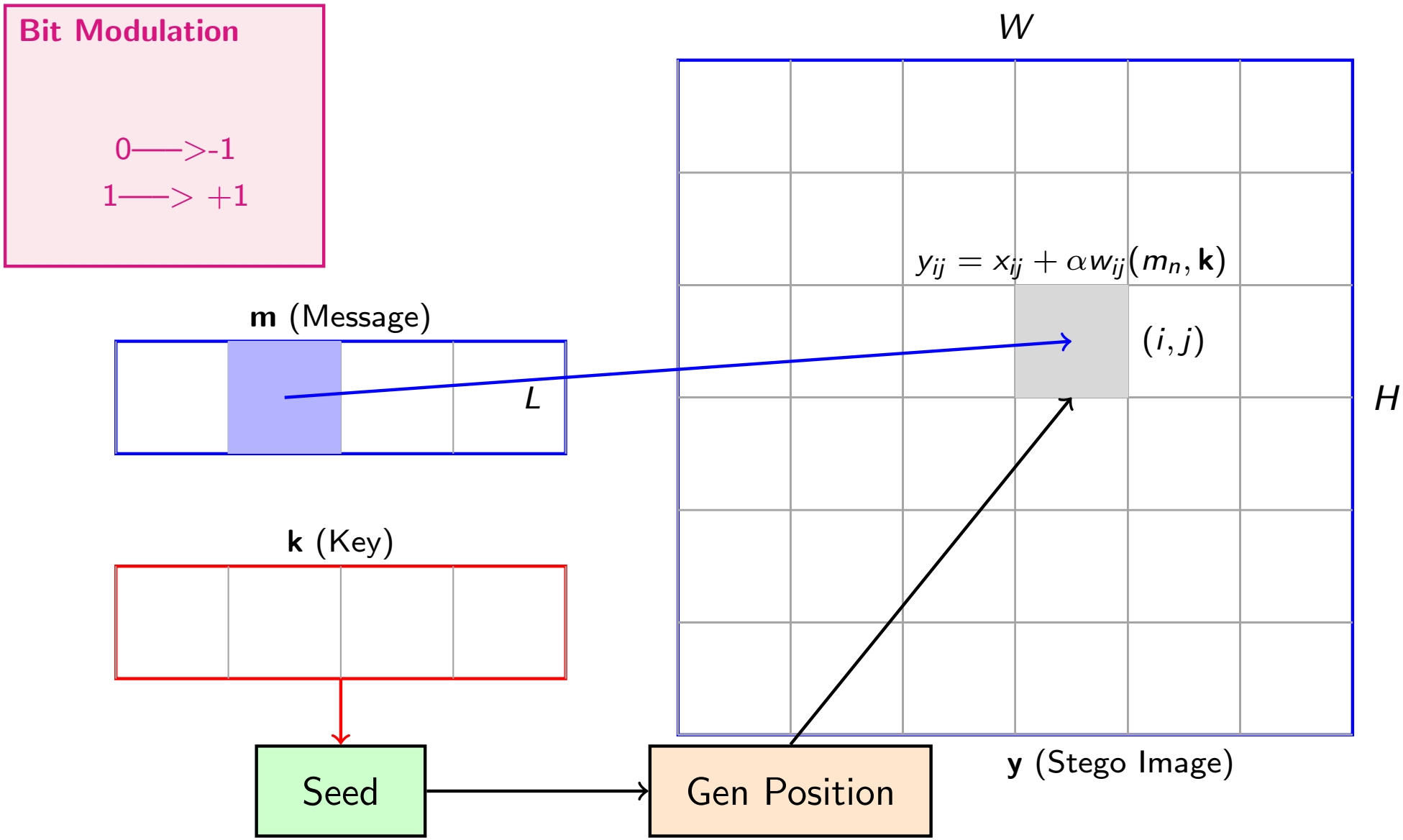
3.2 Single Bit Allocation Based on Secret Key



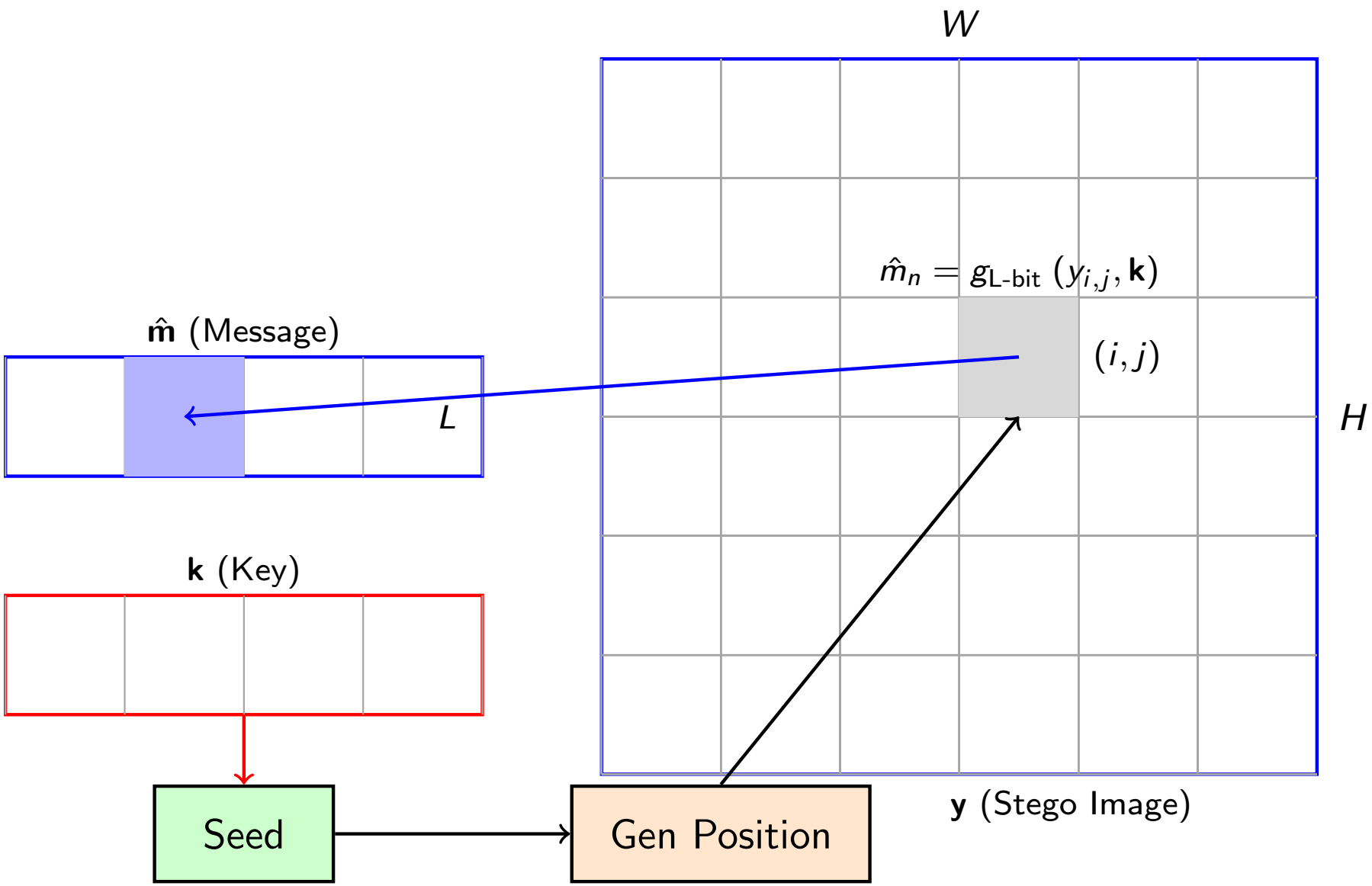
3.2 Single Bit Message Embedding Based on Secret Key



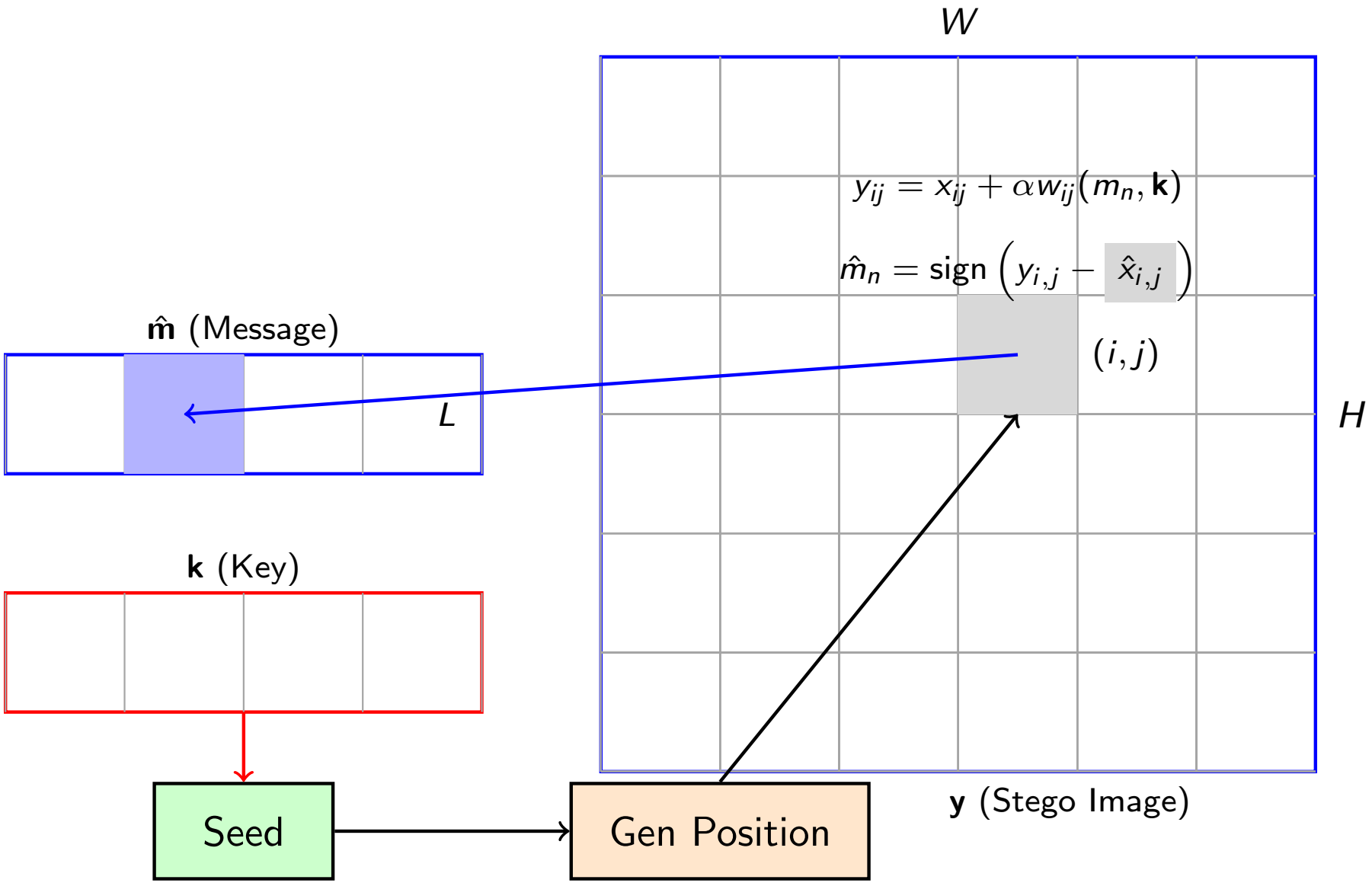
3.2 Single Bit Message Embedding based on Additive Scheme



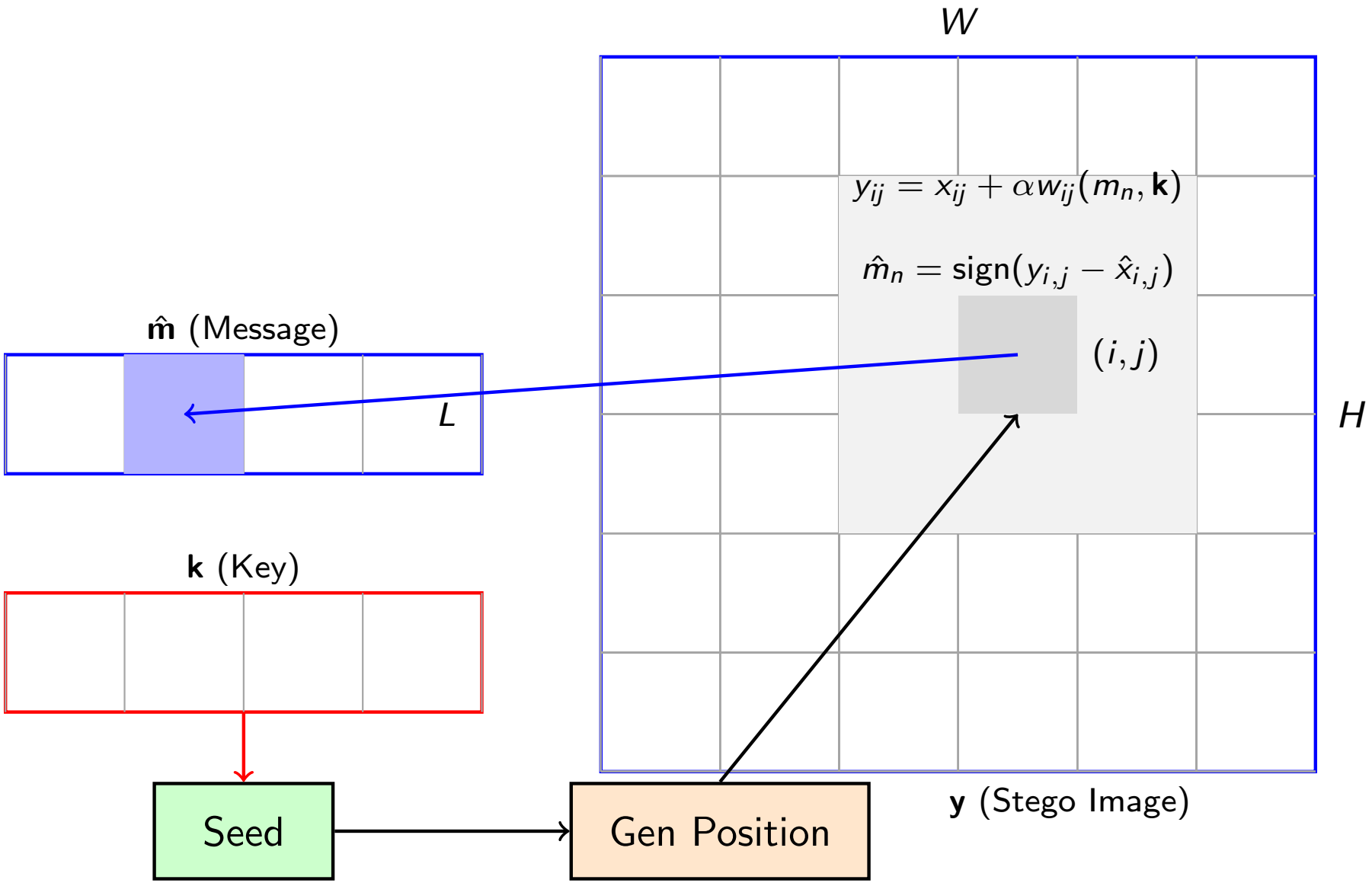
3.2 Single Bit Message Extraction



3.2 Single Bit Message Extraction based on Additive Scheme



3.2 Single Bit Message Extraction based on Additive Scheme



3.2 Observation Model

Observation Model:

$$y_{ij} = x_{ij} + \alpha w_{ij}(m_n, \mathbf{k})$$

Key Insight: Role of the Image in Detection

- Direct detection of m_n by extracting the sign of y_{ij} fails due to the interference caused by x_{ij}.
- The image pixel values x_{ij} typically range between 0 and 255, while the watermark signal αw_{ij} is much weaker, e.g., ± 3 or ± 7.
- Therefore, an accurate estimation of the original pixel x_{ij} is crucial to suppress interference and improve detection accuracy.

3.2 Estimation of the Bit Based on Sign Estimation

Estimation Process:

$$\hat{m}_n = \text{sign}\left(y_{ij} - \hat{x}_{ij}\right)$$

Explanation:

- $\hat{x}_{ij}$ is an estimate of the original pixel x_{ij} obtained using:
  - A **local mean** over a small $w \times w$ window around (i, j).
  - A **median estimator** over the same window, where pixel values are sorted, and the median is selected.
- The estimated watermark value is given by:

$$\hat{w}_{ij} = y_{ij} - \hat{x}_{ij} = \left(x_{ij} - \hat{x}_{ij}\right) + \alpha w_{ij} = \epsilon_{ij} + \alpha w_{ij}$$

  where ϵ_{ij} is the estimation error.

Key Insight: Role of the Estimation Error

- ϵ_{ij} depends on the nature of the region:
  - **Flat regions**: The mean or median estimator provides accurate estimates, leading to small ϵ_{ij}.
  - **Edges and textures**: Estimators are less accurate, resulting in larger ϵ_{ij}.

3.2 Motivation for Repetition

Why Repeat the Message?

Crude Estimation

- Embedding and extracting a single message bit at a single position is unreliable.
- Estimation depends on the type of region:
  - **Flat regions:** Low interference, better estimation accuracy.
  - **Edges and textures:** High interference, poor estimation accuracy.

Robustness Against Transformations

- Image transformations such as cropping, scaling, or rotation may partially destroy embedded information.
- Repeating the message across multiple positions increases robustness.

3.2 Large Image Size and Repetition Strategy

Leveraging Large Image Size:

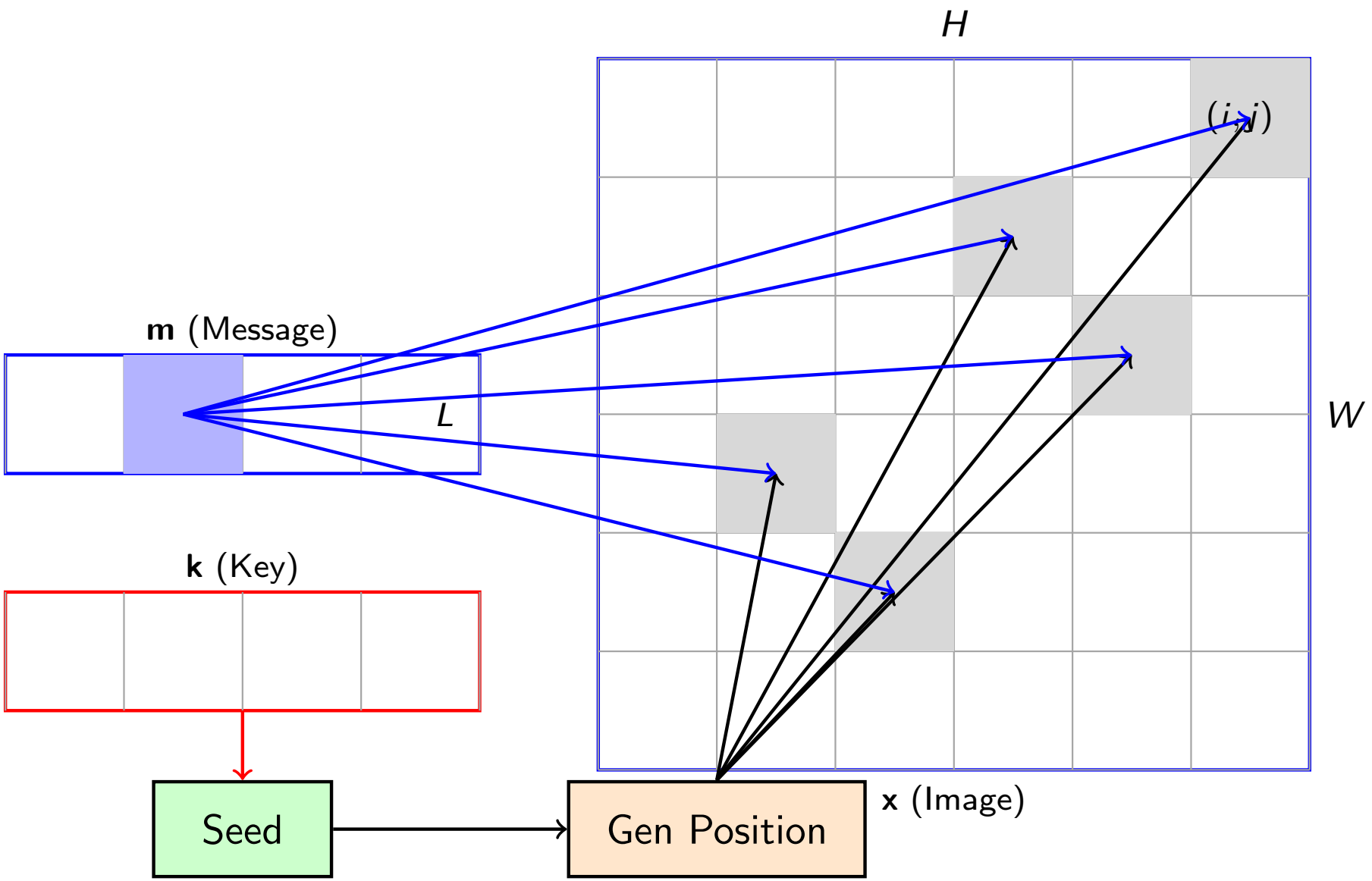
Exploiting Image Size

- Typical image size $H \times W$ is much larger than the message length L.
- This allows embedding each bit of the message multiple times without significant quality degradation.

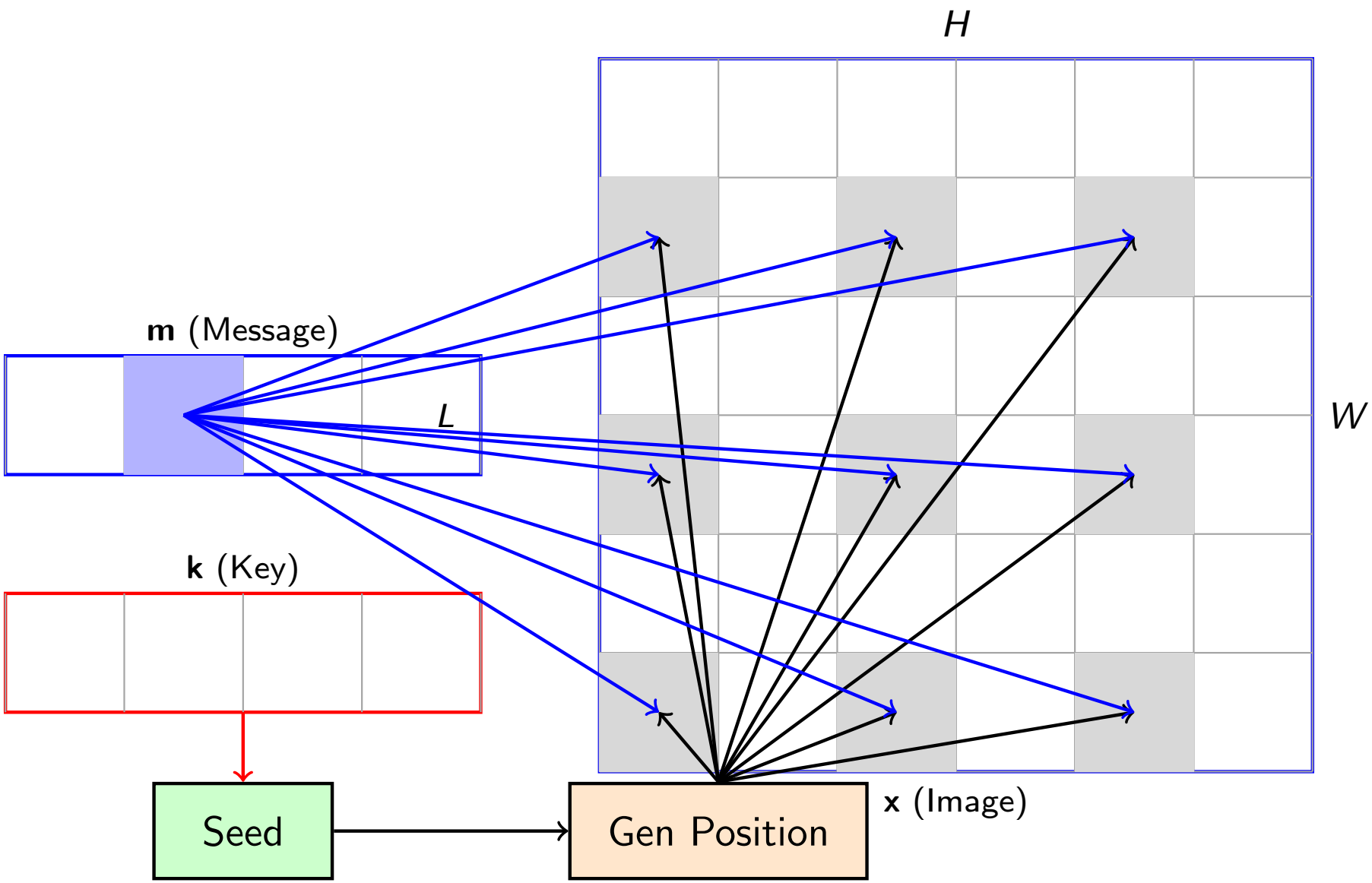
Key Insight: Repetition Improves Robustness

- To improve reliability and robustness, each message bit m_n (for all L bits) is embedded R_{rep} times across the image.
- The number of repetitions R_{rep} ensures message extraction even if parts of the image are lost or distorted.

3.2 Multiple Position Bit Allocation Based on Secret Key



3.2 Regular Position Bit Allocation Based on Secret Key



3.2 Benefit from Redundancy in Bit Estimation

Redundancy for Enhanced Accuracy:

- Each bit m_n is embedded R_{rep} times across different positions.
- The estimated watermark at each position is denoted as $\hat{w}^{(l)}$, where $l = 1, \ldots, R_{\text{rep}}$.
- Final estimated watermark:

$$\hat{w} = \sum_{l=1}^{R_{\text{rep}}} \hat{w}^{(l)}$$

- Extracted bit:

$$\hat{m}_n = \text{sign}\left(\hat{w}\right)$$

Key Insights

- **Bit accumulation:** Bits add coherently over repetitions.
- **Noise cancellation:** Noise averages out due to zero mean.
- **Improved accuracy:** Higher R_{rep} improves bit extraction reliability.

3.2 Advantages of Repetitive Encoding

Key Idea: Repeating message bits randomly across the image improves robustness.

Defender's Perspective

- Defender knows the secret key for bit allocation.
- **Accumulates repeated bits** at known positions.
- **Averages repetitions** (R_{rep}-times) to improve estimation.
- Even weak bits contribute, as noise cancels out over repetitions.

Attacker's Perspective

- Without the key, the attacker cannot locate bits.
- Random summation leads to zero-mean Gaussian noise.
- Probability of correct extraction is reduced to random guessing ($P = 0.5$).

3.2 Trade-Off: Random vs Periodic Repetition

Key Point: Random repetition enhances security, while periodic repetition improves robustness to geometrical attacks (see later Self-synchronization watermarking).

Periodic Repetition Advantages

- Easier recovery under geometrical attacks (e.g., scaling, cropping).
- Enables **self-synchronization** for enhanced robustness.

Trade-Off Summary

- **Random repetition:** High security, hard to guess bit positions.
- **Periodic repetition:** High robustness, supports self-synchronization.
- Choice depends on application needs: security vs robustness.

3.2 Enhancing Robustness with Error Correction Codes

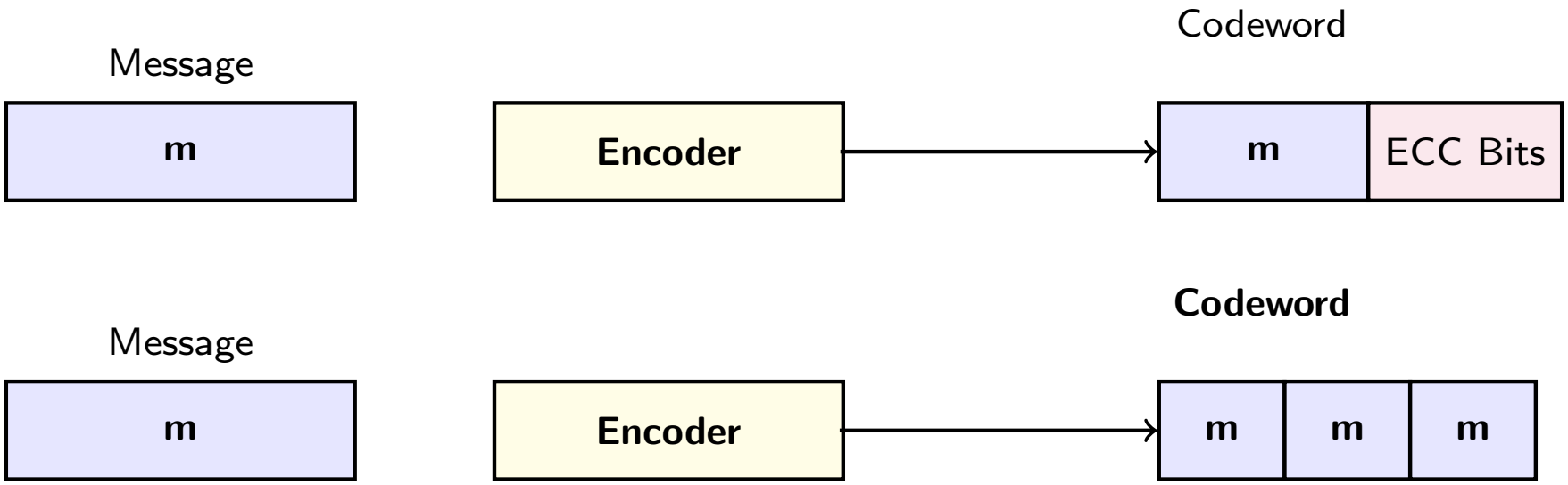
Error Correction Code (ECC) for Enhanced Encoding:

- Prior to embedding, the message **m** is passed through an **encoder** of an error correction code (ECC) with rate R_{ECC}.
- The ECC expands the original message to introduce redundancy, improving robustness against errors caused by transformations, noise, and cropping.
- Example rates:
  - $R_{\text{ECC}} = \frac{1}{3}$: Message length increases threefold.
  - $R_{\text{ECC}} = \frac{1}{9}$: High redundancy for extreme robustness.

Key Insights

- **Redundancy increases robustness:** ECC introduces structured redundancy, ensuring higher reliability in decoding.
- **Better than simple repetition:** Unlike direct repetition, ECC allows correction of errors across multiple embedded bits.
- **Higher ECC rate improves robustness:** More redundancy (lower R_{ECC}) leads to better decodability under severe distortions.

3.2 Encoding Approaches: Error Correction vs Repetition



Explanation:

- **Error Correction Code Encoding:** The message is passed through an ECC encoder, producing a codeword by appending error correction bits.
- **Repetition-Based Encoding:** The message is repeated multiple times to improve robustness, creating a concatenated codeword with identical message segments.
- Both methods ensure better decodability and robustness compared to a single bit embedding.

Digital Watermarking

Mutual Information and Capacity Analysis

Understanding the Fundamental Limits of Multi-Bit Watermarking

Introduction to Mutual Information in Digital Watermarking

Watermarking Model:

$$\mathbf{y} = \mathbf{x} + \mathbf{w}$$

where:

- $\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}_{\mathbf{x}}, \mathbf{K}_{\mathbf{xx}})$ is the original image modeled as a Gaussian random vector.
- $\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \mathbf{K}_{\mathbf{ww}})$ is the watermark modeled as Gaussian noise.
- $\mathbf{y}$ is the observed image containing the watermark.

Goal: To derive the mutual information $I(\mathbf{y}; \mathbf{w})$, which quantifies how much information about the watermark $\mathbf{w}$ can be reliably extracted from the watermarked image $\mathbf{y}$.

Mutual Information for General Covariance Matrices

Definition of Mutual Information:

$$I(\mathbf{y}; \mathbf{w}) = h(\mathbf{y}) - h(\mathbf{y} \mid \mathbf{w})$$

where $h(\cdot)$ denotes the differential entropy.

Step 1: Differential Entropy of y

$$h(\mathbf{y}) = \frac{M}{2} \log \left((2\pi e)^M \det(\mathbf{K_{xx}} + \mathbf{K_{ww}}) \right)$$

Step 2: Differential Entropy of y given w

$$h(\mathbf{y} \mid \mathbf{w}) = \frac{M}{2} \log \left((2\pi e)^M \det(\mathbf{K_{xx}}) \right)$$

Result: Mutual Information

$$I(\mathbf{y}; \mathbf{w}) = \frac{1}{2} \log \left(\frac{\det(\mathbf{K_{xx}} + \mathbf{K_{ww}})}{\det(\mathbf{K_{xx}})} \right)$$

Recall: Differential Entropy for Multivariate Gaussian

Differential Entropy for Gaussian Random Vector:
Given a multivariate Gaussian random vector $\mathbf{y} \in \mathbb{R}^M$ with mean $\boldsymbol{\mu}_\mathbf{y}$ and covariance matrix $\mathbf{K}_{\mathbf{yy}}$, its probability density function is:

$$p(\mathbf{y}) = \frac{1}{(2\pi)^{M/2} \det(\mathbf{K}_{\mathbf{yy}})^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{y} - \boldsymbol{\mu}_\mathbf{y})^\top \mathbf{K}_{\mathbf{yy}}^{-1}(\mathbf{y} - \boldsymbol{\mu}_\mathbf{y})\right)$$

Step: Compute Differential Entropy

$$h(\mathbf{y}) = -\mathbb{E}\left[\log p(\mathbf{y})\right]$$

Substituting $p(\mathbf{y})$ and simplifying:

$$h(\mathbf{y}) = \frac{M}{2} \log\left((2\pi e)^M \det(\mathbf{K}_{\mathbf{yy}})\right)$$

Key Insight: The entropy depends only on the determinant of the covariance matrix $\mathbf{K}_{\mathbf{yy}}$, which reflects the spread of $\mathbf{y}$ around its mean.

Eigenvalue Decomposition in Capacity Analysis

Step 1: Eigenvalue Decomposition of Covariance Matrix

For a Gaussian vector $\mathbf{y} \in \mathbb{R}^M$ with covariance matrix $\mathbf{K_{yy}}$:

$$\mathbf{K_{yy}} = \mathbf{V}\mathbf{\Lambda_{yy}}\mathbf{V}^\top$$

where $\mathbf{V}$ contains eigenvectors and $\mathbf{\Lambda_{yy}} = \text{diag}(\lambda_1, \ldots, \lambda_M)$ contains eigenvalues.

Step 2: Determinant in Terms of Eigenvalues

$$\det(\mathbf{K_{yy}}) = \det(\mathbf{V}\mathbf{\Lambda_{yy}}\mathbf{V}^\top) = \det(\mathbf{\Lambda_{yy}}) = \prod_{i=1}^{M} \lambda_i$$

Step 3: Differential Entropy in Terms of Eigenvalues

$$h(\mathbf{y}) = \frac{M}{2} \log \left((2\pi e)^M \prod_{i=1}^{M} \lambda_i \right)$$

Key Insight

Transform domain watermarking (e.g., DCT or wavelet domain) simplifies embedding by directly controlling the eigenvalues.

Recall: Deriving Mutual Information

Step 1: Differential Entropy of y

For $\mathbf{y} = \mathbf{x} + \mathbf{w}$, where $\mathbf{x}$ and $\mathbf{w}$ are independent Gaussian random vectors with covariance matrices $\mathbf{K_{xx}}$ and $\mathbf{K_{ww}}$ respectively:

$$\mathbf{K_{yy}} = \mathbf{K_{xx}} + \mathbf{K_{ww}}$$

Thus,

$$h(\mathbf{y}) = \frac{M}{2} \log \left((2\pi e)^M \det(\mathbf{K_{xx}} + \mathbf{K_{ww}}) \right)$$

Step 2: Conditional Differential Entropy $h(\mathbf{y} \mid \mathbf{w})$

Given $\mathbf{w}$, the uncertainty in $\mathbf{y}$ comes only from $\mathbf{x}$, whose covariance matrix remains $\mathbf{K_{xx}}$:

$$h(\mathbf{y} \mid \mathbf{w}) = \frac{M}{2} \log \left((2\pi e)^M \det(\mathbf{K_{xx}}) \right)$$

Mutual Information:

$$I(\mathbf{y}; \mathbf{w}) = h(\mathbf{y}) - h(\mathbf{y} \mid \mathbf{w}) = \frac{1}{2} \log \left(\frac{\det(\mathbf{K_{xx}} + \mathbf{K_{ww}})}{\det(\mathbf{K_{xx}})} \right)$$

Simplified Case: Diagonal Covariance Matrices

Assume that:

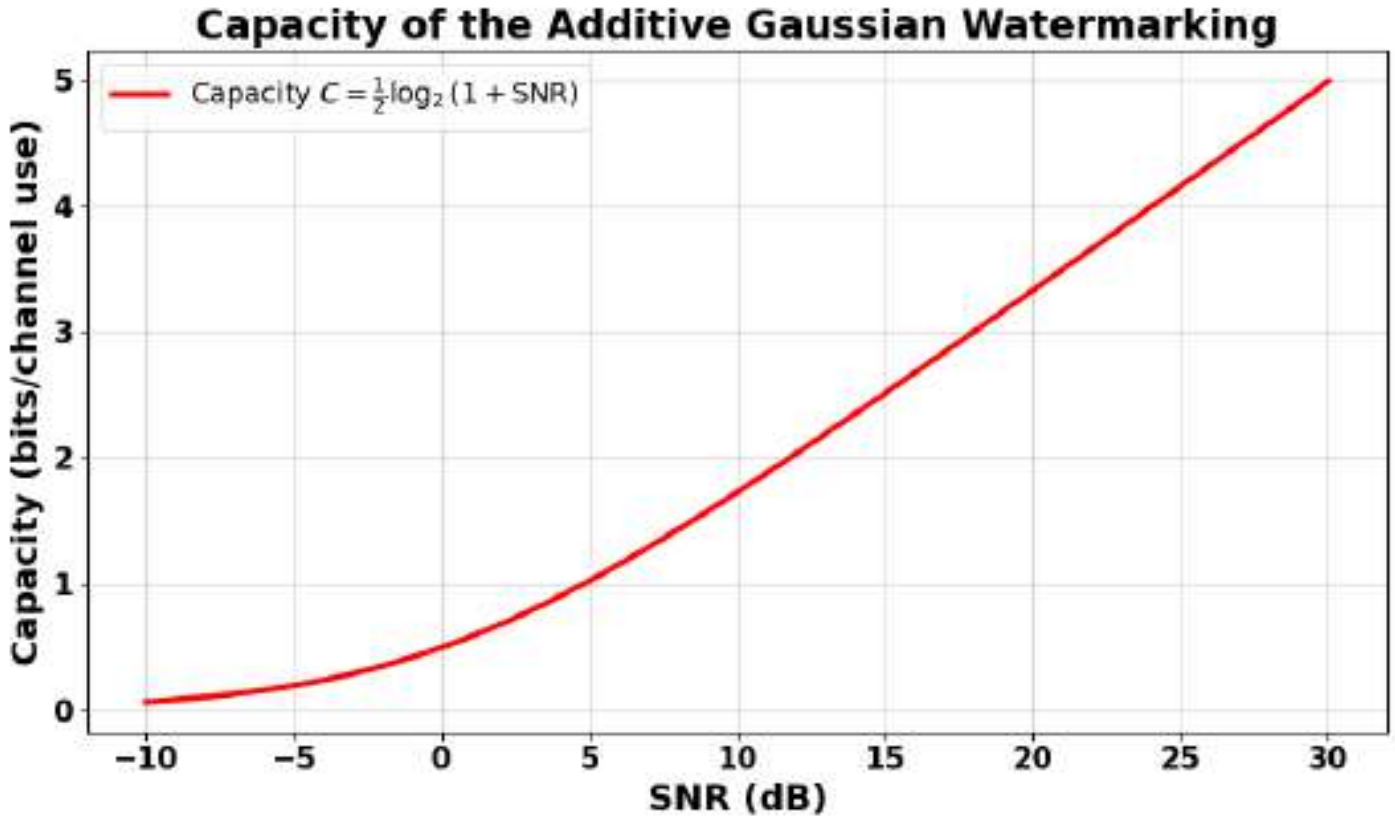
$$\mathbf{K_{xx}} = \sigma_x^2 \mathbf{I}_M, \quad \mathbf{K_{ww}} = \sigma_w^2 \mathbf{I}_M$$

Then, the mutual information simplifies to:

$$I(\mathbf{y}; \mathbf{w}) = \frac{M}{2} \log \left(1 + \frac{\sigma_w^2}{\sigma_x^2} \right)$$

Key Insight: The term $\frac{\sigma_w^2}{\sigma_x^2}$ represents the **signal-to-noise ratio (SNR)**.

3.2 Capacity of the Additive Gaussian Watermarking Channel



Key Explanation

Higher SNR improves capacity (higher information-carrying capability).

Analysis of Factors Affecting Capacity

Capacity and Key Factors:

Key Observations

- The **capacity** $C = I(\mathbf{y}; \mathbf{w})$ represents the maximum number of bits that can be reliably communicated (or embedded) without error.
- **Factors affecting capacity:**
  - **Signal strength** (σ_w^2): Higher σ_w^2 increases SNR and thus improves capacity.
  - **Interference** (σ_x^2): Higher σ_x^2 increases noise and reduces capacity.
  - **Dimensionality** (M): Larger M increases the total capacity, as more independent bits can be embedded.

Conclusion: The capacity C is directly proportional to the signal-to-noise ratio and the dimensionality of the image, while inversely proportional to the noise introduced by the image.

Practical Implications for Digital Watermarking

Why Capacity Matters in Digital Watermarking:

- The capacity C determines how many bits can be embedded in an image while ensuring robustness and imperceptibility.
- Higher capacity allows for embedding larger payloads (multi-bit watermarking) while maintaining robustness.
- **Trade-offs:**
  - Increasing σ_w^2 improves robustness but may reduce imperceptibility.
  - Reducing σ_x^2 improves capacity by reducing interference, but is dependent on the image content.
  - Larger M (higher resolution images) increases capacity, making high-resolution images preferable for watermarking.

Conclusion: Capacity analysis is essential for designing robust digital watermarking schemes and aligns with the principles of communication theory.

Digital Watermarking

Multi-Bit Digital Watermarking from Spread Spectrum Perspective

Exploring Robustness and Capacity through Orthogonal Carriers

3.2 Multi-Bit Watermarking: Spread Spectrum Communication

Key Concept: Multi-bit watermarking can be viewed as a spread spectrum digital communication problem, where each bit m_n is associated with its own carrier sequence $\mathbf{w}_n$.

Message Representation:

- Message $\mathbf{m} \in \{-1, +1\}^L$ consists of L bits.
- Each bit m_n (where $n = 1, 2, \ldots, L$) is modulated by an independent carrier sequence $\mathbf{w}_n$.

Carrier Sequences:

$$\mathbf{w}_n \sim \mathcal{N}\left(0, \sigma_w^2 \mathbf{I}_M\right), \quad n = 1, 2, \ldots, L$$

where σ_w^2 is the variance, M is the length of the carriers, and $\mathbf{I}_M$ is the $M \times M$ identity matrix.

Embedding Process:

$$\mathbf{y} = \mathbf{x} + \alpha \mathbf{w}, \quad \mathbf{w} = \sum_{n=1}^{L} m_n \mathbf{w}_n$$

3.2 Detection Process and Orthogonality

Detection Process: The goal of detection is to reliably decode each bit m_n from the watermarked signal $\mathbf{y}$.

For each bit m_n:

$$\hat{m}_n = \text{sign}\left(\mathbf{y}^\top \mathbf{w}_n\right), \quad n = 1, 2, \ldots, L$$

Key Insight: Orthogonality between carrier sequences $\mathbf{w}_n$ ensures minimal interference between different bits:

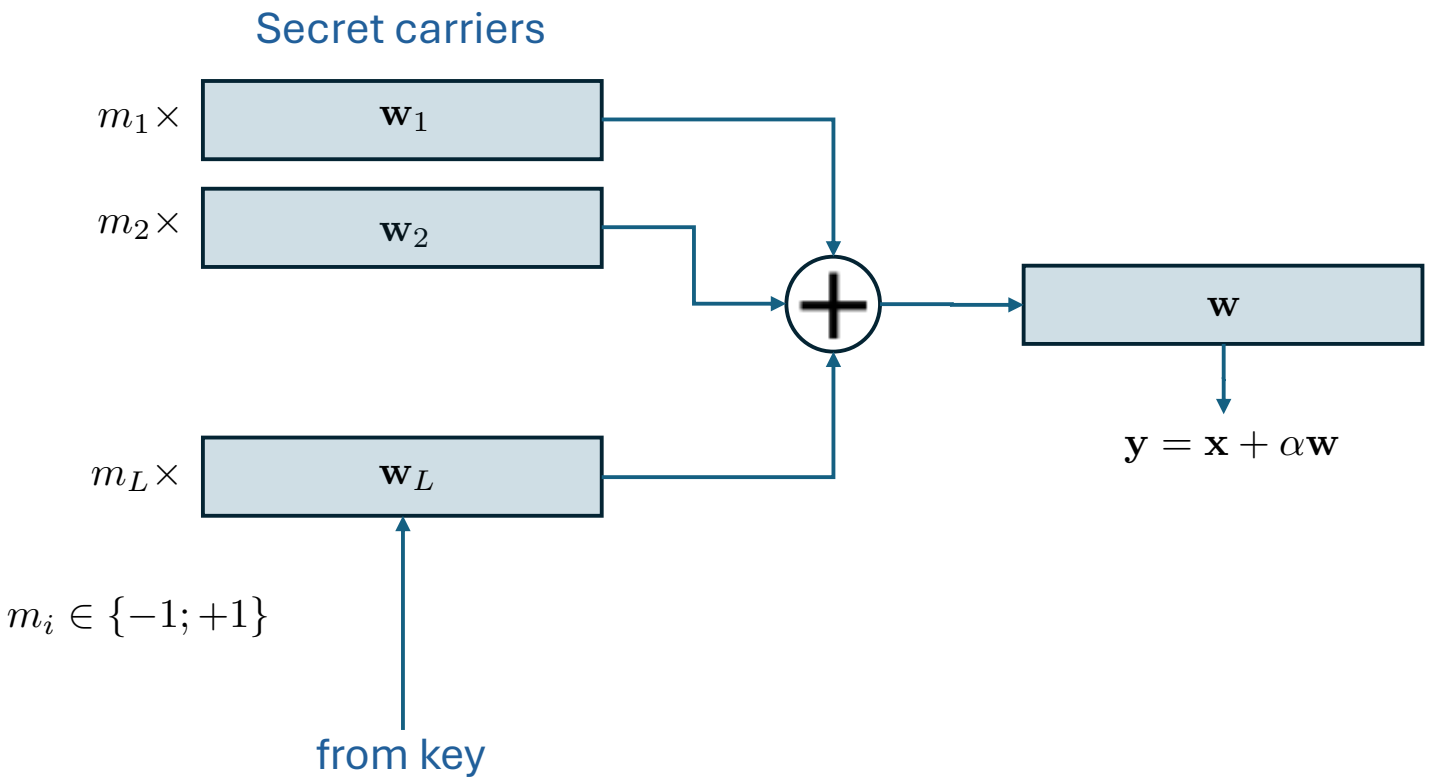
$$\mathbb{E}\left[\mathbf{w}_n^\top \mathbf{w}_m\right] = 0, \quad \forall n \neq m$$

When M (length of carriers) is large, the carriers become approximately orthogonal, ensuring robustness against interference from other bits.

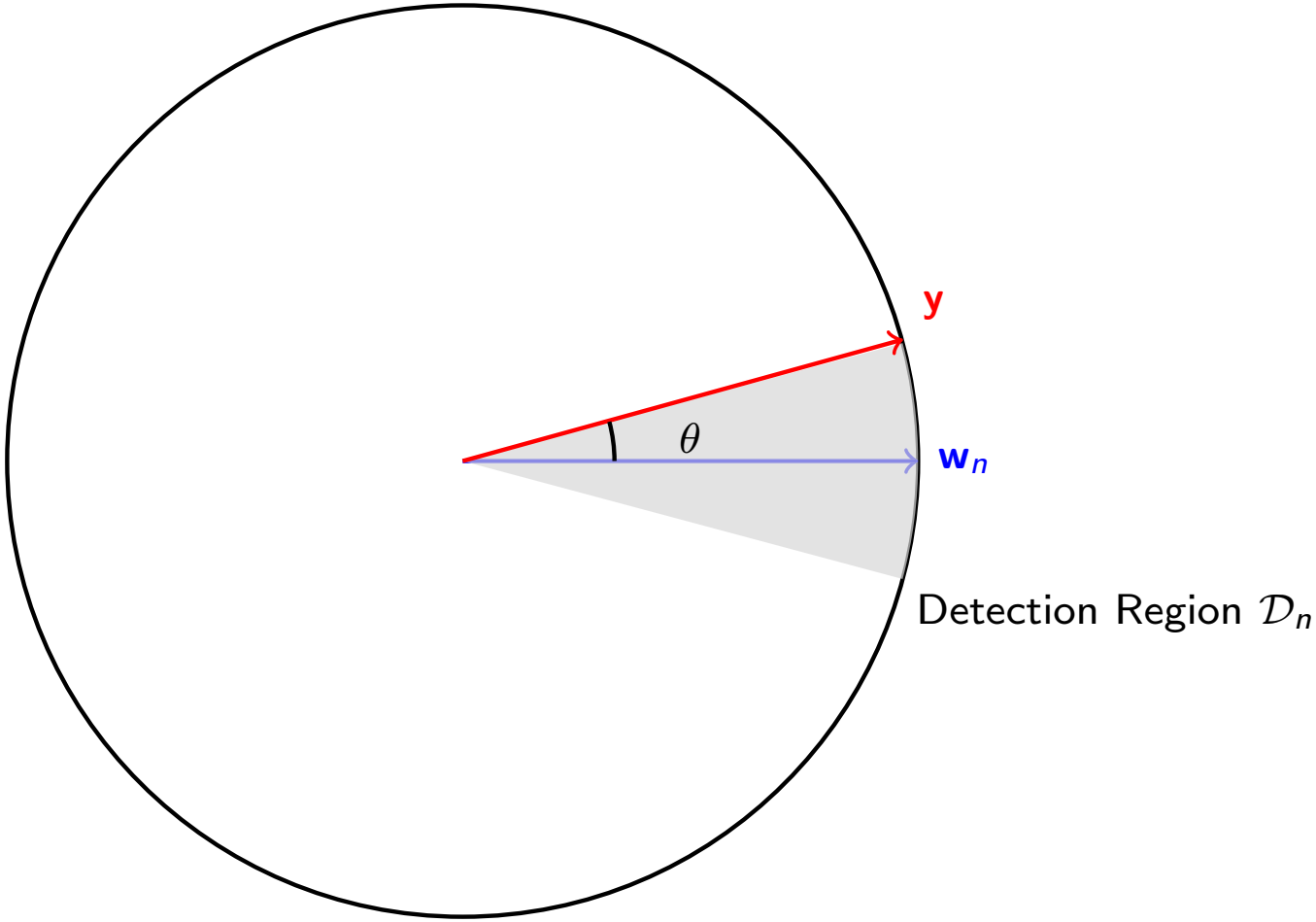
Important Observation

While orthogonality minimizes interference, the presence of $L-1$ other carriers still acts as noise, making multi-bit watermarking detection inherently harder than zero-bit watermarking.

3.2 Multi-bit spread spectrum embedding



Inner Product Test for Multi-Bit Decoding



Key Concept

Each bit m_n uses a unique carrier $\mathbf{w}_n$, creating its own detection region $\mathcal{D}_n$. Decoding relies on the sign of $\mathbf{y}^\top \mathbf{w}_n$, ensuring robustness by using orthogonal carriers.

3.2 Random vs Structured Carriers

Random vs Structured Carriers:

- **Random Carriers:** Each carrier $\mathbf{w}_n$ is generated independently from a Gaussian distribution using a secret key. This approach ensures high unpredictability.
- **Structured Carriers:** Carriers can also be generated with periodic structures, which may enhance robustness against certain types of distortions (e.g., geometric transformations).

Security Aspect: All carriers $\mathbf{w}_n$ are generated using a secret key, making it infeasible for an attacker to reconstruct the watermark without knowing the key.

Conclusion

Multi-bit watermarking introduces a tradeoff between robustness and security. While orthogonality improves detection robustness, structured carriers can enhance robustness against specific attacks but may reduce security if patterns are exploited by attackers.

Section 3

Types of Perceptual Masking

3.3 Justification for Perceptual Masking

Key Idea:

Why Perceptual Masking?

Increasing the robustness of a watermark often requires increasing its embedding strength. However, higher embedding strength can result in higher perceptual visibility, which may be undesirable.

Human Visual System (HVS) Sensitivity:

- The HVS is not uniformly sensitive across different regions of an image.
- Certain regions, such as high-texture or edge areas, can tolerate stronger watermarking with minimal perceptual distortion.
- Low-texture or flat regions are more sensitive to distortions, requiring lower watermark strength to remain imperceptible.

Goal:

- Embed a stronger watermark where it is less perceptible.
- Maintain imperceptibility in sensitive regions by adapting the embedding strength.

Constants and Imports

```
import numpy as np                  # Numerical computations
import cv2                          # Image processing functions
import matplotlib.pyplot as plt  # Visualization

# Constants (defined in [0, 255] range)
NOISE_STD = 1                  # Standard deviation for binary noise ( 1 )
PSNR_MAX_VAL = 255             # Maximum possible pixel value for PSNR computation
D = 50                         # Parameter D for Noise Visibility Function (NVF)
WINDOW_SIZE = 3                # Size of the window for local variance computation
S_SIMPLE = 5                   # Embedding strength for simple stego
S1_WEIGHTED = 3                # Embedding strength for weighted stego (low NVF
    regions)
S_WEIGHTED = 9                 # Manually defined embedding strength for weighted
    stego
```

Explanation

- D controls sensitivity in the Noise Visibility Function (NVF).
- $S_{\text{simple}}, S1_{\text{weighted}}, S_{\text{weighted}}$ are embedding strengths for watermarking.
- Noise is binary with values ± 1.

Embedding Strength Scaling

```
# Scale embedding strengths for normalized image range [0, 1]
S_SIMPLE_SCALED = S_SIMPLE / 255
S1_WEIGHTED_SCALED = S1_WEIGHTED / 255
S_WEIGHTED_SCALED = S_WEIGHTED / 255
```

Explanation

- Embedding strengths are scaled to the normalized range $[0, 1]$, where the pixel values of the image are represented.

MSE Computation

```
# Function to compute Mean Squared Error (MSE)
def compute_mse(original, stego):
    mse = np.mean((original - stego) ** 2)  # L2 norm squared between
        original and stego
    return mse
```

Mathematical Formula

Mean Squared Error (MSE):

$$\text{MSE} = \frac{1}{H \cdot W} \sum_{i=1}^{H} \sum_{j=1}^{W} (x_{i,j} - y_{i,j})^2$$

where **x** is the original image, **y** is the stego image, and $H \times W$ is the image size.

Weighted MSE Computation

```
# Function to compute Weighted Mean Squared Error (Weighted MSE)
def compute_weighted_mse(original, stego, nvf):
    weighted_mse = np.mean(nvf * (original - stego) ** 2)  # NVF-weighted L2
        norm squared
    return weighted_mse
```

Mathematical Formula

Weighted Mean Squared Error (Weighted MSE):

$$\text{Weighted MSE} = \frac{1}{H \cdot W} \sum_{i=1}^{H} \sum_{j=1}^{W} \text{NVF}_{i,j} \cdot (x_{i,j} - y_{i,j})^2$$

where NVF is the noise visibility function.

PSNR and WPSNR Computation

```
# Function to compute PSNR
def compute_psnr(original, stego):
    mse = compute_mse(original, stego)
    psnr = 10 * np.log10((PSNR_MAX_VAL ** 2) / mse)  # PSNR formula with
        PSNR_MAX_VAL squared
    return psnr

# Function to compute Weighted PSNR (WPSNR)
def compute_wpsnr(original, stego, nvf):
    weighted_mse = compute_weighted_mse(original, stego, nvf)
    wpsnr = 10 * np.log10((PSNR_MAX_VAL ** 2) / weighted_mse)  # WPSNR with
        PSNR_MAX_VAL squared
    return wpsnr
```

Mathematical Formulas

PSNR:

$$\text{PSNR} = 10 \cdot \log_{10} \left(\frac{\text{PSNR\_MAX\_VAL}^2}{\text{MSE}} \right)$$

WPSNR:

$$\text{WPSNR} = 10 \cdot \log_{10} \left(\frac{\text{PSNR\_MAX\_VAL}^2}{\text{Weighted MSE}} \right)$$

Local Variance Computation

```
# Function to compute local variance
def compute_local_variance(image, window_size):
    kernel = np.ones((window_size, window_size), np.float32) / (window_size
        ** 2)
    mean_image = cv2.filter2D(image, -1, kernel)
    variance_image = cv2.filter2D(image ** 2, -1, kernel) - mean_image ** 2
    return variance_image
```

Mathematical Formula

Local variance $\sigma^2(\mathbf{x})$ is computed as:

$$\sigma^2(\mathbf{x}) = \frac{1}{W^2} \sum_{i=1}^{W} \sum_{j=1}^{W} x_{i,j}^2 - \left(\frac{1}{W^2} \sum_{i=1}^{W} \sum_{j=1}^{W} x_{i,j} \right)^2$$

where W is the window size.

NVF Computation

```
# Compute NVF (Noise Visibility Function)
local_variance = compute_local_variance(image, WINDOW_SIZE)
max_variance = np.max(local_variance)
nvf = 1 / (1 + D * local_variance / max_variance)
```

Mathematical Formula

Noise Visibility Function NVF:

$$\text{NVF} = \frac{1}{1 + D \cdot \frac{\sigma^2(\mathbf{x})}{\sigma^2_{\text{max}}}}$$

Binary Watermark Generation

```
# Generate binary watermark W (-1 and +1 noise)
watermark = np.random.choice([-1, 1], size=image.shape)
```

Explanation

- A binary watermark **w** is generated with values ± 1 using random sampling.
- The watermark has the same shape as the original image **x**.
- This binary watermark will be used for embedding into the stego image.

Stego Image Creation

```
# Simple Stego Image (S-global embedding)
stego_simple = np.clip(image + watermark * S_SIMPLE_SCALED, 0, 1)

# Weighted Stego Image
masked_watermark = (1 - nvf) * watermark * S_WEIGHTED_SCALED + nvf *
    watermark * S1_WEIGHTED_SCALED
stego_weighted = np.clip(image + masked_watermark, 0, 1)
```

Mathematical Formulas

Simple stego image $\mathbf{y}_{\text{simple}}$:

$$\mathbf{y}_{\text{simple}} = \text{clip}(\mathbf{x} + \mathbf{w} \cdot S_{\text{simple}}, 0, 1)$$

Weighted stego image $\mathbf{y}_{\text{weighted}}$:

$$\mathbf{y}_{\text{weighted}} = \text{clip}\left(\mathbf{x} + (1 - \text{NVF}) \cdot \mathbf{w} \cdot S_{\text{weighted}} + \text{NVF} \cdot \mathbf{w} \cdot S1_{\text{weighted}}, 0, 1\right)$$

PSNR and WPSNR Computation

```
# Compute PSNR and WPSNR for both simple and weighted Stego
psnr_simple = compute_psnr(image, stego_simple)
wpsnr_simple = compute_wpsnr(image, stego_simple, nvf)

psnr_weighted = compute_psnr(image, stego_weighted)
wpsnr_weighted = compute_wpsnr(image, stego_weighted, nvf)
```

Explanation

- PSNR and WPSNR are computed for the simple and weighted stego images.
- PSNR measures overall distortion, while WPSNR accounts for visibility using NVF.

Wiener Filter Formula and Prediction

Wiener Filter with Dynamic Noise Variance: For each pixel (i, j) in the image, the Wiener filter is computed as:

$$\hat{x}_{i,j} = \mu_{i,j} + \frac{\sigma_{i,j}^2}{\sigma_{i,j}^2 + \sigma_{\text{noise}}^2} (x_{i,j} - \mu_{i,j})$$

where:

- $\hat{x}$: Wiener-filtered image.
- $\mu_{i,j}$: Local mean at pixel (i, j).
- $\sigma_{i,j}^2$: Local variance at pixel (i, j).
- σ_{noise}^2: Noise variance, computed using `compute_noise_variance` in Python.

Prediction of Watermark from Image x:

$$\hat{\mathbf{w}}_{\mathbf{x}} = \mathbf{x} - \hat{\mathbf{x}}$$

where $\hat{\mathbf{x}}$ is the Wiener-filtered version of $\mathbf{x}$, using noise variance computed from the watermark.

Python Code for Wiener Filter and Prediction

```
# Function to compute noise variance dynamically
def compute_noise_variance(watermark):
    return np.var(watermark)

# Proper Wiener filter implementation with dynamic noise variance
def apply_wiener_filter(image, window_size, noise_var):
    local_mean, local_variance = compute_local_stats(image, window_size)
    wiener = local_mean + (local_variance / (local_variance + noise_var)) * (
        image - local_mean)
    return wiener

# Predict watermarks from images using adaptive noise variance
w_pred_x = image - apply_wiener_filter(image, WINDOW_SIZE,
    compute_noise_variance(watermark))
w_pred_simple = stego_simple - apply_wiener_filter(stego_simple, WINDOW_SIZE,
     compute_noise_variance(watermark))
w_pred_weighted = stego_weighted - apply_wiener_filter(stego_weighted,
    WINDOW_SIZE, compute_noise_variance(masked_watermark))
```

Explanation:

- `apply_wiener_filter` applies the Wiener filter using local statistics and the dynamically computed noise variance.
- `w_pred_x`, `w_pred_simple`, and `w_pred_weighted` are the predicted watermarks from $\mathbf{x}$, $\mathbf{y}_{\text{simple}}$, and $\mathbf{y}_{\text{weighted}}$.

Mathematical Formulas for Watermark Prediction

Prediction from Image x:

$$\hat{\mathbf{w}}_{\mathbf{x}} = \mathbf{x} - \hat{\mathbf{x}}$$

Prediction from Simple Stego Image $\mathbf{y}_{\text{simple}}$:

$$\hat{\mathbf{w}}_{\text{simple}} = \mathbf{y}_{\text{simple}} - \hat{\mathbf{y}}_{\text{simple}}$$

Prediction from Weighted Stego Image $\mathbf{y}_{\text{weighted}}$:

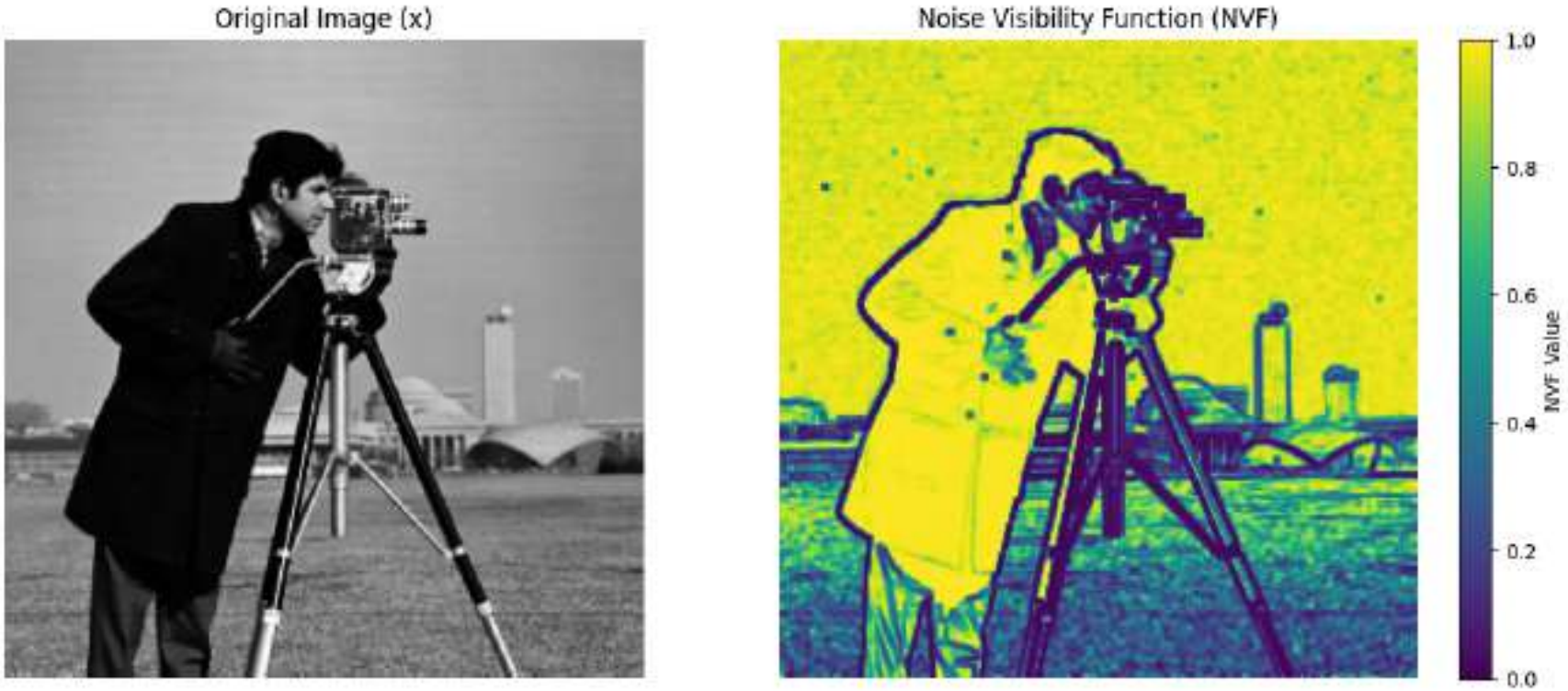
$$\hat{\mathbf{w}}_{\text{weighted}} = \mathbf{y}_{\text{weighted}} - \hat{\mathbf{y}}_{\text{weighted}}$$

Wiener Filter Application:

$$\hat{y}_{i,j} = \mu_{i,j} + \frac{\sigma_{i,j}^2}{\sigma_{i,j}^2 + \sigma_{\text{noise}}^2} (y_{i,j} - \mu_{i,j})$$

where $\mathbf{y}_{\text{simple}}$ and $\mathbf{y}_{\text{weighted}}$ are the stego images.

Noise Visibility Function (NVF)



NVF with values ranging from 0 to 1.

Parameters: $D = 150$ and $WINDOW = 3$

Based on the code: Lecture 3 2025-01-13 WM with scalled noise in Wiener

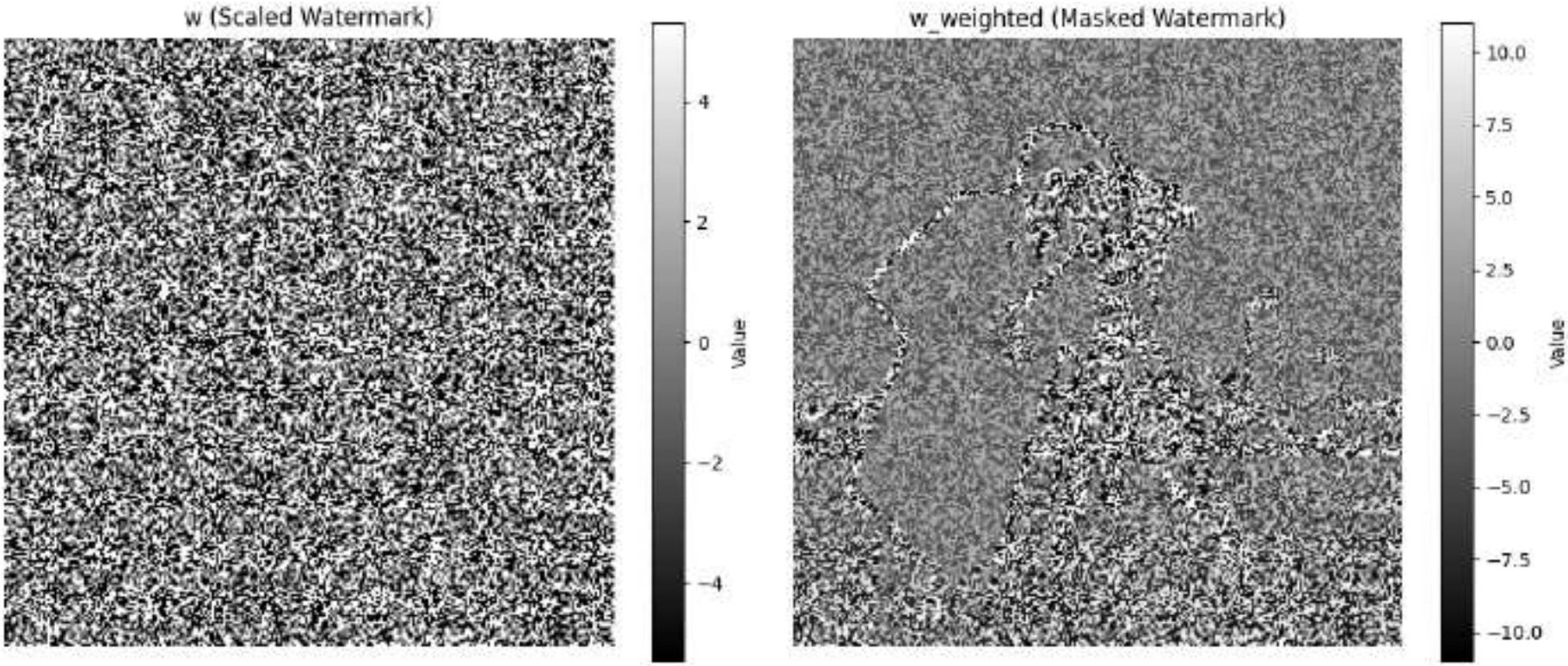
Original Image $\mathbf{x}$, $\mathbf{y}_{\text{simple}}$, and $\mathbf{y}_{\text{weighted}}$

Visualization of the original image $\mathbf{x}$, simple stego image $\mathbf{y}_{\text{simple}}$, and weighted stego image $\mathbf{y}_{\text{weighted}}$.

Parameters:

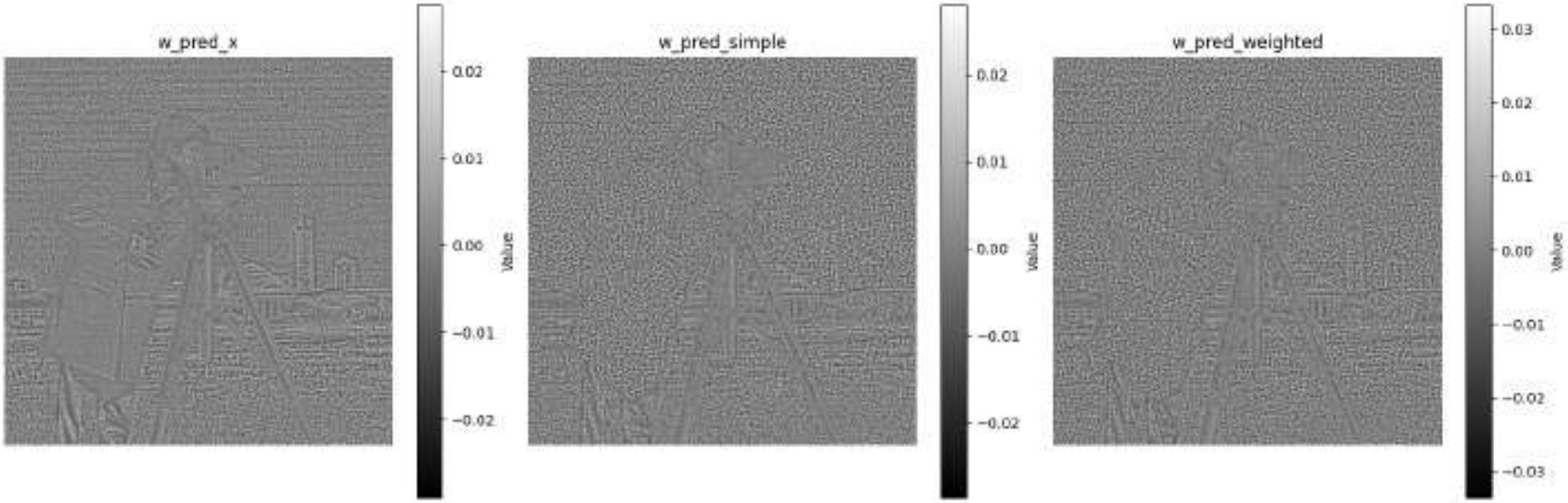
$$S_{\text{simple}} = 5.3, S_{\text{weighted}} = 11.0, 5, S1_{\text{weighted}} = 3.8$$

Watermark $\mathbf{w}$ and Masked Watermark $\mathbf{w}_{\text{weighted}}$



Visualization of the watermark $\mathbf{w}$ and the masked watermark $\mathbf{w}_{\text{weighted}}$.

Predicted Watermarks from $\mathbf{x}$, $\mathbf{y}_{\text{simple}}$, and $\mathbf{y}_{\text{weighted}}$



Visualization of the predicted watermarks from the original image $\hat{\mathbf{w}}_{\mathbf{x}}$, simple stego $\hat{\mathbf{w}}_{\text{simple}}$, and weighted stego $\hat{\mathbf{w}}_{\text{weighted}}$.

Inner Product and Cosine Similarity Computation

```
# Generic function to compute inner product and cosine similarity
def compute_metrics(a, b):
    inner_product = np.dot(a.flatten(), b.flatten())
    cosine_sim = np.dot(a.flatten(), b.flatten()) / (np.linalg.norm(a) * np.
        linalg.norm(b))
    return inner_product, cosine_sim
```

Mathematical Formulas

Inner product:

$$\langle \mathbf{a}, \mathbf{b} \rangle = \sum_{i=1}^{H} \sum_{j=1}^{W} a_{i,j} \cdot b_{i,j}$$

Cosine similarity:

$$\cos(\theta) = \frac{\langle \mathbf{a}, \mathbf{b} \rangle}{\|\mathbf{a}\| \cdot \|\mathbf{b}\|}$$

where $\mathbf{a}, \mathbf{b}$ represent images or watermarks of size $H \times W$.

Inner Product and Cosine Similarity Results

Comparison	Inner Product	Cosine Similarity
$\mathbf{x}$ vs $\mathbf{w}$	78.25	0.0023
$\mathbf{y}_{\text{simple}}$ vs $\mathbf{w}$	1440.36	0.0418
$\mathbf{y}_{\text{weighted}}$ vs $\mathbf{w}$	1633.10	0.0473
$\mathbf{x}$ vs $\mathbf{w}_{\text{weighted}}$	0.38	0.0004
$\mathbf{y}_{\text{simple}}$ vs $\mathbf{w}_{\text{weighted}}$	32.71	0.0368
$\mathbf{y}_{\text{weighted}}$ vs $\mathbf{w}_{\text{weighted}}$	43.85	0.0493
$\hat{\mathbf{w}}_{\mathbf{x}}$ vs $\mathbf{w}$	-2.53	-0.0068
$\hat{\mathbf{w}}_{\text{simple}}$ vs $\mathbf{w}$	418.75	0.7279
$\hat{\mathbf{w}}_{\text{weighted}}$ vs $\mathbf{w}$	491.85	**0.7427**
$\hat{\mathbf{w}}_{\mathbf{x}}$ vs $\mathbf{w}_{\text{weighted}}$	-0.06	-0.0059
$\hat{\mathbf{w}}_{\text{simple}}$ vs $\mathbf{w}_{\text{weighted}}$	7.60	0.5126
$\hat{\mathbf{w}}_{\text{weighted}}$ vs $\mathbf{w}_{\text{weighted}}$	9.58	**0.5616**

Table: Results of Inner Product and Cosine Similarity for Different Comparisons. The highest cosine similarity is highlighted in **bold**.

Conclusions from (w)PSNR and Robustness Metrics

- **Lower PSNR for simple embedding**: The PSNR for **weighted embedding** is lower (**31.79 dB**) compared to **simple embedding** (**33.65 dB**), indicating stronger distortions in MSE sense..
- **Higher wPSNR for weighted embedding**: Weighted PSNR (**WPSNR**) for **weighted embedding** is slightly higher (**35.88 dB**) than for **simple embedding** (**35.28 dB**), confirming improved perceptual quality.
- **Superior robustness metrics for weighted embedding**: All similarity metrics (inner product and cosine similarity) demonstrate superior performance for the **weighted watermark** compared to the simple one.
- **Significant difference in non-watermarked image metrics**: Metrics for the **non-watermarked image** (e.g., cosine similarity) are two orders of magnitude lower compared to the watermarked images, ensuring reliable detection and distinction of watermarked content.
- **Key insight**: Weighted embedding using perceptual masking significantly improves both imperceptibility and robustness, making it highly effective for practical watermarking applications.

Section 3

Types of Embedding Domains

3.4 Types of Embedding Domains

Overview of Embedding Domains in Digital Watermarking:

- **Direct (Pixel Domain) Watermarking:** No transformation applied; watermark embedding occurs directly in the pixel domain.
- **Fixed, Non-Learnable Transforms:** Commonly used handcrafted transforms include:
  - Discrete Cosine Transform (DCT)
  - Discrete Fourier Transform (DFT)
  - Discrete Wavelet Transform (DWT)
- **Learnable but Linear Transforms:** Principal Component Analysis (PCA)
- **Learnable Nonlinear Transforms:**
  - Autoencoder (Learnable encoder-decoder structure trained on distortions)
- **Learnable Foundation Models:**
  - Latent space of pre-trained foundation models (e.g., large vision models trained on diverse datasets)

3.4 Fixed or Non-Learnable Transforms

Digital Watermarking in Fixed Transform Domains:

- Most commonly used transforms are orthogonal but not invariant to geometrical transformations.
- Examples:
  - **DCT, DFT, DWT:** Popular due to efficient computation but sensitive to affine distortions.
- Exception: **Fourier Magnitude**—Invariant to translation.
- Specialized transform: **Fourier-Mellin Transform**—Invariant to both rotation and translation, but lacks general applicability for generic affine transformations.

Limitation: Sensitivity to affine transformations limits robustness for practical applications.

3.4 Fixed or Non-Learnable Transforms (Cont'd)

Digital Watermarking in Compressed Transform Domains:

- Despite sensitivity to geometrical transformations, transforms such as **DCT** and **DWT** are widely used in popular compression algorithms.
- Basis for many standards:
  - **JPEG**, **JPEG 2000**, and proprietary formats like **HEIC** (Apple).
- **Advantage:** Watermarking in these compressed domains leverages knowledge of the compression process, offering better integration and efficiency.

Perceptual Masking in Transform Domains:

- Frequency decomposition in **DCT** and **DWT** domains allows extending perceptual masking techniques beyond the pixel domain.
- Significantly lowers PSNR without noticeable visual degradations.

Next: Examples will demonstrate these advantages.

S. Voloshynovskiy, F. Deguillaume, T. Pun, Content adaptive watermarking based on a stochastic multiresolution image modeling

https://cvml.unige.ch/publications/postscript/2000/VoloshynovskiyDeguillaumePun_es2000.pdf

Examples of DWT Domain Embedding Based on NVF

Examples of DWT Domain Embedding Based on NVF

- **First Image:** Original image Barbara
- **Second Image:** Watermarked image with PSNR 24.6 dB and wPSNR 26.4 dB
- **Third Image:** Watermarked image with Perceptually Adapted Noise with PSNR 24.6 dB and wPSNR 29.3 dB

Examples of DWT Domain Embedding Based on NVF

Examples of DWT Domain Embedding Based on NVF

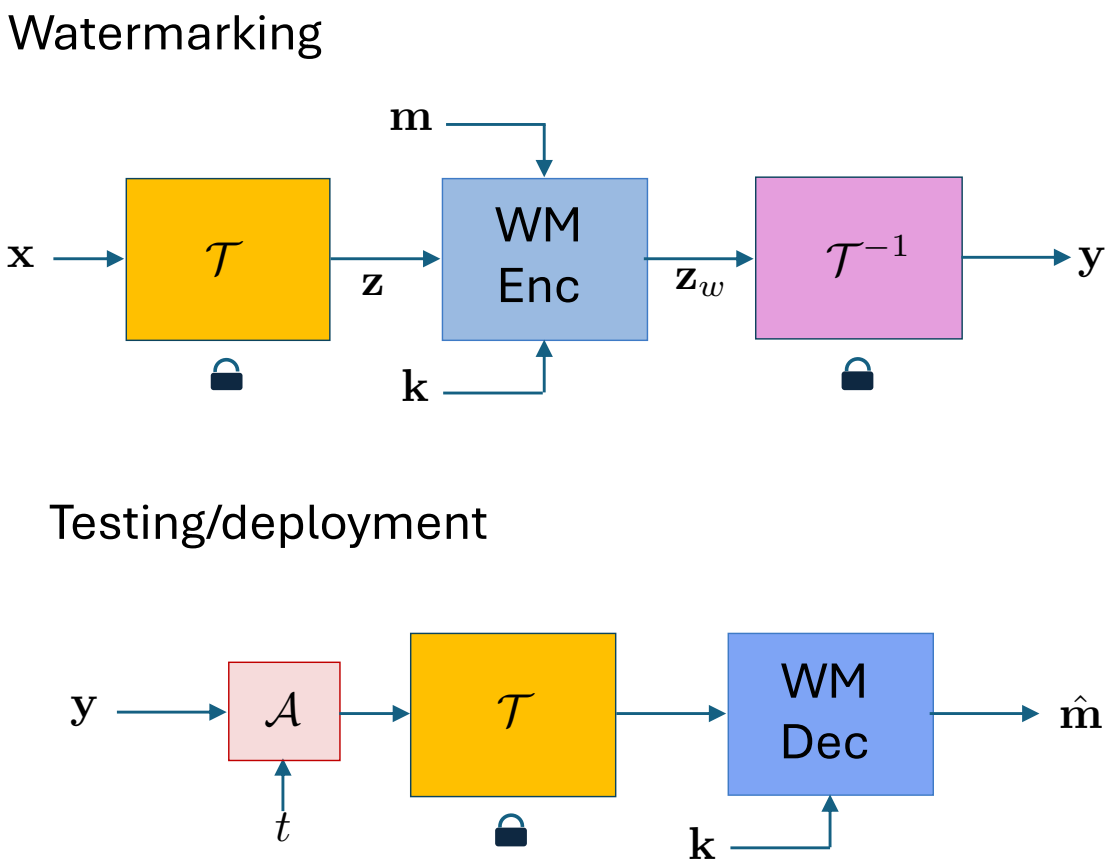
- **First Image:** PSNR 33 dB
- **Second Image:** PSNR 37 dB
- **Third Image:** Original Image

Examples of DWT Domain Embedding Based on NVF

Examples of DWT Domain Embedding Based on NVF

- **First Image:** PSNR 37 dB
- **Second Image:** Original Image
- **Third Image:** PSNR 35 dB

3.4 Fixed, Non-Learnable Transform Watermarking



Description: Watermarking in fixed transform domains, such as DCT, DFT, and DWT, involves embedding in the frequency or wavelet domain. These transforms are orthogonal but typically sensitive to geometrical distortions.

3.4 Learnable Transform Domain – PCA

Principal Component Analysis (PCA):

- PCA is a learnable but **linear** transform domain.
- It provides dimensionality reduction and energy compaction.
- However, similar to DCT, DFT, and DWT, PCA is:
  - Linear and thus cannot ensure robustness to complex transformations.
  - Not commonly used for watermarking due to its sensitivity to geometrical distortions.

Limitation: While PCA offers energy compaction, it is not effective in scenarios involving complex image transformations.

3.4 Learnable Transform Domain – Autoencoders

Autoencoders:

- Autoencoders consist of a learnable encoder-decoder structure.
- They can be trained on various types of distortions to ensure robustness during watermark extraction.
- The encoder learns an embedding domain suitable for watermarking, while the decoder reconstructs the image.

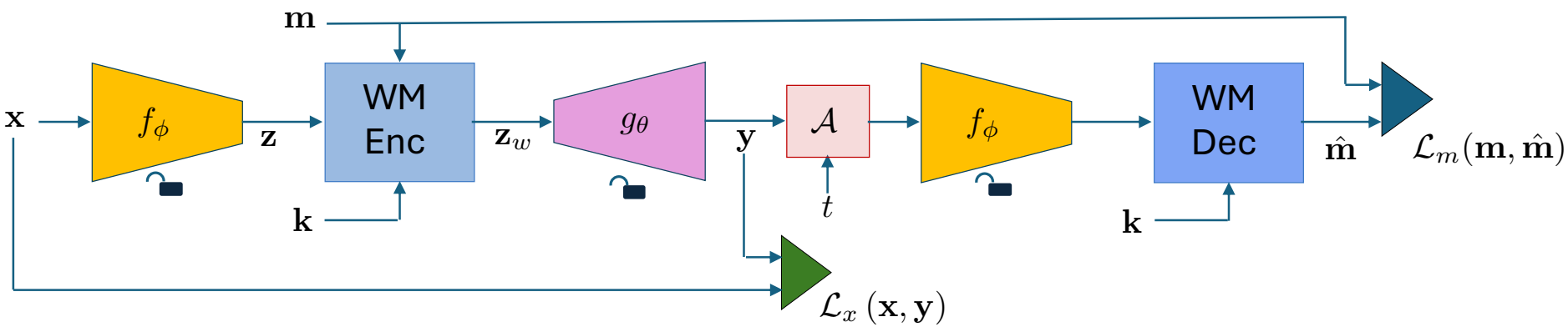
Advantages:

- Robust to noise and minor geometric distortions when properly trained.
- Flexible and adaptable for different application requirements.

Limitation: Requires significant training effort and careful tuning for optimal robustness.

3.4 Learnable Transform Watermarking – Autoencoder

Training AE encoder and decoder

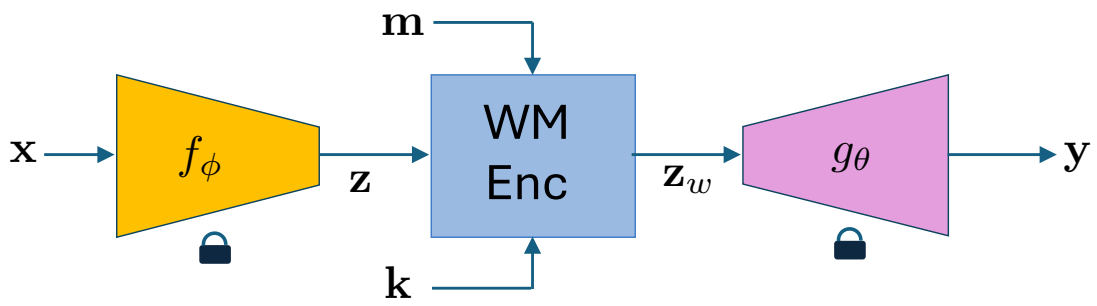


Description: At the watermarking and testing stages, the encoder and decoder are fixed.

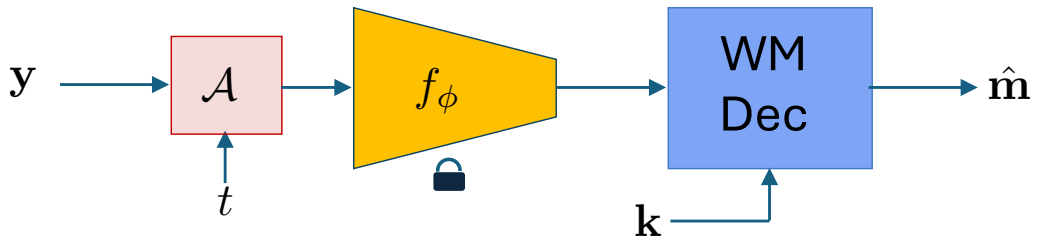
https://arxiv.org/pdf/1807.09937.pdf

3.4 Learnable Transform Watermarking – Autoencoders

Watermarking



Testing/deployment



Description: Denoising autoencoders provide a learnable transform domain where both the encoder and decoder can be trained under various distortions, ensuring robustness during watermark extraction.

3.4 Learnable Transform Domain – Foundation Models

Latent Space of Foundation Models:

- Foundation models, pre-trained on large datasets, offer latent spaces that can be used as embedding domains.
- These models are robust to a broad spectrum of distortions due to extensive pre-training.

Advantages:

- High robustness to common distortions.
- Can leverage pre-trained models without extensive retraining.

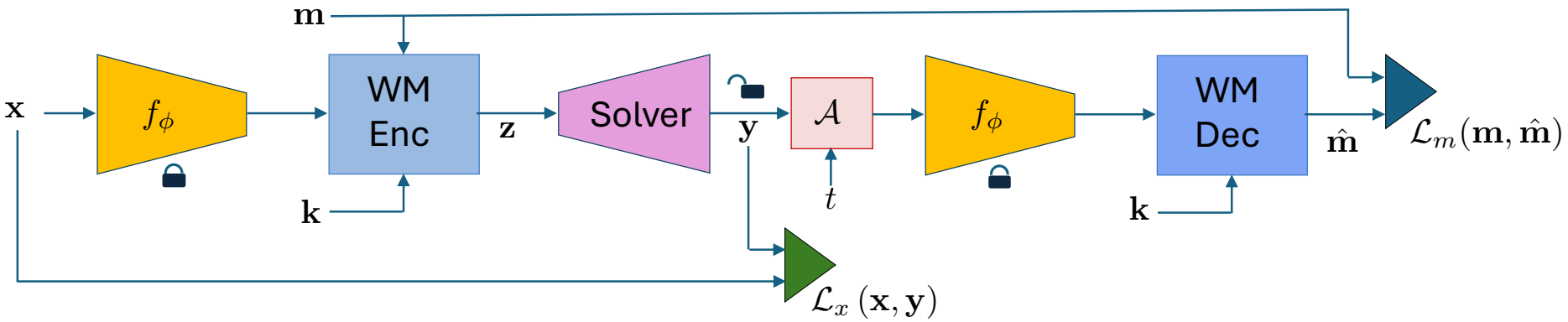
Limitations:

- Solvers used for embedding are computationally intensive.
- Vulnerable to adversarial attacks—latent space manipulation can compromise watermark extraction.

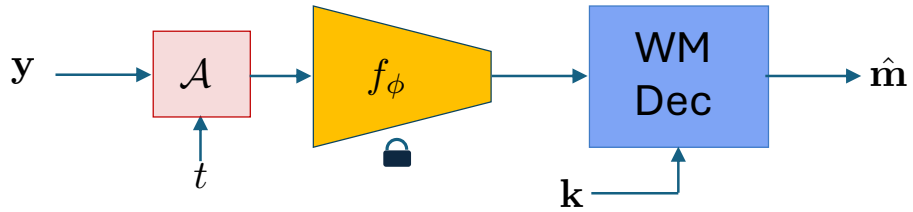
Note: Further exploration of adversarial robustness will be covered later.

3.4 Learnable Transform Watermarking – Foundation Models

Watermarking



Testing/deployment



f_ϕ is a vision foundation model

Description: Foundation models, trained on large datasets, offer robust latent spaces for watermark embedding. Despite their robustness, computational complexity and susceptibility to adversarial attacks remain significant challenges.

https://arxiv.org/pdf/2112.09581.pdf

Section 5

Types of Geometrical Synchronizations

3.5 Types of Geometrical Synchronization Techniques

Overview of Methods:

1. **Hand-crafted invariant domain watermarking**
   Methods leveraging Fourier-Mellin Transform to achieve invariance to geometric transformations such as rotation, scaling, and translation.
2. **Feature-Based Synchronization**
   Methods based on detecting and using external feature points (e.g., SIFT, ORB) to estimate and reverse geometric distortions.
3. **Self-Synchronization Techniques**
   Methods embedding a repetitive watermark structure to enable synchronization using autocorrelation properties.
4. **Foundation Models for Invariant Representations**
   Leveraging pre-trained invariants of foundation models (e.g., CLIP, DINO) to handle certain geometric transformations in the latent space.

Goal: To explore synchronization techniques and invariant representations, focusing on robustness to geometric distortions and practical applicability.

3.5 Handcrafted Invariant Domains in Digital Watermarking

Main Idea

Invariant Domain for Watermarking: The goal of handcrafted invariant domains is to find a mathematical domain where the image representation remains invariant to generic geometric transformations. This helps ensure robust watermark detection under distortions.

Key Statements

- The magnitude of the **Fourier Transform** is invariant to **translations**.
- The **Fourier-Mellin Transform** provides invariance to **rotation**, **scaling**, and **translation**.

3.5 Fourier Transform Magnitude Invariance to Translation

Key Concept:

Translation Invariance of Fourier Magnitude

The magnitude of the Fourier Transform of an image is invariant to translations. This property can be used to embed and detect watermarks robustly under translational distortions.

Mathematical Details:

- Given an image $\mathbf{x}_{i,j}$, its Fourier Transform is:

$$\mathbf{F}_{u,v} = \mathcal{F}\{\mathbf{x}_{i,j}\}$$

- If the image is translated by $(\Delta i, \Delta j)$, the Fourier magnitude remains unchanged, while the phase is modified.

3.5 Fourier Magnitude Invariance to Translation

Fourier Transform of Image $\mathbf{X}_{i,j}$:

$$\mathbf{F}_{u,v} = \sum_{i=0}^{M-1} \sum_{j=0}^{N-1} \mathbf{x}_{i,j} e^{-2\pi i \left(\frac{ui}{M} + \frac{vj}{N} \right)}$$

where $\mathbf{F}_{u,v}$ denotes the complex-valued Fourier transform of the image $\mathbf{X}_{i,j}$.

Magnitude Invariance to Translation:

- When the image $\mathbf{x}_{i,j}$ undergoes a translation $\mathbf{X'}_{i,j} = \mathbf{x}_{i+\Delta_i, j+\Delta_j}$, the Fourier magnitude remains unchanged:

$$|\mathbf{F'}_{u,v}| = |\mathbf{F}_{u,v}|$$

- Translation affects only the **phase** of the Fourier transform, but not its magnitude.

Key Insight: By using the Fourier magnitude, we can achieve invariance to image translations, making it useful for synchronization in the presence of shifts.

3.5 Fourier-Mellin Transform for RST Invariance

Key Concept:

Fourier-Mellin Transform for RST Invariance

The Fourier-Mellin Transform (FMT) converts scaling and rotation into translations in the log-polar coordinate system, providing invariance to rotation, scaling, and translation.

Mathematical Details:

- Apply the Fourier Transform to the image $\mathbf{x}_{i,j}$.
- Convert the result to log-polar coordinates to transform scaling and rotation into translations.
- Normalize the magnitude of the transformed image to achieve invariance.

3.5 Fourier-Mellin Transform for RST Invariance

Fourier-Mellin Transform (FMT) Steps:

1. Apply the 2D Fourier Transform to the image $\mathbf{X}_{i,j}$ to obtain $\mathbf{F}_{u,v}$.
2. Convert to log-polar coordinates:

$$(u, v) \rightarrow \left(\log \sqrt{u^2 + v^2}, \tan^{-1}\left(\frac{v}{u}\right)\right)$$

3. Apply a 1D Fourier Transform along the angular direction to handle rotations.

Mathematical Invariance Properties:

- **Scaling Invariance:** A scaling factor s in the spatial domain corresponds to a shift in the log-polar domain.
- **Rotation Invariance:** A rotation by an angle θ results in a cyclic shift along the angular axis.

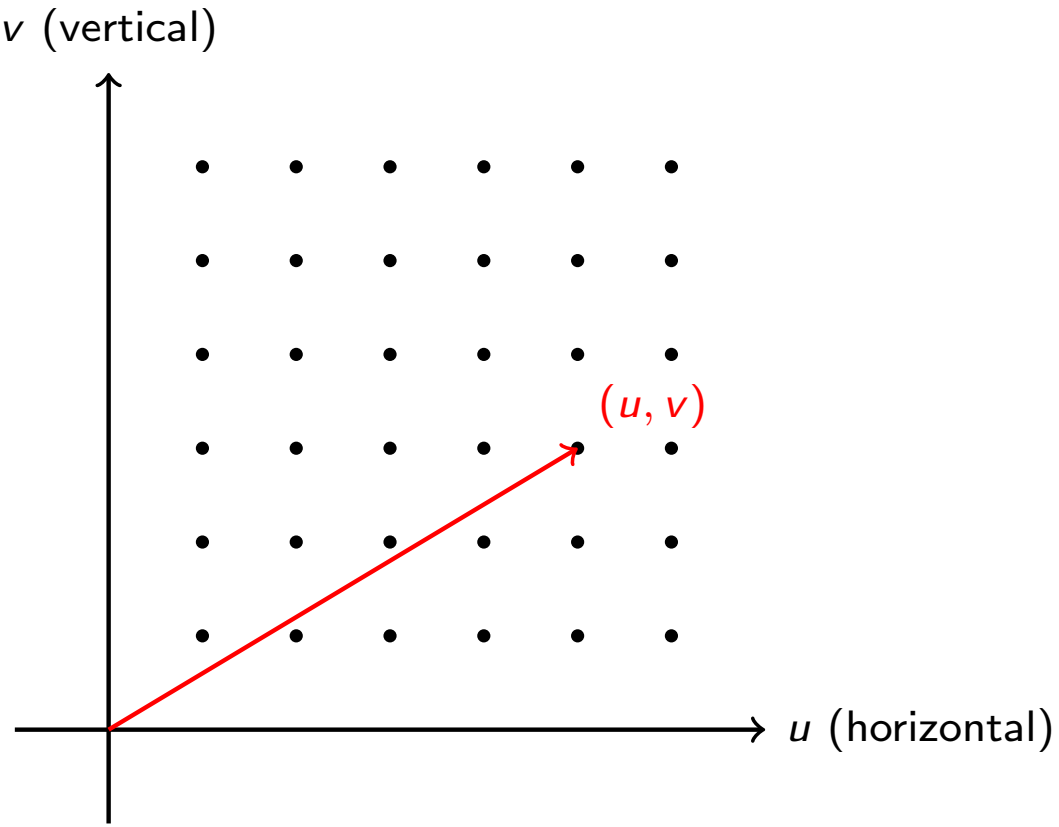
$$\mathbf{F}_{\text{FMT}}(\rho, \phi) = \mathcal{T}\left\{\mathbf{F}_{\log \rho, \phi}\right\}$$

where $\mathcal{T}$ denotes the 1D Fourier transform along the angular axis.

Conclusion: The Fourier-Mellin Transform provides a robust approach for achieving invariance to **rotation**, **scaling**, and **translation**, making it a powerful tool for geometrical synchronization.

3.5 Cartesian Coordinate System

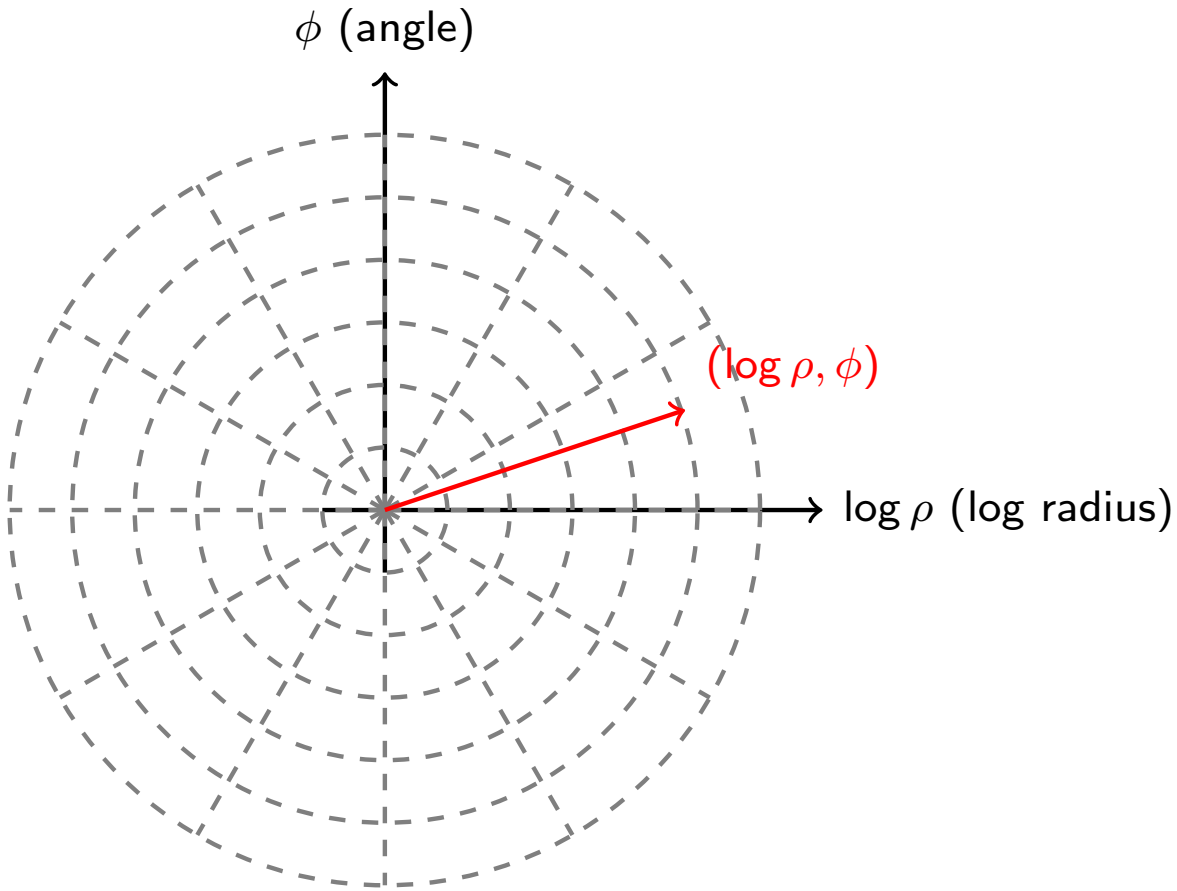
Cartesian Coordinate System



Explanation: In the Cartesian coordinate system, a point is represented by its horizontal and vertical distances (u, v) from the origin.

3.5 Log-Polar Coordinate System

Log-Polar Coordinate System



Explanation: In the log-polar coordinate system, the point (u, v) is transformed into $(\log \rho, \phi)$, where:

- $\rho = \sqrt{u^2 + v^2}$ is the radial distance.
- $\phi = \tan^{-1}\left(\frac{v}{u}\right)$ is the angle.

3.5 Practical Illustration of Fourier-Mellin Transform

Illustration: The image below demonstrates the application of the Fourier-Mellin Transform (FMT).

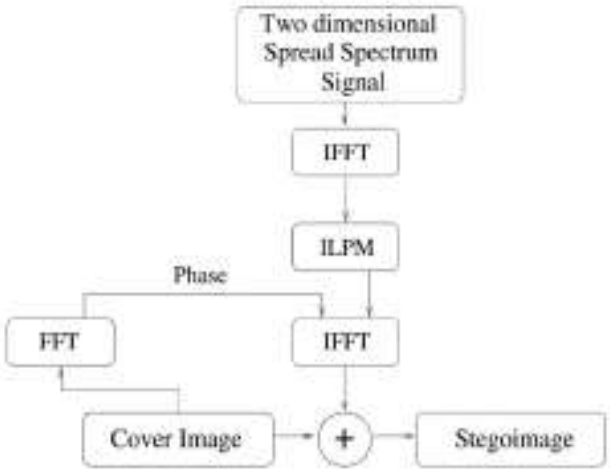


Fig. 7. A method of embedding a mark in an image which avoids mapping the original image into the RST invariant domain. The terms 'cover image' and 'stego image' denote the original image and watermarked image, respectively [18].

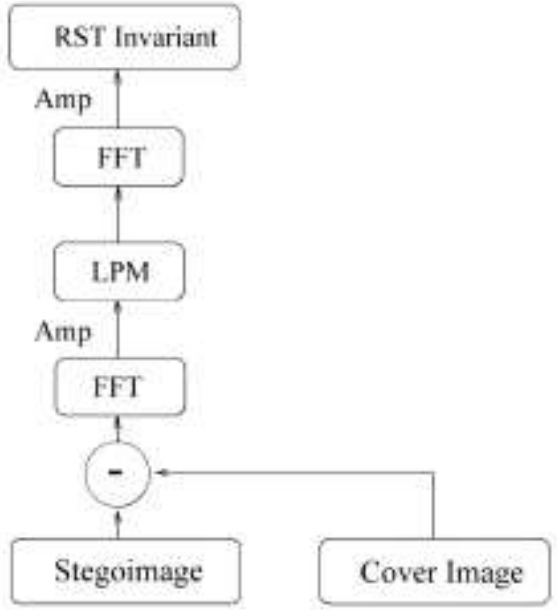


Fig. 8. A scheme to extract a mark from an image.

Reference: For more details, refer to the original work:
https://www.researchgate.net/profile/Joseph-O-Ruanaidh-2/publication/350087153_Rotation_scale_and_translation_invariant_spread_spectrum_digital_image_watermarking/links/

3.5 Practical Illustration of Fourier-Mellin Transform

Fig. 9. A standard 512 × 512 image of Lena heavily watermarked using the Fourier–Mellin transform.

Fig. 10. A Fourier–Mellin watermarked image of Lena that has been rotated by 45°, scaled by 75% and compressed using JPEG. The embedded mark was recovered from this image.

Fig. 11. A log polar map of the image 'Lena'. The log-polar map employs bilinear interpolation and the log-polar grid is 512 × 512 samples.

Fig. 12. The image of Lena reconstructed from a log polar map of size 128 × 128 samples. This reconstruction uses nearest-neighbour interpolation.

3.5 Limitations of Fourier-Mellin Transform

Limitations:

- The Fourier-Mellin Transform provides invariance only to **rotation**, **scaling**, and **translation**. It does not cover more complex distortions such as cropping, shearing, or perspective transformations.
- Applying the inverse transform results in significant degradation of image quality, making it unsuitable for high-quality watermarking.
- Sensitive to localized noise and distortions.

Conclusion: While the Fourier-Mellin Transform offers a promising theoretical framework for achieving geometric invariance, its practical applicability is limited due to image degradation and sensitivity to noise.

3.5 Feature-Based Synchronization

Key Concept:

Feature-Based Synchronization Techniques

These methods rely on detecting key feature points (e.g., SIFT, ORB) and using their correspondences to estimate and reverse geometric distortions.

Mathematical Details:

- Detect feature points $\{p_k\}_{k=1}^{K}$ in the original and distorted images using SIFT or ORB.
- Compute point correspondences and estimate the geometric transformation $\mathcal{T}$ using RANSAC.
- Apply the inverse transformation $\mathcal{T}^{-1}$ to synchronize the distorted image.

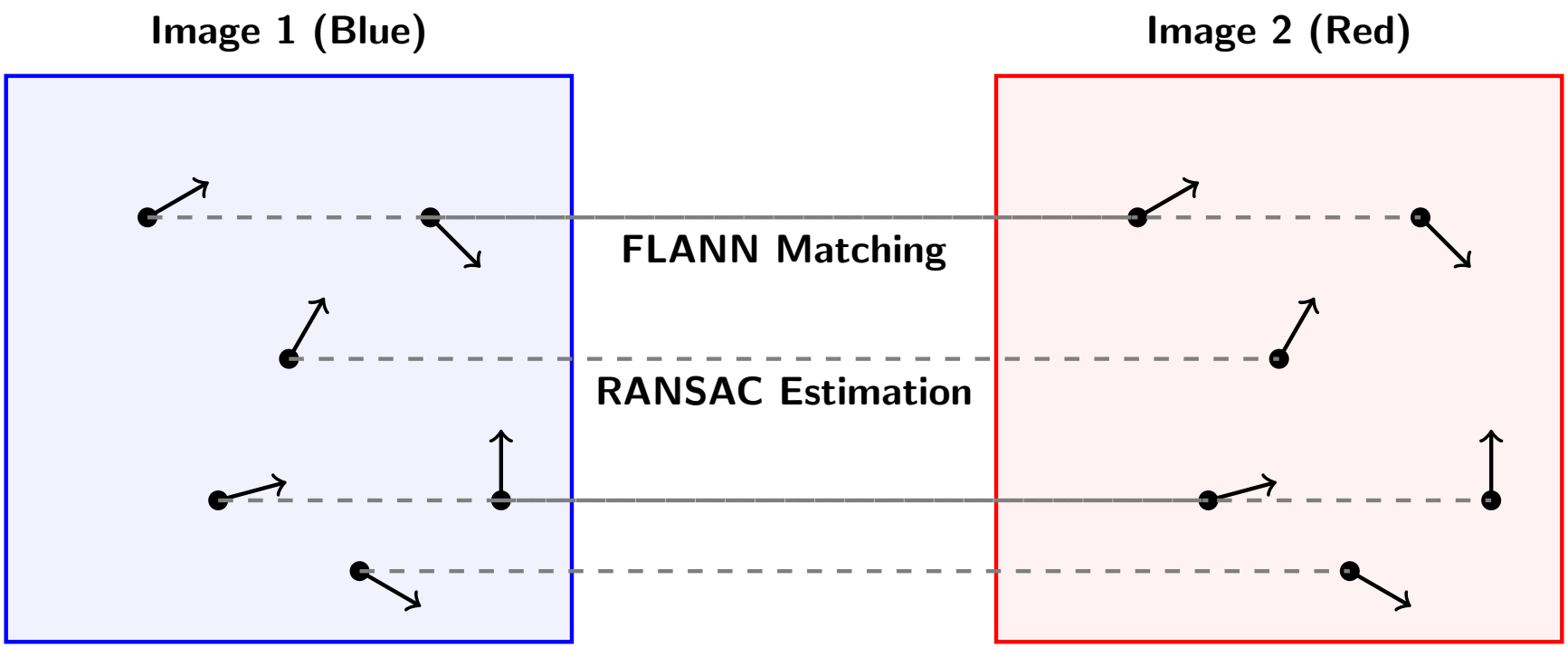
Advantages:

- Robust to complex geometric transformations.
- Effective under various real-world distortions.

Limitations:

- Computationally expensive.
- Requires sufficient distinct feature points for accurate synchronization.

3.5 Image Matching using Local Descriptors and RANSAC



Explanation: The diagram illustrates how keypoints with local descriptors (e.g., SIFT or ORB) can be extracted from two images. Corresponding keypoints are matched using techniques like **FLANN**, and geometric consistency is ensured using **RANSAC** to estimate transformations (e.g., rotation, scaling, and translation).

3.5 Affine Transformation of Points

Affine Transformation of a Point:

- Given a point $\mathbf{p}_k = (i, j)^\top$ in the original image, an affine transformation $\mathcal{T}$ is defined as:

$$\mathcal{T}(i, j) = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} i \\ j \end{bmatrix} + \begin{bmatrix} t_1 \\ t_2 \end{bmatrix}$$

  where a, b, c, d are scaling and rotation parameters, and t_1, t_2 are translation parameters.

- The resulting point in the transformed image is:

$$\mathbf{p}'_k = (w, z)^\top = \mathcal{T}(i, j)$$

Goal: Estimate the affine transformation parameters (a, b, c, d, t_1, t_2) given a set of matched points $\{\mathbf{p}_k\}_{k=1}^N$ in both images using a robust algorithm like RANSAC.

3.5 Pseudo Algorithm for Point Matching

Pseudo Algorithm for Matching and Estimating Transformation:

1. **Detect key points:** Use a feature detector (e.g., SIFT, ORB) to identify key points $\mathbf{p}_k$ in both images.
2. **Extract descriptors:** Compute local descriptors around each key point to capture its local neighborhood.
3. **Match descriptors:** Use a matching algorithm (e.g., FLANN) to find correspondences between descriptors in both images based on Euclidean distance.
4. **Estimate transformation using RANSAC:**
    1. Randomly sample a subset of matched points.
    2. Compute the affine transformation $\mathcal{T}$ for the sampled points.
    3. Compute the inliers—points that satisfy the transformation within a small threshold.
    4. Repeat until the transformation with the maximum number of inliers is found.
5. **Output:** The estimated affine transformation parameters (a, b, c, d, t_1, t_2).

3.5 Matching and RANSAC

Step 1: Detecting and Matching Points

- Use SIFT or ORB to detect key points $\{\mathbf{p}_k\}_{k=1}^{N}$ in both images.
- Extract descriptors $\mathbf{d}_k$ for each key point.
- Use FLANN (Fast Library for Approximate Nearest Neighbors) to match descriptors by minimizing their Euclidean distance.

Step 2: Robust Estimation using RANSAC

- RANSAC algorithm iteratively samples subsets of matched points and computes affine transformations.
- For each sampled transformation, count the number of inliers—points for which the transformed coordinates are within a threshold distance of their corresponding points.
- The transformation with the maximum number of inliers is selected as the final affine transformation.

3.5 Summary of Point Matching Approach

Key Steps in the Approach:

1. **Key Point Detection:** Identify feature-rich regions in both images using SIFT, ORB, or another feature detector.
2. **Descriptor Matching:** Find corresponding points by matching descriptors using FLANN.
3. **Affine Transformation Estimation:** Use RANSAC to estimate the affine transformation while eliminating outliers.

Key Insight: By combining robust feature detection, descriptor matching, and RANSAC-based estimation, we can achieve accurate point correspondences and geometric transformations even in the presence of noise and outliers.

Summary: Feature-Based Synchronization for Image Matching

Strengths:

- Feature-based synchronization using local descriptors works **fantastically well** when a **known reference image** is available.
- It provides **robust matching** under any **generic geometrical transformations** (e.g., scaling, rotation, translation).
- Perfectly justifiable when **reference points** for synchronization are clearly defined.

Limitations:

- The approach requires **exact prior knowledge** of the reference image.
- If the image is unknown and there are N images in a database, matching each image to a reference is **computationally expensive** and **non-scalable** in practice.

Conclusion:

- Self-synchronization techniques are **robust and accurate** when exact reference images are known.
- For **practical scalability**, other techniques must be explored when reference images are unknown.

3.5 Self-Synchronization Techniques

Key Concept:

Self-Synchronization Techniques

These methods embed a repetitive watermark structure, enabling synchronization by estimating the watermark via denoising and using autocorrelation properties.

Steps:

1. Embed a repetitive watermark $\mathbf{w}$ over the image.
2. Estimate the predicted watermark $\hat{\mathbf{w}}_{\text{pred}}$ by subtracting the Wiener-filtered version of the image $\mathbf{y}$ from the observed image $\mathbf{y}$:

$$\hat{\mathbf{w}}_{\text{pred}} = \mathbf{y} - \text{Wiener}(\mathbf{y})$$

3. Compute the autocorrelation function of $\hat{\mathbf{w}}_{\text{pred}}$ to estimate geometric distortions.

Advantages:

- Inherently robust to various geometric distortions.
- Does not require external feature detection.

3.5 Overview of Watermark Embedding Process

Overview:

- The goal is to embed a watermark **w** into an image **x** to produce a stego image **y**.
- The watermark **w** is generated as a binary block, upsampled, and tiled to match the size of the image.
- The embedding model follows:

$$\mathbf{y} = \mathbf{x} + \alpha\mathbf{w}, \quad \text{where } \alpha \text{ is the embedding strength.}$$

- The image may undergo attacks such as noise addition, downsampling, and rotation.

https://cvml.unige.ch/publications/postscript/2001/VoloshynovskiyDeguillaumePun_ICIP2001.pdf

3.5 Wiener Filtering for Watermark Prediction

Wiener Filtering for Prediction:

- The Wiener filter estimates the local mean and variance of the image to reduce noise.
- The predicted watermark is computed as:

$$\mathbf{w}_{\text{pred}} = \mathbf{y} - \text{Wiener}(\mathbf{y}, \text{window size } = 9).$$

Watermark Prediction under Different Attacks:

- Noise attack: $\mathbf{y}_{\text{attack}} - \text{Wiener}(\mathbf{y}_{\text{attack}})$.
- Downsampling attack: Rescale $\mathbf{y}$ and apply the Wiener filter.
- Rotation attack: Rotate $\mathbf{y}$ back and apply the Wiener filter.

3.5 Autocorrelation for Watermark Detection

Autocorrelation:

- Autocorrelation measures the similarity of a signal with a shifted version of itself.
- The autocorrelation of the watermark is computed using FFT:

$$\text{Auto}(\mathbf{w}) = \mathcal{F}^{-1}(\mathcal{F}(\mathbf{w}) \cdot \overline{\mathcal{F}(\mathbf{w})}),$$

  where $\mathcal{F}$ denotes the Fourier transform.

Application: Autocorrelation helps in detecting repetitive patterns of the watermark, even under noise and geometric distortions.
Code: 2025-01-15 Geometrical robustness 0-bit based n ACF with enahced visualization.py

Constants and Initialization

```
# Constants
BLOCK_SIZE = 16
UPSAMPLE_FACTOR = 2
EMBEDDING_STRENGTH = 4.5
ATTACK_NOISE_STD = 30
K = 42
WIENER_WINDOW_SIZE = 9
PSNR_MAX_VAL = 255
```

Explanation

- **BLOCK_SIZE**: Size of the initial watermark block (16×16).
- **UPSAMPLE_FACTOR**: Factor by which the watermark block is upsampled.
- **EMBEDDING_STRENGTH**: Coefficient controlling watermark visibility.
- **ATTACK_NOISE_STD**: Standard deviation of additive noise during attack.
- **WIENER_WINDOW_SIZE**: Size of the Wiener filter window.

Functions for Watermark Generation

```
def generate_block_watermark(block_size, seed):
    np.random.seed(seed)
    return np.random.choice([-1, 1], (block_size, block_size))

def upsample_block(block, upsample_factor):
    return cv2.resize(block, (block.shape[1] * upsample_factor,
                              block.shape[0] * upsample_factor),
                      interpolation=cv2.INTER_NEAREST)

def tile_watermark(block, image_shape):
    tiled_rows = (image_shape[0] + block.shape[0] - 1) // block.shape[0]
    tiled_cols = (image_shape[1] + block.shape[1] - 1) // block.shape[1]
    return np.tile(block, (tiled_rows, tiled_cols))[:image_shape[0], :
        image_shape[1]]
```

Explanation

- **generate_block_watermark**: Generates a random watermark block.
- **upsample_block**: Upsamples the watermark block to match the embedding resolution.
- **tile_watermark**: Tiles the upsampled watermark to match the image dimensions.

Watermark Embedding and Variants of Y

```
# Embed watermark into the original image
y = np.clip(x + w * EMBEDDING_STRENGTH, 0, 255)

# Generate noisy stego image
z = np.random.normal(0, ATTACK_NOISE_STD, x.shape)
y_attack = np.clip(y + z, 0, 255)

# Generate downsampled stego image
y_downsampled = cv2.resize(y, (x.shape[1] // 2, x.shape[0] // 2),
                           interpolation=cv2.INTER_LINEAR)

# Generate rotated stego image
y_rotated = cv2.warpAffine(y, cv2.getRotationMatrix2D(
                           (x.shape[1] / 2, x.shape[0] / 2), 45, 1),
                           (x.shape[1], x.shape[0]))
```

Explanation

- $\mathbf{y}$: Stego image with embedded watermark.
- $\mathbf{y}_{\text{attack}}$: Noisy stego image with additive Gaussian noise.
- $\mathbf{y}_{\text{downsampled}}$: Stego image resized to half its dimensions.
- $\mathbf{y}_{\text{rotated}}$: Stego image rotated by 45 degrees.

Wiener Filter for Watermark Prediction

```
def wiener_filter(image, window_size, noise_variance):
    kernel = np.ones((window_size, window_size), np.float32) / (window_size
        ** 2)
    local_mean = cv2.filter2D(image, -1, kernel)
    local_var = cv2.filter2D(image ** 2, -1, kernel) - local_mean ** 2
    local_var = np.maximum(local_var, noise_variance)
    return local_mean + (image - local_mean) * (local_var / (local_var +
        noise_variance))

# Predict watermark from stego images
w_pred_y = y - wiener_filter(y, WIENER_WINDOW_SIZE, sigma_w ** 2)
w_pred_y_attack = y_attack - wiener_filter(y_attack, WIENER_WINDOW_SIZE,
    sigma_w ** 2)
```

Explanation

- The Wiener filter is used to predict the watermark $\hat{\mathbf{w}}_{\mathbf{y}}$ from the stego images.
- The predicted watermark is obtained by subtracting the Wiener-filtered image from the stego image.

Autocorrelation Function (ACF) Computation

```
def autocorrelation_fft(image):
    f = np.fft.fft2(image)
    f_conj = np.conj(f)
    return np.fft.fftshift(np.real(np.fft.ifft2(f * f_conj)))
```

Explanation

- Computes the 2D autocorrelation function (ACF) of an image using the Fourier Transform.
- Steps involved:
  - $\mathcal{F}(\mathbf{x})$: Apply 2D FFT to the image.
  - $\mathcal{F}(\mathbf{x}) \cdot \overline{\mathcal{F}(\mathbf{x})}$: Multiply with the conjugate.
  - Apply inverse FFT and shift for visualization.
- Returns the real part of the autocorrelation result.

$$\text{ACF}(\mathbf{x}) = \mathcal{F}^{-1}\left(\mathcal{F}(\mathbf{x}) \cdot \overline{\mathcal{F}(\mathbf{x})}\right)$$

Non-Maximum Suppression (NMS)

```
def non_maximum_suppression(image, size=31):
    max_filtered = maximum_filter(image, size=size)
    nms_result = (image == max_filtered) * image  # Retain only local maxima
    return nms_result
```

Explanation

- **Goal**: Retain only local maxima in the autocorrelation result to highlight distinct peaks.
- **Steps**:
  - Apply a maximum filter over a window of size 31×31.
  - Retain pixels where the value matches the maximum filter output.
- Output: A binary image where local maxima are retained.

Peak Expansion

```
def expand_peaks(image, expansion_size=3):
    expanded = np.zeros_like(image)
    peaks = np.argwhere(image > 0)
    for peak in peaks:
        x, y = peak
        expanded[max(0, x-expansion_size):min(image.shape[0], x+
            expansion_size+1),
                max(0, y-expansion_size):min(image.shape[1], y+
                    expansion_size+1)] = 1
    return expanded
```

Explanation

- Expands each detected peak in the NMS result by a window of size 3×3.
- This improves the visibility of detected peaks during visualization.
- The function iterates over all detected peaks and applies the expansion.

Autocorrelation and NMS

```
# Compute autocorrelation of watermarks
auto_w = autocorrelation_fft(w)
auto_w_pred_y = autocorrelation_fft(w_pred_y)

# Apply Non-Maximum Suppression (NMS)
nms_auto_w = expand_peaks(non_maximum_suppression(auto_w))
nms_auto_w_pred_y = expand_peaks(non_maximum_suppression(auto_w_pred_y))
```

Explanation

- The autocorrelation function (ACF) of the original and predicted watermarks is computed.
- NMS is applied to enhance the visibility of peaks in the autocorrelation results.

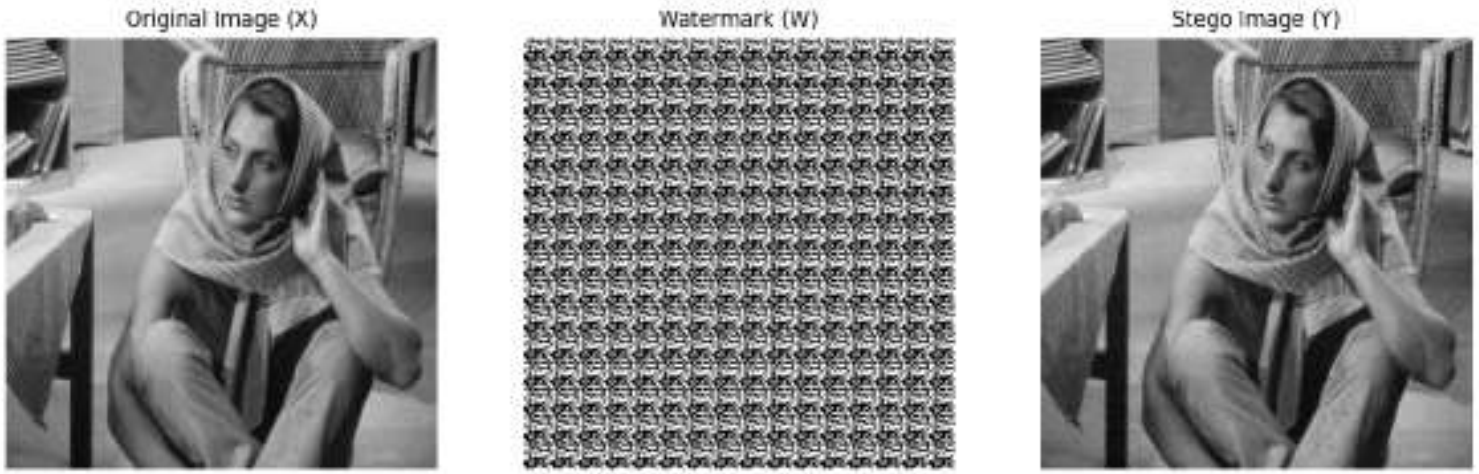
Applying Functions to All Variants of Y

- The steps for computing the predicted watermark, ACF, and NMS-enhanced ACF are applied to:
  1. $\mathbf{y}$ — Simple stego image.
  2. $\mathbf{y}_{\text{attack}}$ — Noisy stego image.
  3. $\mathbf{y}_{\text{rotated}}$ — Rotated stego image.
  4. $\mathbf{y}_{\text{downsampled}}$ — Downsampled stego image.
- Each output is visualized, showing:
  - The stego image.
  - The predicted watermark $\hat{\mathbf{w}}$.
  - The autocorrelation function (ACF).
  - The NMS-enhanced autocorrelation function (NMS ACF).

Note

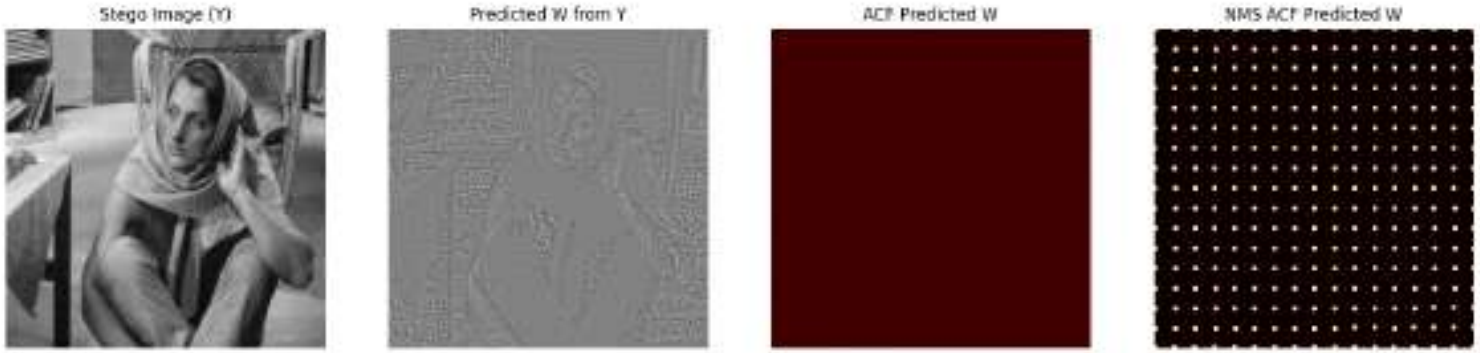
The next slides will show visualizations for all the variants of $\mathbf{y}$.

Demonstration of Self-Synchronized Digital Watermarking

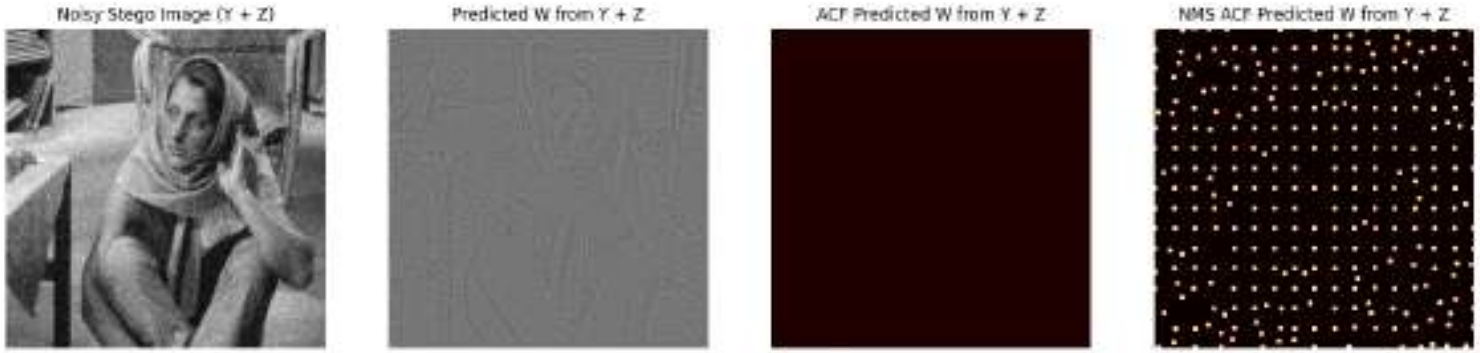


Demonstration of Self-Synchronized Digital Watermarking

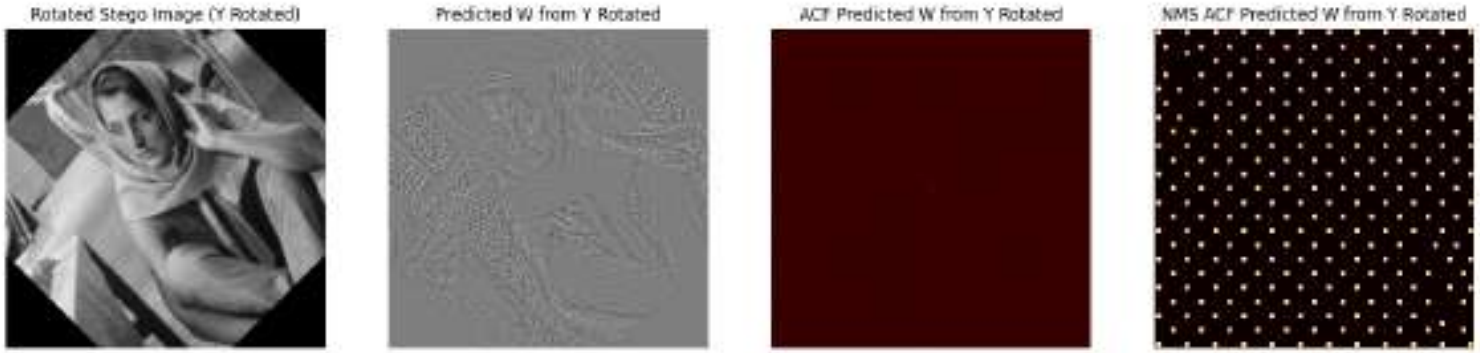
Demonstration of Self-Synchronized Digital Watermarking



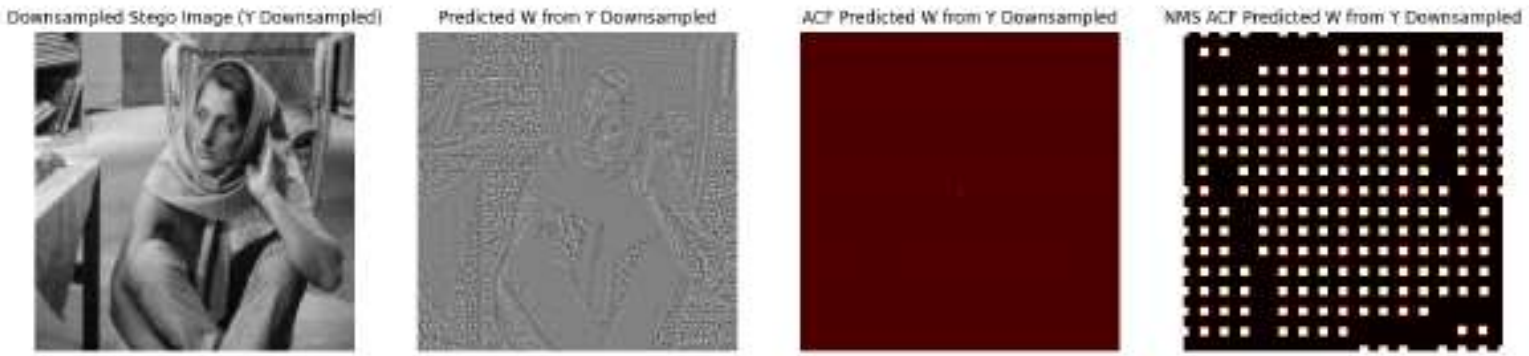
Demonstration of Self-Synchronized Digital Watermarking



Demonstration of Self-Synchronized Digital Watermarking



Demonstration of Self-Synchronized Digital Watermarking



3.5 Summary of Watermarking Process and Attacks

Key Steps in Watermark Embedding and Detection:

1. Generate a block watermark, upsample, and tile it.
2. Embed the watermark into the image using additive embedding.
3. Apply various attacks: noise, downsampling, and rotation.
4. Predict the watermark using Wiener filtering.
5. Use autocorrelation to detect and analyze the predicted watermark.

Conclusion: This process demonstrates the robustness of watermarking techniques under different types of attacks, highlighting the importance of Wiener filtering and autocorrelation in watermark detection.

3.5 Limitations of Self-Synchronization Techniques

Key Limitations

Self-synchronization techniques using repetitive watermarking are highly efficient but have certain limitations.

Limitations:

- **Security Leakage:** Attackers can estimate the watermark period via Wiener filtering, potentially regenerating and suppressing the watermark.
- **Size Constraint:** A minimum block repetition of 2×2 requires image sizes of at least 64×64 pixels for effective autocorrelation.
- **Perceptual Visibility:** Repetitive patterns may be detected by the human visual system, reducing stealthiness.

Conclusion: Despite these limitations, repetitive watermarking remains a powerful and robust approach.

3.5 Foundation Models for Invariant Representations

Key Concept:

Foundation Models for Invariant Representations

Foundation models (e.g., CLIP, DINO) trained on large datasets exhibit invariance to many geometric transformations, making their latent spaces suitable for robust watermarking.

Approach:

- Use a pre-trained foundation model to extract features from the watermarked image.
- Compare the latent representations to detect and decode the watermark.

Advantages:

- Robust to various real-world transformations.
- No explicit geometric synchronization required.

Limitations:

- High computational complexity.
- Vulnerable to adversarial attacks on the latent space.

Section 4

Types of Modulation Functions

3.6 Additive Watermarking (Spectrum Modulation)

Embedding Process:

$$\mathbf{y} = \mathbf{x} + \alpha \mathbf{w}, \quad \text{where} \quad \mathbf{w} = \sum_{n=1}^{L} m_n \mathbf{w}_n$$

- $\mathbf{x}$ is the original host signal, α is the scaling factor, and $\mathbf{w}_n$ are orthogonal carrier sequences corresponding to message bits m_n.
- $\mathbf{w}_n \sim \mathcal{N}(\mathbf{0}, \sigma_w^2 \mathbf{I}_M)$: Each carrier sequence is generated from a Gaussian distribution with zero mean and variance σ_w^2.

Detection Process:

$$\hat{m}_n = \text{sign}(\mathbf{y}^\top \mathbf{w}_n)$$

- The detection for each bit m_n is based on the inner product test between $\mathbf{y}$ and the corresponding carrier sequence $\mathbf{w}_n$.
- This approach faces interference from the host signal with variance σ_x^2, affecting detection accuracy.

Key Observations:

- Higher host signal variance σ_x^2 degrades performance by increasing interference.

3.6 Binary Quantization Index Modulation (QIM)

Embedding Process:

- Two quantizers $\mathcal{Q}_0$ and $\mathcal{Q}_1$ are defined for embedding bits 0 and 1, respectively.
- For each pixel x_i, the embedded value y_i is:

$$y_i = \mathcal{Q}_{m_i}(x_i)$$

where m_i is the bit to be embedded.

Detection Process:

- The detector compares the received pixel value y_i to the centroids of $\mathcal{Q}_0$ and $\mathcal{Q}_1$.
- The detected bit $\hat{m}_i$ is determined as:

$$\hat{m}_i = \begin{cases} 0 & \text{if } y_i \in \mathcal{Q}_0 \\ 1 & \text{if } y_i \in \mathcal{Q}_1 \end{cases}$$

Example:

- Given $x_i = 7.5$ and quantization step size $\Delta = 2$:

$$\mathcal{Q}_0 : \{0, 2, 4, 6, 8, \ldots\} \quad \text{(centroids: circles)}$$
$$\mathcal{Q}_1 : \{1, 3, 5, 7, 9, \ldots\} \quad \text{(centroids: crosses)}$$

3.6 Binary QIM: Constellation Diagram

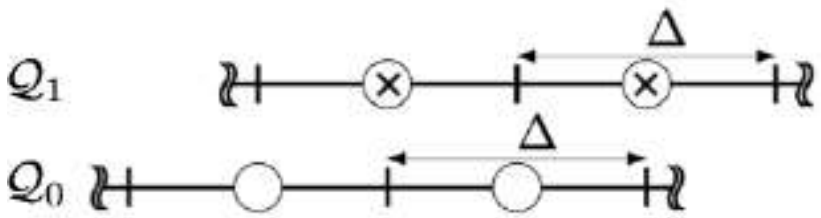


Figure Explanation:

- Circles represent centroids of quantizer $\mathcal{Q}_0$ (bit 0).
- Crosses represent centroids of quantizer $\mathcal{Q}_1$ (bit 1).
- The quantization index modulation ensures bit embedding by shifting pixel values to the nearest centroid.

3.6 Vector Quantization and Product Quantization

Vector Quantization (VQ):

- In vector quantization, the input vector is mapped to the nearest centroid in a predefined codebook.
- While effective, VQ is computationally expensive for large vectors.

Product Quantization (PQ):

- The vector is divided into B blocks, each quantized independently using C_K centroids:

$$\mathbf{c} = [c_{K_1}, c_{K_2}, \ldots, c_{K_B}]$$

- PQ reduces complexity while maintaining robustness, making it suitable for transform domain watermarking and perceptual hashing.

3.6 Comparative Analysis of Modulation Techniques

General Formulation:

$$\mathbf{y} = \phi(\mathbf{x}, \mathbf{m}, \mathbf{k})$$

- **Additive Watermarking:** Simple, flexible, but sensitive to host interference.
- **Binary QIM:** Robust to noise, but sensitive to scaling and volumetric operations.
- **Vector Quantization:** Effective but computationally expensive.
- **Product Quantization:** Balances complexity and robustness, suitable for large-scale systems.

Section 6

Key Management

3.7 Key Management Specificities in Digital Watermarking

Key management in digital watermarking presents unique characteristics compared to classical cryptographic systems.

- **Symmetric key usage:** The same key $\mathbf{k}$ is used for both embedding and extraction/decoding:

$$\mathbf{x} \xrightarrow{\mathbf{k}} \mathbf{y}, \quad \mathbf{y} \xrightarrow{\mathbf{k}} \hat{\mathbf{w}} \text{ or } \hat{\mathbf{m}}$$

- No asymmetric key pair (public/private) as in classical cryptography.
- Embedding and decoding require shared access to the same secret.

3.7 Key Reuse and Security Risks

Reusing the same key across multiple images introduces significant vulnerabilities.

- In detection-based watermarking, using the same key $\mathbf{k}$ for multiple images allows adversaries to learn or estimate $\mathbf{k}$.
- This weakens security by making watermark locations and patterns predictable.
- Good practice: use a distinct key $\mathbf{k}_i$ for each image $\mathbf{x}_i$ to prevent compromise.
- Analogous to key reuse issues in stream ciphers or one-time pad cryptosystems.

3.7 Key Decomposition in Multi-Bit Watermarking

In modern systems, multiple keys can serve different roles.

- **Position key:** Determines the embedding locations of the watermark $\mathbf{w}$ in $\mathbf{x}$.
- **Encoding key:** Controls how the multi-bit message $\mathbf{m} \in \{0,1\}^L$ is transformed into the watermark $\mathbf{w}$.
- **XOR key:** Adds randomness by XORing the message prior to embedding:

$$\mathbf{m}' = \mathbf{m} \oplus \mathbf{k}_{\text{xor}}$$

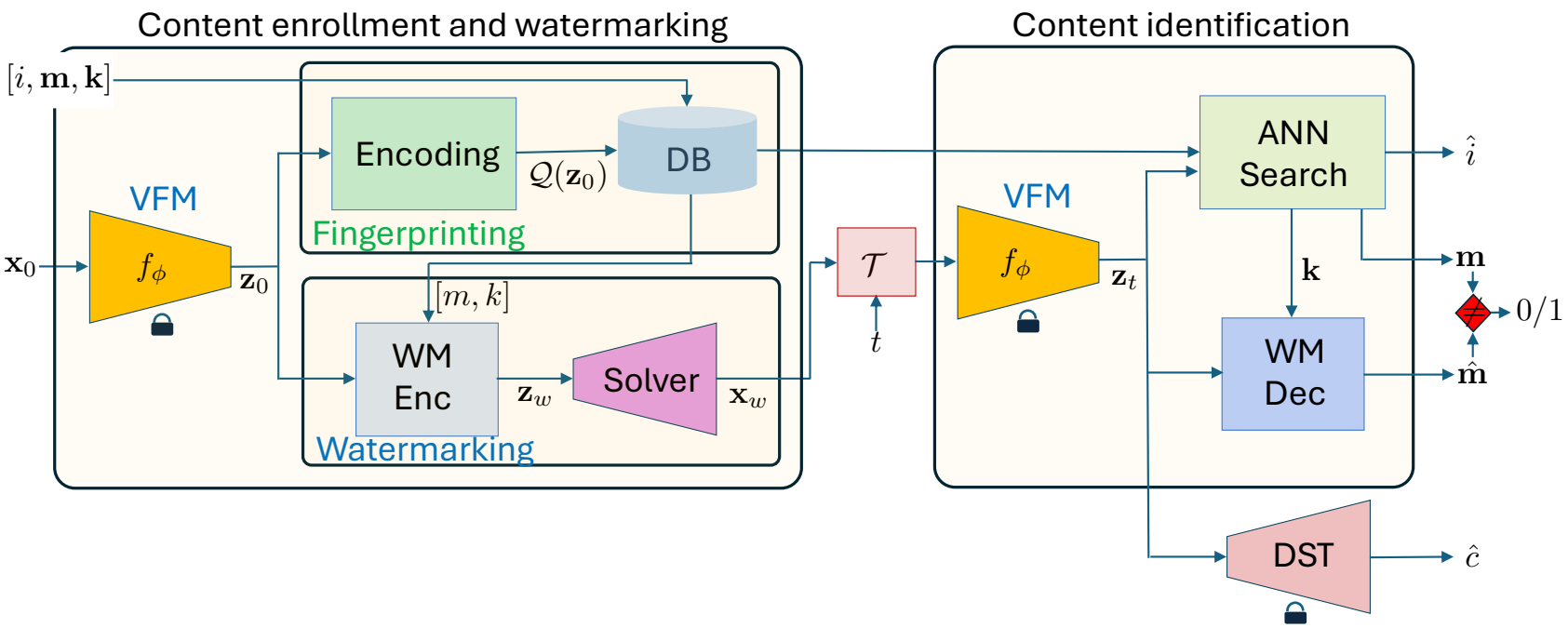
- Separation of keys improves flexibility and security of the embedding process.

3.7 Content-Aware Key Management

Advanced systems combine digital fingerprinting and sequential key search.

- In real-world applications, the decoder may not know which key/message was used.
- **Fingerprinting:** Identify the class of the input image to narrow the key/message candidates.
- **Key-message search:** Sequentially test $(\mathbf{k}, \mathbf{m})$ pairs until decoding succeeds.
- Efficient for systems where multiple watermarked images are processed under diverse key/message pairs.
- Enables scalable and practical deployment of watermark detectors without centralized key tracking.

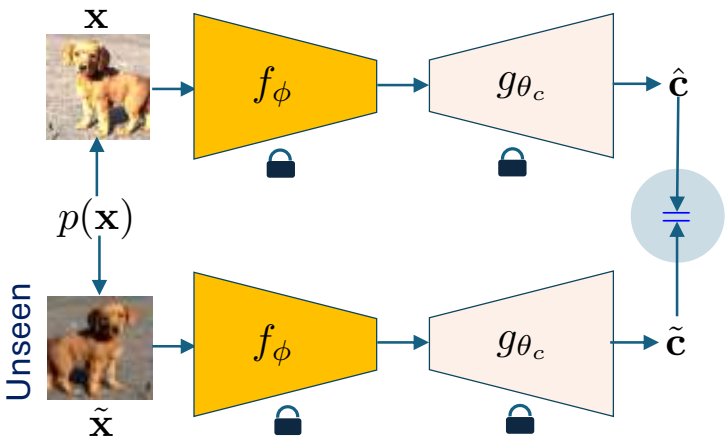
3.7 Content-Aware Key Management

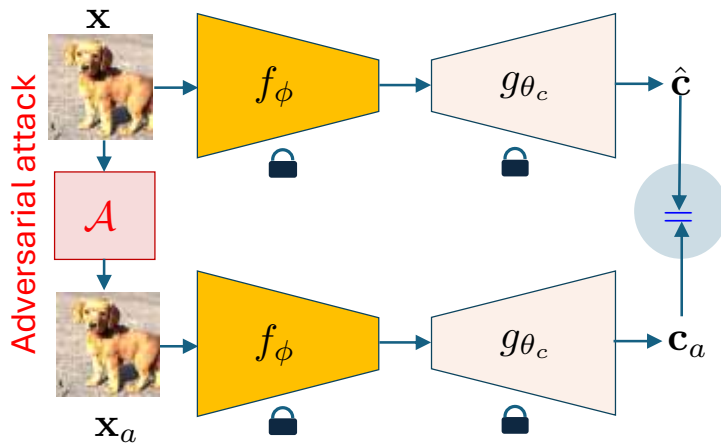


Section 7

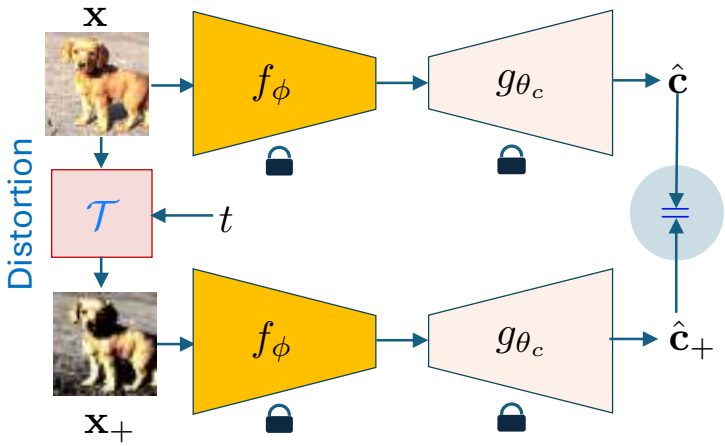
Robustness and Security (adversarial robustness)

Generalization, Robustness and Security





Generalization $\neq$ **Robustness** $\neq$ **Security**



Adversarial Robustness and Security

Definition: Adversarial robustness refers to cases where an **attacker uses partial knowledge** of the method (e.g., improper key or message management). This enables crafting **adversarial examples** that degrade performance.

Scope:

1. **Generic Image-Based Attacks:** Exploit weaknesses in method design using generic adversarial images.
2. **Foundation Model-Based Watermarking:** Analyze adversarial robustness in watermarking with **SSL or foundation models**.

Takeaway: Adversarial robustness is vital to defend against adaptive attacks targeting weaknesses in multimedia security.

Generic image-based attacks

Attacks based on improper use of keys and messages

Applied to any watermarking technology

Attacks on Watermarking

We consider two types of attacks:

1. **Removal Attacks:**
   - Removal by denoising and noise addition
   - Removal by averaging
   - Removal by collusion
2. **Copy Attack:**
   - Copying watermark from one image to another

Goal: To exploit knowledge of watermarking methods to compromise the embedded information.

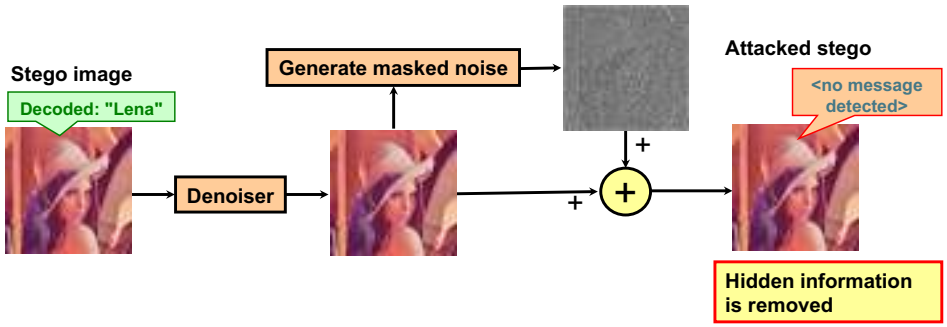
Removal by Denoising and Noise Addition Attack

Description:

- The watermark is embedded into the image as $\mathbf{y} = \mathbf{x} + \mathbf{w}$.
- The attacker aims to estimate $\hat{\mathbf{x}}$ (a denoised version of $\mathbf{y}$) while adding controlled noise ϵ.
- The denoising process uses filters such as:
  - Basic denoising filters or DNN-based denoisers.

Goal of the Attack:

- Remove the watermark $\mathbf{w}$, treating it as noise.
- Use denoising techniques to approximate $\hat{\mathbf{x}} \approx \mathbf{x}$.
- Further complicate detection by adding noise ϵ, resulting in $\mathbf{y}_a = \hat{\mathbf{x}} + \epsilon$.



https://cvml.unige.ch/publications/postscript/99/VoloshynovskiyPereiraPun_eww99.pdf

3.8 Removal by Certified!-Inspired Attacks

Classical Removal:

- Watermark embedded as $\mathbf{y} = \mathbf{x} + \mathbf{w}$.
- Attacker estimates a denoised image $\hat{\mathbf{x}}$ from $\mathbf{y}$, treating $\mathbf{w}$ as noise.
- Adds adversarial noise $\boldsymbol{\epsilon}$ to produce $\mathbf{y}_a = \hat{\mathbf{x}} + \boldsymbol{\epsilon}$.
- Denoisers can include traditional filters or DNNs.

Modern Extension: Certified! Method for Adversarial Perturbation Removal

- The **Certified!** method is designed for defending against adversarial perturbations.
- Input: image $\mathbf{x}$ and adversarial perturbation $\boldsymbol{\epsilon}$.
- Applies a randomized generative module D_θ to produce an image:

$$\hat{\mathbf{x}} = \mathrm{D}_\theta(\mathbf{x} + \boldsymbol{\epsilon} + \boldsymbol{\eta})$$

  where $\boldsymbol{\eta}$ denotes random noise infused into the generative process.
- Key property: non-differentiability and stochasticity make it difficult to reverse-engineer.

3.8 Removal by Certified!-Inspired Attacks

Relevance to Watermarking:

- Certified!'s randomized denoising process can be repurposed to remove $\mathbf{w}$, treating it similarly to an adversarial perturbation.
- Highlights the convergence between watermark removal and adversarial purification strategies.

Remark:

- Under a strong noise, the generative process might produce completely new images that might differ from the watermarked one.
- Certified! does not preserve PSNR like the previous attack.

https://arxiv.org/pdf/2206.10550

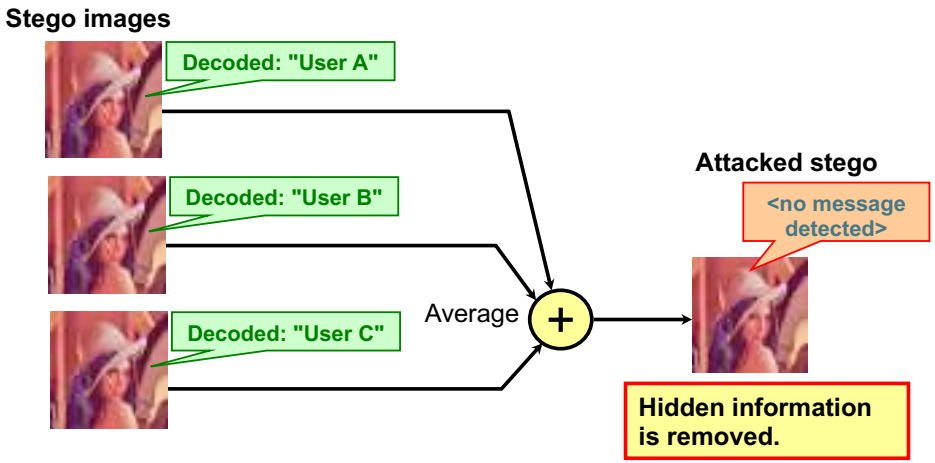
Removal by Averaging Attack

Description:

- Different watermark versions are embedded in the same image but with distinct messages.
- Variations can occur due to different keys or different multi-bit/zero-bit watermarks used for unique customers.

Goal of the Attack:

- Combine all versions of the watermarked images.
- Since the watermark is a micro-modulation signal (invisible), it cancels out upon averaging.
- The remaining content is preserved.



Removal by Collusion Attack

Description:

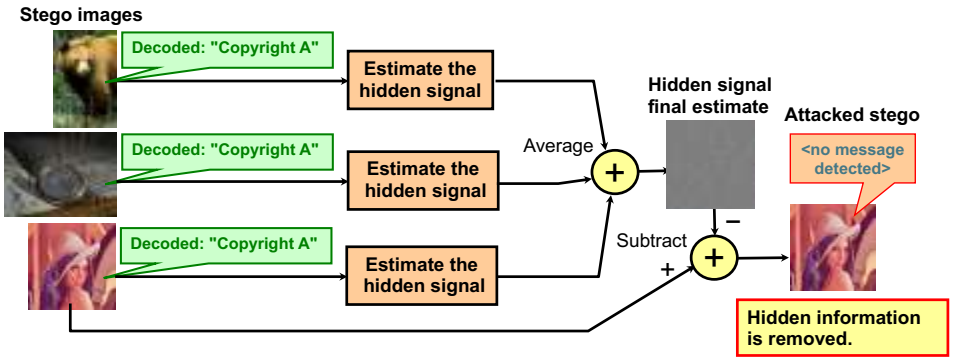
- Multiple watermarked images contain the **same embedded message**.
- Variations arise from different keys or multi-bit/zero-bit watermarks generated from the same key.

Goal of the Attack:

- Estimate the watermark signal from each image, without knowing the key.
- Align the estimated watermark with each image and subtract it.
- This removes the watermark while preserving the content.

Alternative Method:

- If the watermarking scheme is periodic, the attacker can estimate the autocorrelation function.
- Using autocorrelation, the attacker aggregates and regenerates the watermark, then subtracts it.



Copy Attack

Description:

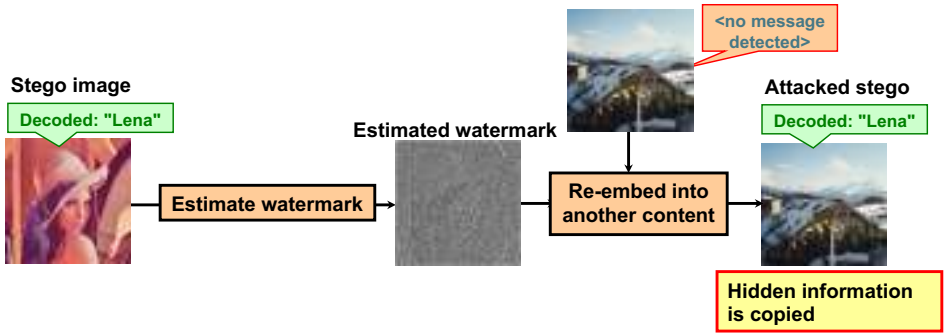
- The attacker estimates the watermark from a watermarked image.
- The extracted watermark is copied directly into a new image that was never watermarked before.

Applications:

- Multi-bit watermarking
- Zero-bit watermarking

Goal of the Attack:

- Increase probability of false acceptance.
- https://en.wikipedia.org/wiki/Copy_attack



Attacks against ML based watermarking

The knowledge of Foundation Model is assumed

Applied to AE and SSL-based watermarking technologies

Foundation Model-Based Watermarking

Overview:

- Utilizes large pre-trained neural networks, known as Vision Foundation Models (VFMs), for embedding watermarks.
- Embedding is performed in the latent space of these models using adversarial techniques.

Advantages:

- Robustness across varying image resolutions.
- Adjustable trade-off between robustness and quality.

Notations and Definitions

Notations:

- $\mathbf{x}$: Original image.
- $\mathbf{y}$: Watermarked image.
- $\mathbf{z}$: Latent representation in the VFM.
- $\mathbf{y}_a$: Attacked image.
- f_ϕ: Vision Foundation Model (VFM) mapping images to latent space.
- $\mathbf{w}$: Secret carrier vector generated from a secret key.

Detection Region:

- Defined as $D_k := \{\mathbf{z} \in \mathbb{R}^d : |\mathbf{z}^T \mathbf{w}| > \|\mathbf{z}\| \cos(\gamma)\}$.
- Angle γ is determined by the targeted false acceptance rate P_{fa}.

https://arxiv.org/pdf/2112.09581
WATERMARKING IMAGES IN SELF-SUPERVISED LATENT SPACES Pierre Fernandez, Alexandre Sablayrolles, Teddy Furon, Hervé Jégou, Matthijis Douze

Zero-Bit Watermarking in Latent Space

- **Objective:** Detect if an image $\mathbf{x}$ contains a zero-bit watermark.
- Let $\mathbf{w} \in \mathbb{R}^d$, with $\|\mathbf{w}\| = 1$, be a secret carrier generated from a secret key $\mathbf{k}$.
- Detection is performed in the latent space: $\mathbf{z} = f_\phi(\mathbf{x})$.

Detection region (dual hypercone):

$$\mathcal{D}_\mathbf{k} := \left\{ \mathbf{z} \in \mathbb{R}^d : \left| \mathbf{z}^\top \mathbf{w} \right| > \|\mathbf{z}\| \cos(\gamma) \right\}$$

False acceptance rate P_{fa}^t:

$$P_{\text{fa}}^t := \mathbb{P}\left[f_\phi(\mathbf{x}) \in \mathcal{D}_\mathbf{K} \mid \mathbf{K} \sim \mathcal{U} \right] = 1 - I_{\cos^2(\gamma)}\left(\tfrac{1}{2}, \tfrac{d-1}{2} \right)$$

Detection loss:

$$\mathcal{L}_\mathcal{Z}^I(\mathbf{z}, \mathbf{w}) = \|\mathbf{z}\|^2 \cos^2(\theta) - (\mathbf{z}^\top \mathbf{w})^2$$

Multi-Bit Watermarking in Latent Space

- **Objective:** Decode a multi-bit message $\mathbf{m} \in \{-1, 1\}^{\ell}$.
- Use an orthogonal family of carriers $\{\mathbf{w}_1, \ldots, \mathbf{w}_{\ell}\} \subset \mathbb{R}^d$ derived from $\mathbf{k}$.

Decoding rule:

$$\hat{\mathbf{m}} = \big(\text{sign}(\mathbf{z}^{\top}\mathbf{w}_1), \ldots, \text{sign}(\mathbf{z}^{\top}\mathbf{w}_{\ell})\big)$$

Margin-based decoding loss:

$$\mathcal{L}_{\mathcal{Z}}^{II}(\mathbf{z}, \mathbf{m}) = \frac{1}{\ell} \sum_{i=1}^{\ell} \max\left(0, \mu - (\mathbf{z}^{\top}\mathbf{w}_i) \cdot m_i\right)$$

Watermark Embedding and Robustness (I)

- **Objective:** Modify $\mathbf{x}_0 \in \mathcal{X}$ into $\mathbf{y} \in \mathcal{X}$ such that $f_\phi(t(\mathbf{y}))$ lies deep inside the decoding region.
- Use typical visual transformations $t \in \mathcal{T}$ to improve robustness (e.g., crop, blur, rotation).
- The embedding minimizes a latent loss and controls perceptual distortion.

Combined loss function:

$$\mathcal{L}_{\mathcal{W}}(\mathbf{x}, \mathbf{x}_0, t) := \lambda \mathcal{L}_{\mathcal{Z}}(f_\phi(t(\mathbf{x}))) + \mathcal{L}_{\mathcal{X}}(\mathbf{x}, \mathbf{x}_0)$$

Latent loss: chosen as $\mathcal{L}^{I}_{\mathcal{Z}}$ or $\mathcal{L}^{II}_{\mathcal{Z}}$
Image distortion loss: $\mathcal{L}_{\mathcal{X}}$ enforces visual similarity (e.g., PSNR or SSIM-based).

Watermark Embedding and Robustness (II)

- Embedding is cast as an adversarial optimization under transformation distribution.
- Use Expectation over Transformation (EoT) to find $\mathbf{y}$ robust to all $t \in \mathcal{T}$.

EoT-based optimization:

$$\mathbf{y} := \arg \min_{\mathbf{x} \in C(\mathbf{x}_0)} \mathbb{E}_{T \sim \mathcal{U}(\mathcal{T})} \left[\mathcal{L}_{\mathcal{W}}(\mathbf{x}, \mathbf{x}_0, T) \right]$$

Admissible set $C(\mathbf{x}_0)$**:** Defined via constrained perturbation $\delta = \mathbf{x} - \mathbf{x}_0$:

1. Attenuated by SSIM heatmap (to hide in less perceptually sensitive regions)
2. Rescaled to match a target PSNR

Copy Attack: Objective

- **Goal:** Falsely induce detection on a non-watermarked image $\mathbf{x}_t$ by copying the watermark from a watermarked image $\mathbf{x}_w$.
- **Assumption:** The attacker has access to $\mathbf{x}_w$ but not to the message $\mathbf{m}$ or key $\mathbf{k}$.
- **Approach:** Transfer latent signature of $\mathbf{x}_w$ onto $\mathbf{x}_t$ while preserving visual similarity to $\mathbf{x}_t$.
- Generalizes traditional copy attacks without requiring linear embedding.

Overview

We optimize an attacked image $\mathbf{x}_a$ such that:

- $\mathbf{x}_a \approx \mathbf{x}_t$ in pixel space
- $f_\phi(\mathbf{x}_a) \approx f_\phi(\mathbf{x}_w)$ in latent space

Copy Attack: Objective

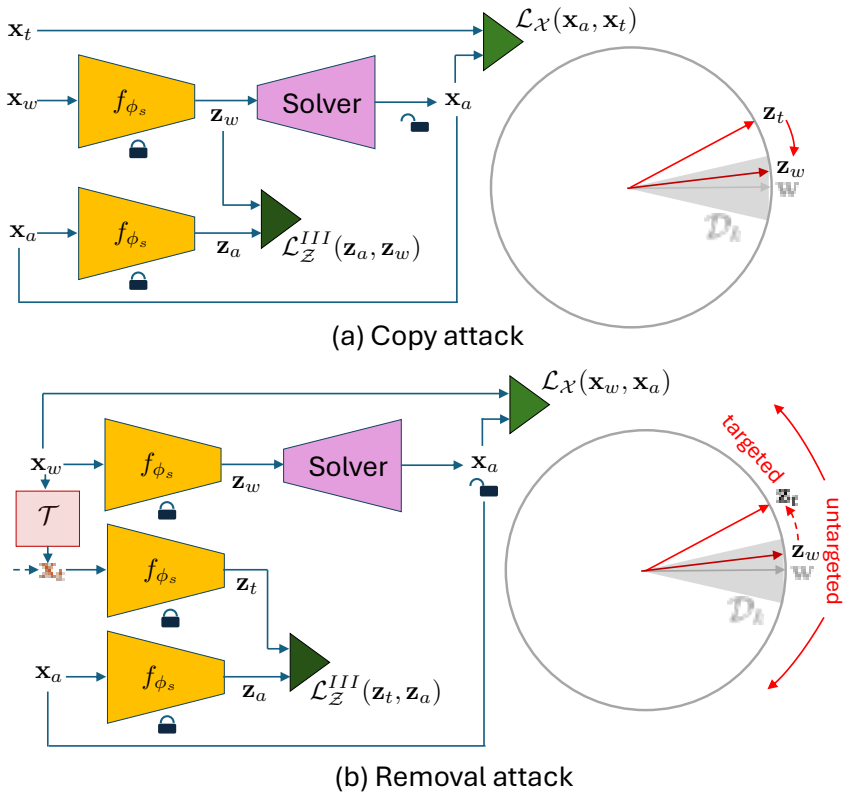


Figure: Generalized diagram explaining the proposed (a) copy and (b) untargted and targeted removal attacks (on the example of zero-bit watermarking in the latent space). The secret carrier **w** and the decision region $\mathcal{D}_k$ (show in gray) are unknown for the attacker.

Copy Attack: Optimization Objective

- **Pixel loss:** Keeps $\mathbf{x}_a$ visually close to $\mathbf{x}_t$
- **Latent similarity loss:** Forces $f_\phi(\mathbf{x}_a)$ to imitate $f_\phi(\mathbf{x}_w)$

Total loss:

$$\mathcal{L}_{\mathcal{A}}^{\mathrm{C}}(\mathbf{x}_a, \mathbf{x}_w, \mathbf{x}_t) = \mathcal{L}_{\mathcal{X}}(\mathbf{x}_a, \mathbf{x}_t) + \lambda \mathcal{L}_{\mathcal{Z}}^{III}(\mathbf{z}_a, \mathbf{z}_w)$$

Latent similarity:

$$\mathcal{L}_{\mathcal{Z}}^{III}(\mathbf{z}_a, \mathbf{z}_w) = -\frac{\mathbf{z}_a^\top \mathbf{z}_w}{\|\mathbf{z}_a\| \|\mathbf{z}_w\|}$$

- Optimized using gradient descent over N iterations.
- $\delta_{at} = \mathbf{x}_a - \mathbf{x}_t$ is normalized via SSIM masking and rescaled to satisfy PSNR_a.

Algorithm 1 — Copy Attack

Algorithm 1 Copy Attack

Require: Watermarked image $\mathbf{x}_w$, target image $\mathbf{x}_t$, feature extractor f_ϕ

$\mathbf{z}_w \leftarrow f_\phi(\mathbf{x}_w)$ // extract features from $\mathbf{x}_w$

$\mathbf{x}_a \leftarrow \mathbf{x}_t$ // initialize attack image

for $t = 0$ **to** $N - 1$ **do**

  $\mathbf{x}_a \leftarrow \text{Proj}_{\delta_{at}}(\mathbf{x}_a)$ // apply constraints

  $\mathbf{z}_a \leftarrow f_\phi(\mathbf{x}_a)$ // extract latent features

  $\mathbf{x}_a \leftarrow \mathbf{x}_a + \eta \cdot \text{Adam}(\nabla_{\mathbf{x}_a} \mathcal{L}_{\mathcal{A}}^{\mathcal{C}}(\mathbf{x}_a, \mathbf{x}_w, \mathbf{x}_t))$ // gradient update

end for

$\mathbf{x}_a \leftarrow \text{Proj}_{\delta_{at}}(\mathbf{x}_a)$ // final constraint + rounding

return Attacked image $\mathbf{x}_a$

Copy Attack: Practical Strategies

- **Output:** Attacked image $\mathbf{x}_a$ that imitates the watermark in $\mathbf{x}_w$ while resembling $\mathbf{x}_t$.
- **Transfer Strategies:**
  1. Use any non-watermarked $\mathbf{x}_t$
  2. Use a degraded version of $\mathbf{x}_w$ for which detection fails (and restore via attack)
  3. Use a different watermark carrier as latent target
- **Robust:** Works across watermark domains; does not rely on embedding linearity.

Watermark Removal: Untargeted Attack

- **Goal:** Suppress watermark detection or decoding without knowing $\mathbf{k}$ or $\mathbf{m}$.
- **Input:** Watermarked image $\mathbf{x}_w$
- **Output:** Attacked image $\mathbf{x}_a$ such that:
  - $\mathbf{x}_a \approx \mathbf{x}_w$
  - $f_\phi(\mathbf{x}_a)$ diverges from $f_\phi(\mathbf{x}_w)$

Untargeted removal loss:

$$\mathcal{L}_{\mathcal{A}}^{\mathrm{R-U}}(\mathbf{x}_w, \mathbf{x}_a) = \mathcal{L}_{\mathcal{X}}(\mathbf{x}_w, \mathbf{x}_a) - \lambda \mathcal{L}_{\mathcal{Z}}^{IV}(\mathbf{z}_w, \mathbf{z}_a)$$

Latent divergence loss:

$$\mathcal{L}_{\mathcal{Z}}^{IV}(\mathbf{z}_w, \mathbf{z}_a) = \frac{(\mathbf{z}_a^\top \mathbf{z}_w)^2}{\|\mathbf{z}_a\| \|\mathbf{z}_w\|}$$

Watermark Removal: Targeted Attack and Algorithm

Targeted attack loss:

$$\mathcal{L}_{\mathcal{A}}^{\mathrm{R-T}}(\mathbf{x}_w, \mathbf{x}_t, \mathbf{x}_a) = \mathcal{L}_{\mathcal{X}}(\mathbf{x}_w, \mathbf{x}_a) + \lambda \mathcal{L}_{\mathcal{Z}}^{III}(\mathbf{z}_t, \mathbf{z}_a)$$

Algorithm 2 — Watermark Removal Attack

Algorithm 2 Watermark Removal Attack

Require: Watermarked image $\mathbf{x}_w$, target image $\mathbf{x}_t$, feature extractor f_ϕ, $\texttt{attack\_type} \in \{\texttt{targeted}, \texttt{untargeted}\}$

$\mathbf{z}_t \leftarrow f_\phi(\mathbf{x}_t)$ // extract features from target

$\mathbf{x}_a \leftarrow \mathbf{x}_w$ // initialize attack image

for $t = 0$ **to** $N - 1$ **do**

  $\mathbf{x}_a \leftarrow \text{Proj}_{\delta_{aw}}(\mathbf{x}_a)$ // apply constraints

  $\mathbf{z}_a \leftarrow f_\phi(\mathbf{x}_a)$ // compute latent representation

  if $\texttt{attack\_type} == \text{"untargeted"}$ **then**

    $\mathbf{x}_a \leftarrow \mathbf{x}_a + \eta \cdot \text{Adam}(\nabla_{\mathbf{x}_a} \mathcal{L}_{\mathcal{A}}^{\text{R-U}}(\mathbf{x}_w, \mathbf{x}_a))$ // untargeted update

  else if $\texttt{attack\_type} == \text{"targeted"}$ **then**

    $\mathbf{x}_a \leftarrow \mathbf{x}_a + \eta \cdot \text{Adam}(\nabla_{\mathbf{x}_a} \mathcal{L}_{\mathcal{A}}^{\text{R-T}}(\mathbf{x}_w, \mathbf{x}_t, \mathbf{x}_a))$ // targeted update

  end if

end for

$\mathbf{x}_a \leftarrow \text{Proj}_{\delta_{aw}}(\mathbf{x}_a)$ // final constraint + rounding

return Attacked image $\mathbf{x}_a$

Experimental Evaluation

Evaluation Setup:

- Tested on a large-scale image dataset, with images resized to a fixed resolution.
- Embedding strength varied to assess robustness under different attack scenarios.

Metrics:

- Detection Accuracy (%): Proportion of correctly detected watermarks.
- PSNR (Peak Signal-to-Noise Ratio): Measures perceptual similarity.
- SSIM (Structural Similarity Index): Evaluates image similarity in terms of luminance, contrast, and structure.

Results and Analysis

Key Findings:

- **Copy Attack:**
  - High success rate in transferring watermarks to target images.
- Perceptual quality of attacked images remained high (PSNR $>40dB$).
- **Removal Attack:**
  - Successfully removed watermarks with minimal perceptual distortion.
  - Untargeted removal was more effective than targeted removal in most cases.

Conclusion:

- Foundation model-based watermarking systems are susceptible to copy and removal attacks.
- Further research is required to enhance the robustness and security of these systems.